

Bilgisayar Mühendisliğinde Özel Konular

Digital Image Processing

Digital Image Processing

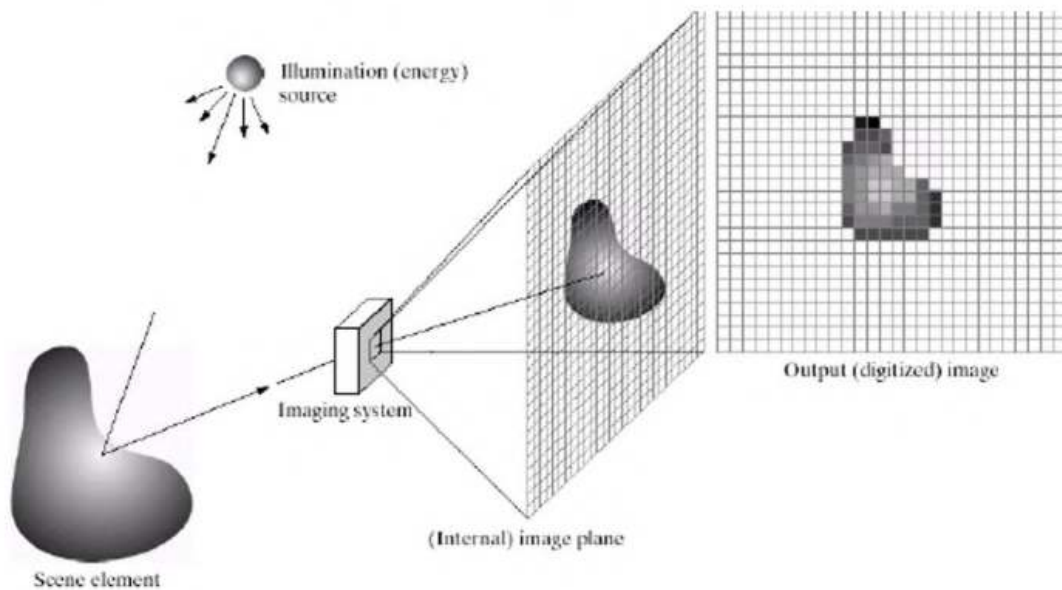
- The process of transforming an image into a digital form and performing certain operations to get some useful information from it.
 - The image processing system usually treats all images as 2D signals when applying certain predetermined signal processing methods.
-
- Low-level : input, output are images
 - Primitive operations such as image preprocessing to reduce noise, contrast enhancement, and image sharpening
 - Mid-level : inputs may be images, outputs are attributes extracted from those images
 - Segmentation
 - Description of objects
 - Classification of individual objects
 - High-level :
 - Image analysis

Görüntü işleme, bir görüntüyü dijital forma dönüştürüp ondan yararlı bilgiler elde etme sürecidir ve sistem görüntüleri **2B sinyaller** olarak ele alır. İşlemler üç ana seviyeye ayrılır:

- **Düşük Seviye (Low-level):** Girdi ve çıktı görüntüdür. Gürültü azaltma, kontrast artırma ve keskinleştirme gibi ön işlemler buradadır.
- **Orta Seviye (Mid-level):** Girdi görüntüdür ama çıktı özniteliklerdir. Segmentasyon, nesne tanımı ve sınıflandırma bu aşamadır.
- **Yüksek Seviye (High-level):** Görüntü analizi ve anlamlandırma sürecidir.

What is Digital Image ?

- Two-or-three dimensional collection of light values (or gray values)
- discrete and defined on a discrete grid
- each square in the grid contains a single pixel.

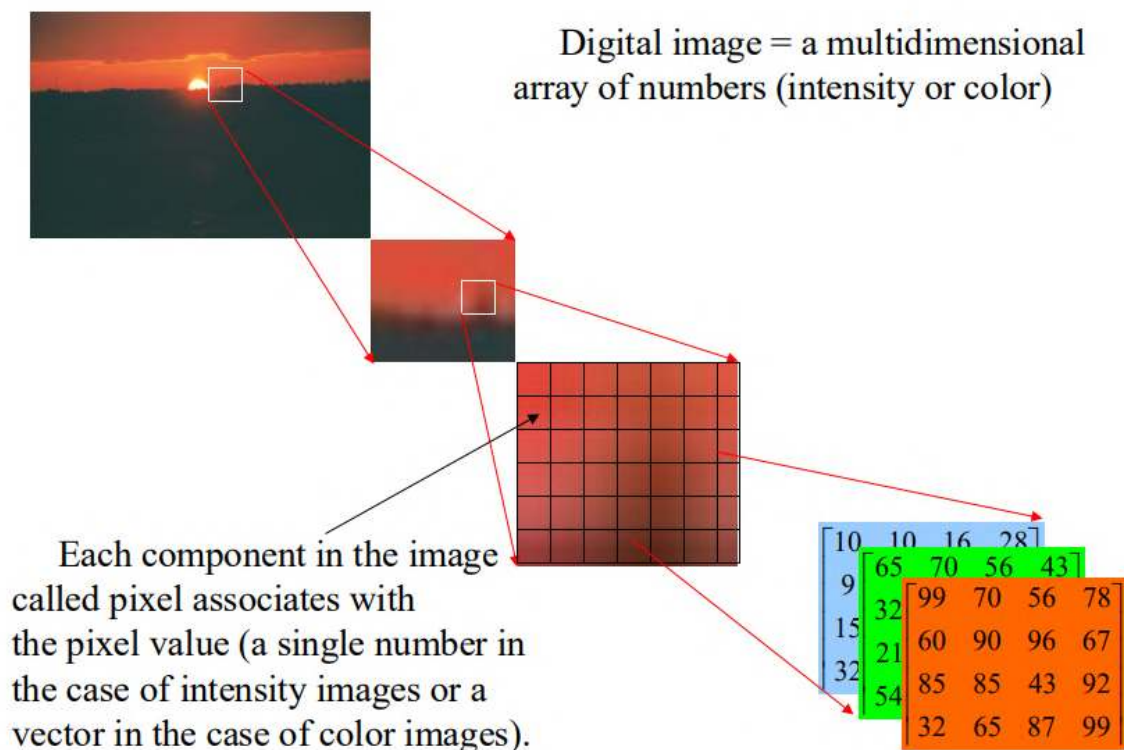
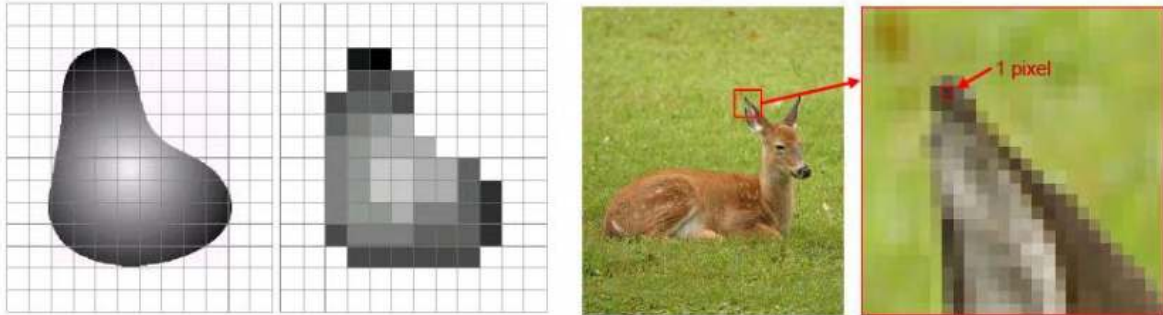


$$f(x, y) = \begin{bmatrix} f(0,0) & f(0,1) & \dots & f(0,M-1) \\ f(1,0) & \dots & \dots & f(1,M-1) \\ \dots & \dots & \dots & \dots \\ f(N-1,0) & f(N-1,1) & \dots & f(N-1,M-1) \end{bmatrix}$$

↓
↓

Digital Image
 Image Elements (Pixels)

- A **pixel** is the smallest addressable element in a digital image
- A pixel is the **colour** or **intensity** of light that appears at a given point in our image.

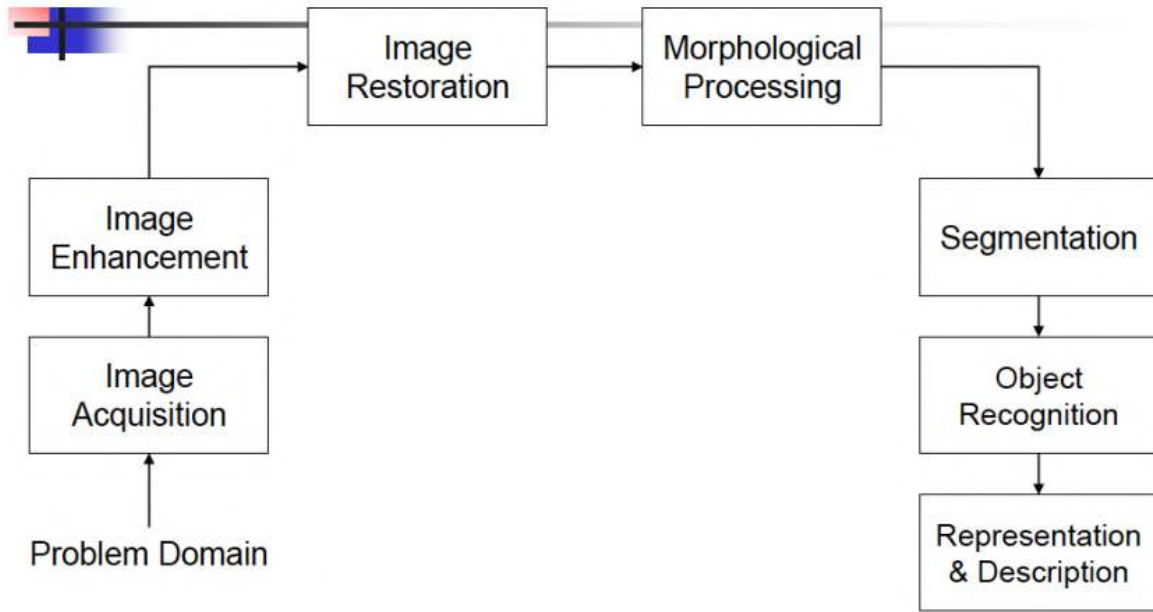


Analog to Digital Conversion →

Görüntüyü karelere bölmek **sampling**, her kareye bir sayı vermek ise **quantization** işlemidir.

Quantization azaldıkça (gri seviyesi azalır) detaylar kaybolur.

Fundemantal Steps



- **Problem Domain (Problem Alanı):** Çözülmesi gereken spesifik uygulama veya ele alınan ortamdır (örneğin; plaka okuma, tıbbi teşhis veya defolu ürün tespiti).
- **Image Acquisition (Görüntü Edinme):** Sensörler veya kameralar yardımıyla fiziksel ortamdaki görüntünün dijital formata dönüştürülerek sisteme alınması sürecidir.
- **Image Enhancement (Görüntü Belirginleştirme/İyileştirme):** Görüntünün kalitesini, kontrastını veya detaylarını insan gözü ya da sonraki adımlar için daha belirgin hale getirme işlemidir (Örn: Parlaklık ayarı, kontrast artırma). Bu adım genellikle öznedir.
- **Image Restoration (Görüntü Onarımı):** Görüntüdeki bozulmaları, gürültüleri (noise) veya bulanıklıkları matematiksel ve nesnel modeller kullanarak temizleme ve görüntüyü orijinal haline döndürme işlemidir.
- **Morphological Processing (Morfolojik İşleme):** Görüntüdeki nesnelerin yapısal ve şekilsel özelliklerini (biçimlerini) düzenlemek, ince bağları koparmak veya boşlukları doldurmak için kullanılan matematiksel küme işlemleridir (Örn: Aşındırma, yayma).

- **Segmentation (Bölütleme/Segmentasyon):** Görüntüyü anlamlı parçalara, nesnelere veya arka plana ayırma işlemidir. Genellikle nesne tespitinin ilk ve en kritik adımıdır (Örn: Kenar bulma, eşikleme).
- **Object Recognition (Nesne Tanıma):** Bölütlenen parçaların veya nesnelerin ne olduğunu (örneğin; "araba", "insan", "A harfi") önceden tanımlanmış etiketlere göre teşhis etme sürecidir.
- **Representation & Description (Temsil ve Tanımlama):** Tanınan nesnelerin bilgisayar ortamında daha efektif işlenebilmesi için matematiksel veya mantıksal olarak ifade edilmesidir. Bölütlenmiş bir bölgenin sınırları (temsil) ya da alan, bölge genişliği gibi ayırt edici özellikleri (tanımlama) bu adımda netleştirilir.

Görüntü işlemedeki temel matematiksel işlemler

Point Operations

- $f(x,y)$ is a color value of a pixel

$$g(x, y) = T[f(x, y)]$$

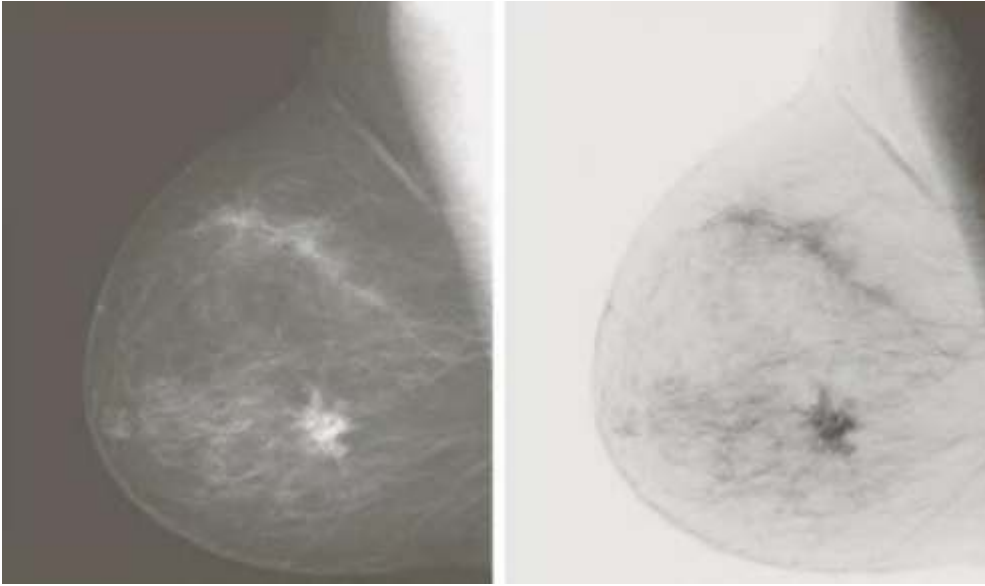
- Apply **T** transformation to each pixel

$$s = T(r)$$

Point operations changes a pixel's intensity value according to some function →

pixellerin her birindeki yoğunluk değerleri değiştirilerek parlaklık, kontrast gibi değerlerle oynanabilir

Image Negative



$$s = T(r)$$

$$s = L - 1 - r$$

Intensity Level : $[0, L-1]$

L: toplam gri seviye sayısı (standart 8-bit görüntü için 256 (0-255))
r giriş pikseli, s çıkış pikseli

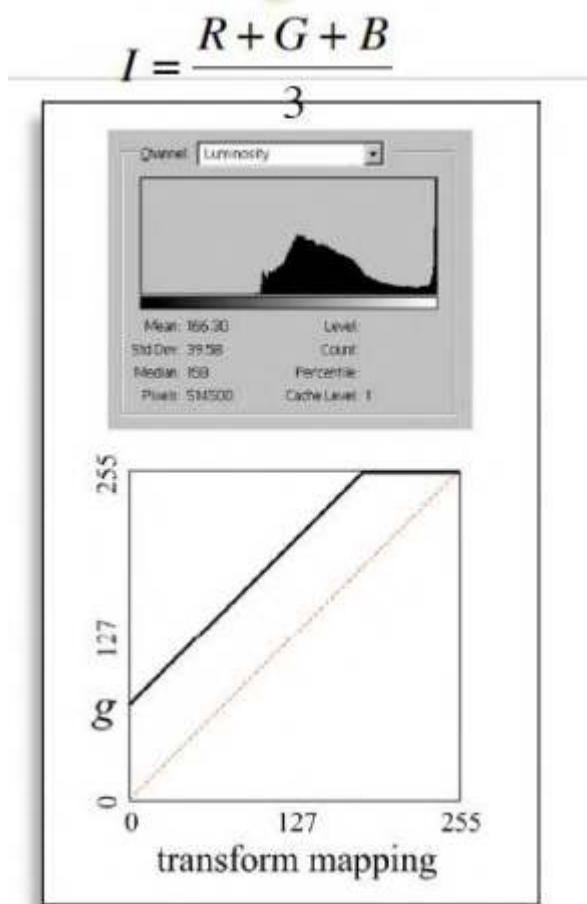
Increase Brightness



$$J_k(r,c) = \begin{cases} I_k(r,c) + g, & \text{if } I_k(r,c) + g < 256 \\ 255, & \text{if } I_k(r,c) + g > 255 \end{cases}$$

$g \geq 0$ and $k \in \{1, 2, 3\}$ is the band index.

- Piksele sabit bir g değeri eklenir.
- **Eşik Değeri:** Eğer sonuç 255'ten büyük çıkarsa değer 255'e sabitlenir (clipping).
- **Histogram Etkisi:** Histogram grafiği olduğu gibi **sağa** (yüksek değerlere doğru) kayar.

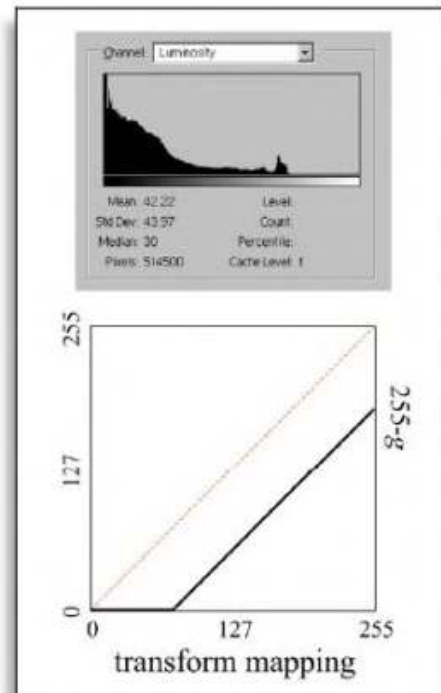


Decrease Brightness



$$J_k(r,c) = \begin{cases} 0, & \text{if } I_k(r,c) - g < 0 \\ I_k(r,c) - g, & \text{if } I_k(r,c) - g \geq 0 \end{cases}$$

$g \geq 0$ and $k \in \{1, 2, 3\}$ is the band index.



Eşik Değeri: Eğer sonuç 0'dan küçük çıkarsa değer 0'a sabitlenir.
Histogram sola kayar

gri seviyenin dışına taşınca ilgi kaybı oluşuyor, nasıl engellenebilir? -
>

1. Clipping (→ eşik değeri aşan değerleri max veya min değere sabitlemek)

2. Normalizasyon

Min-max normalizasyonu çözüm örneklerinden biri

$$s = \frac{r - r_{min}}{r_{max} - r_{min}} \times (L - 1)$$

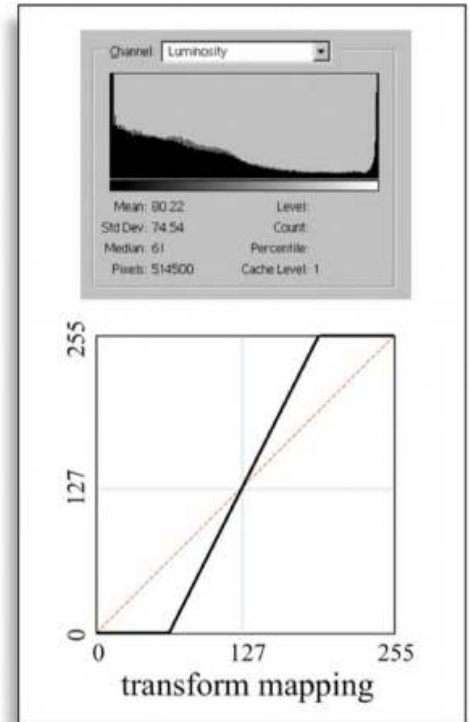
- $r - r_{min}$: Görüntüdeki en küçük değeri 0'a çeker (Kaydırma).
- $\frac{1}{r_{max} - r_{min}}$: Tüm değerleri $[0, 1]$ arasına sıkıştırır (Ölçekleme).
- $\times (L - 1)$: Sonucu tekrar $[0, 255]$ (veya kullanılan bit derinliği neyse ona) yayar.

Increase Contrast

Görüntüde açık ve koyu bölgeler arasındaki farkı belirginleştirir



$$\text{Let } T_k(r, c) = a [I_k(r, c) - 127] + 127, \text{ where } a > 1.0$$
$$J_k(r, c) = \begin{cases} 0, & \text{if } T_k(r, c) < 0, \\ T_k(r, c), & \text{if } 0 \leq T_k(r, c) \leq 255, \\ 255, & \text{if } T_k(r, c) > 255. \end{cases} \quad k \in \{1, 2, 3\}$$



127 (orta nokta) referans alınır; bu değerden büyük olanlar daha büyük, küçük olanlar ise daha küçük hale getirilerek aradaki fark

açılır.

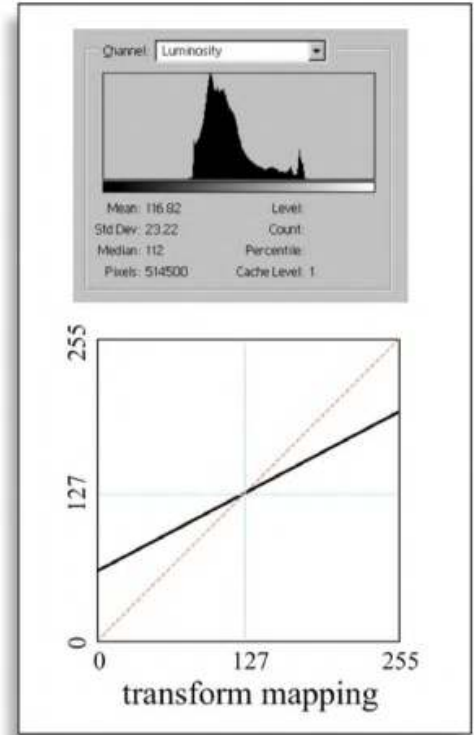
Histogram Etkisi: Histogram yayılır ve genişler (stretching)

Decrease Contrast



$$T_k(r, c) = a[I_k(r, c) - 127] + 127,$$

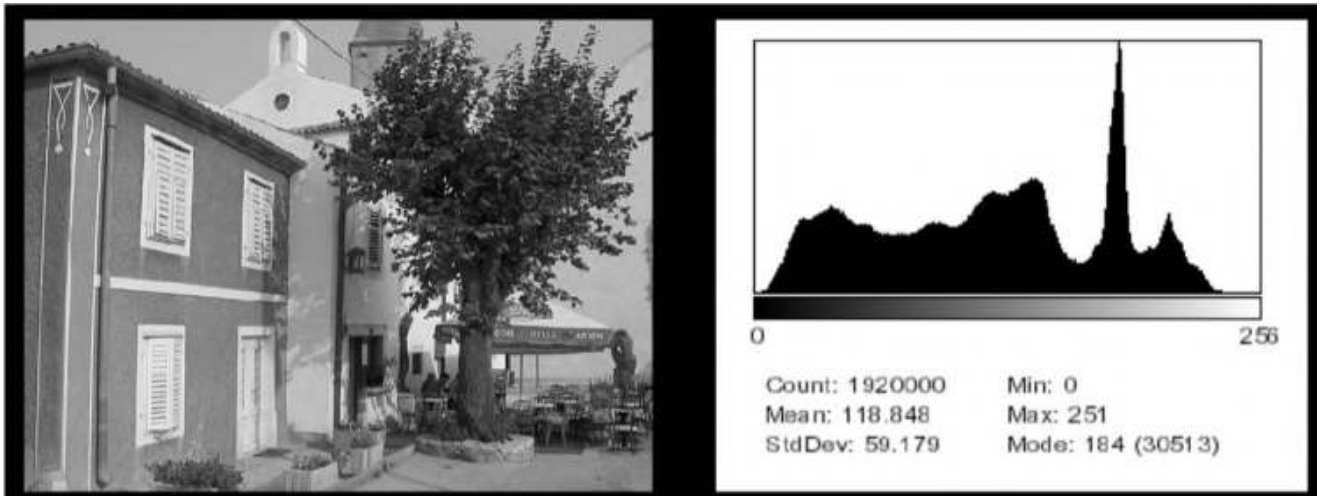
where $0 \leq a < 1.0$ and $k \in \{1, 2, 3\}$.

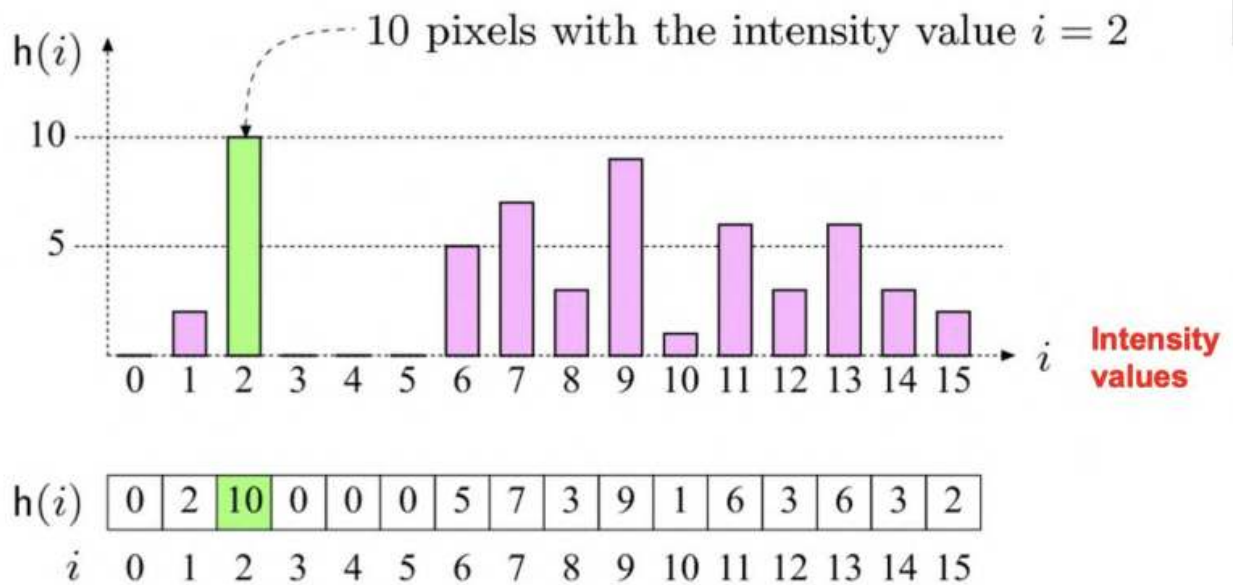


Histogram daralır

Histogram

Bir görüntüdeki her bir yoğunluk (intensity) değerinin kaç kez (frekans) tekrarlandığını gösteren bir grafik





★ Normalize edilmiş histogram

- Gri seviyesi değerleri $r_0, r_1, \dots, r_{L-1}$ olan bir resmin histogramı:

$$p(r_k) = \frac{n_k}{n}$$

n_k : Gri seviyesi r_k olan piksel sayısı

n : Görüntüdeki toplam piksel sayısı

Normalizasyon sayesinde histogram değerleri 0 ile 1 arasına çekilir.

Bu, o yoğunluk değerinin görüntüde bulunma **olasılığını**

(probability) verir

Pixelin bulunma oranı, sayısından kesinlikle daha önemli ve kullanışlıdır, eğer normalizasyon yapılmazsa 10x10 luk bir resimle 400x400 lük resimdeki piksel sayıları çok farklı ölçülür, normalizasyon yapılırsa dağılım aynı kalmış olur. Bu yüzden normalizasyon şarttır

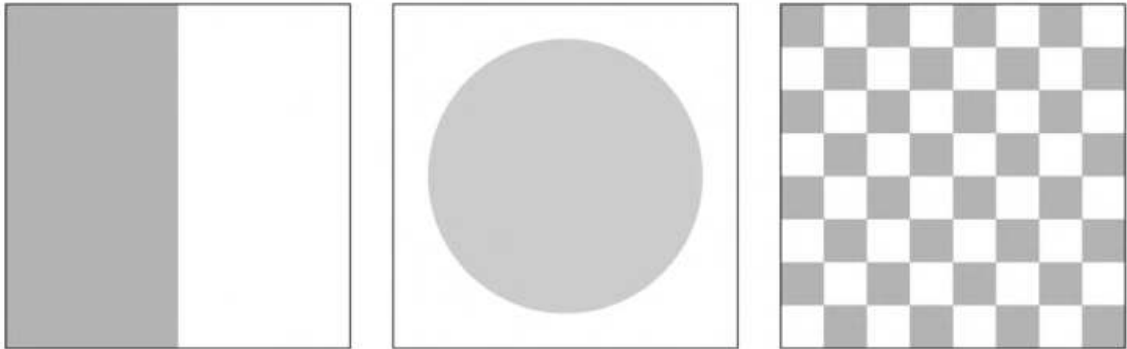
Örnek

r_k	n_k	$p_r(r_k) = n_k/MN$
$r_0 = 0$	790	0.19
$r_1 = 1$	1023	0.25
$r_2 = 2$	850	0.21
$r_3 = 3$	656	0.16
$r_4 = 4$	329	0.08
$r_5 = 5$	245	0.06
$r_6 = 6$	122	0.03
$r_7 = 7$	81	0.02

TABLE 3.1
Intensity
distribution and
histogram values
for a 3-bit,
 64×64 digital
image.

Farklı görüntüler aynı histograma sahip olabilir.

- **3 images below have same histogram**



Bir bakışta görüntünün nasıl olduğu histogramdan anlaşılabilir:

- **Dark (Karanlık):** Pikseller histogramın **sol tarafına** (düşük değerlere) yığılmıştır.
- **Light (Aydınlık):** Pikseller histogramın **sağ tarafına** (yüksek değerlere) yığılmıştır.
- **Low Contrast (Düşük Kontrast):** Histogram **dar bir alana** (genelde merkeze) sıkışmıştır; renkler birbirine çok yakındır.
- **High Contrast (Yüksek Kontrast):** Histogram tüm spektruma **yayılmıştır**; açık ve koyu tonlar arasında belirgin farklar vardır.

resim tipleri

Binary



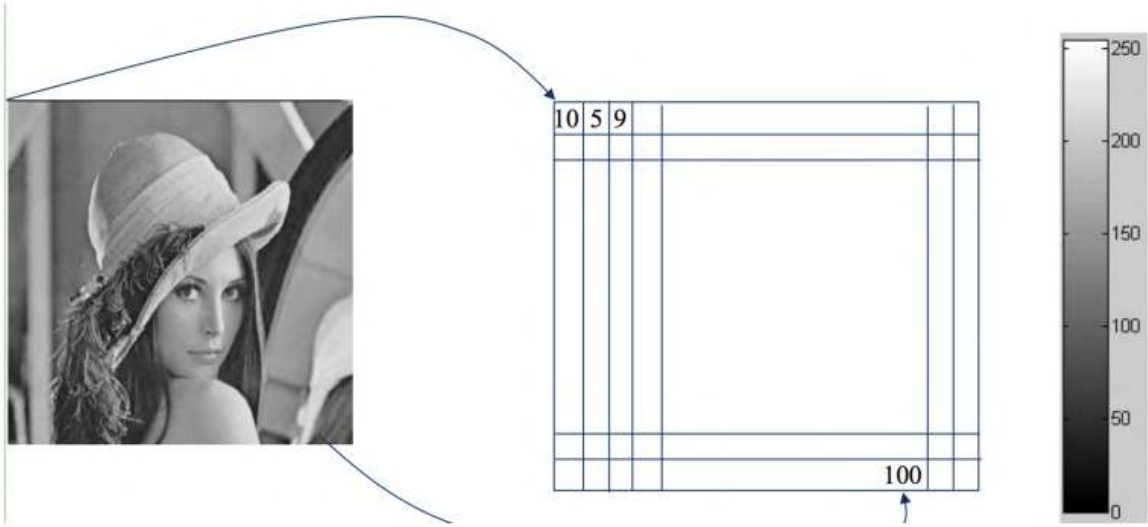
Gray Scale



Color



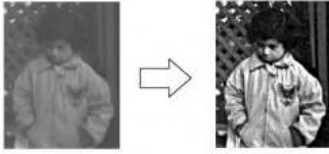
gri resmi matraste gösterirsek:



Spatial Domain Process

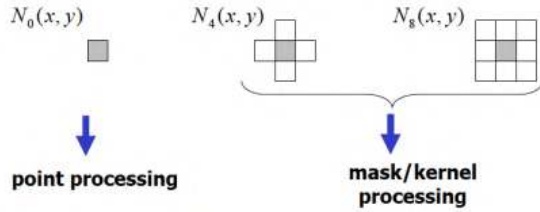
Bir maskeyi(kernel) bilgi edinmek için görüntü üzerinde pixel pixel gezdirelim.

Bir piksel tek başına net bilgi vermeyebilir, bu yüzden point operations'da yapılanın aksine işlenilecek pikselin etrafındaki pikseller de kullanılır



$$f \xrightarrow{T_N(\cdot)} g = T_N(f) \quad f(x,y) \quad , \quad 1 \leq x \leq M, 1 \leq y \leq N \quad g(x,y) \quad , \quad 1 \leq x \leq M, 1 \leq y \leq N$$

$T_N(\cdot)$: Spatial operator defined on a neighborhood N of a given pixel



Hard to tell anything from a single pixel

Example: you see a reddish pixel. Is this the object's color? Illumination? Noise?

Maskenin her bir hücreesindeki katsayı (w), o hücrenin altına gelen görüntü pikseli (f) ile çarpılır ve tüm bu çarpımlar toplanarak merkezdeki pikselin yeni değeri (g) elde edilir.

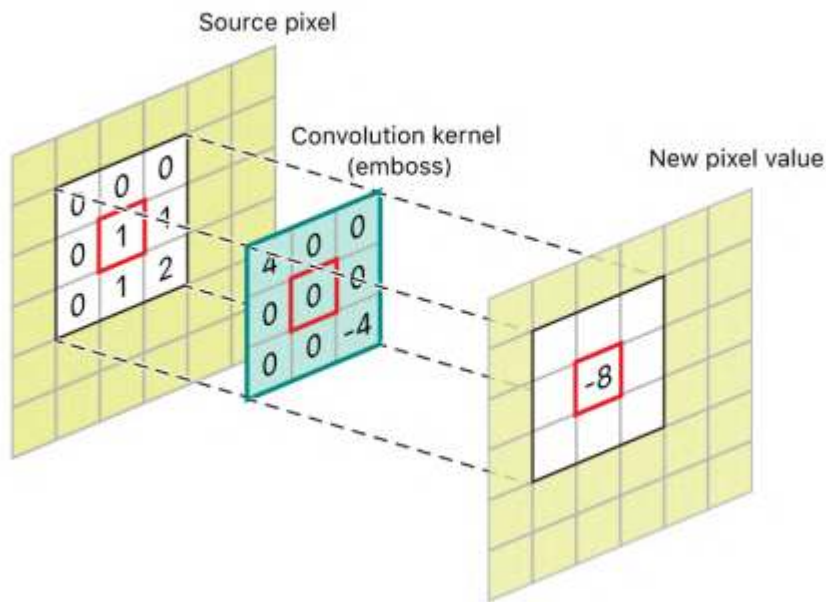
Area(Mask) Processing Methods

Bir pikselin yeni değeri, kendi eski değeri ve yakın çevresindeki piksellerin değerleri kullanılarak hesaplanır.

- basit noktasal işlemlere göre hesaplama açısından daha maliyetlidir ancak çok daha güçlü sonuçlar (kenar belirleme, yumuşatma vb.) verir

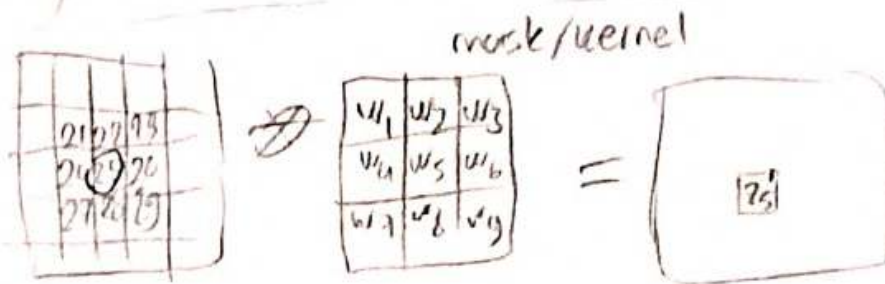
Maske $\rightarrow$ ağırlık adı verilen değerlerden oluşan matris

Konvolüsyon



Applying mask to image

Spatial Domain Process



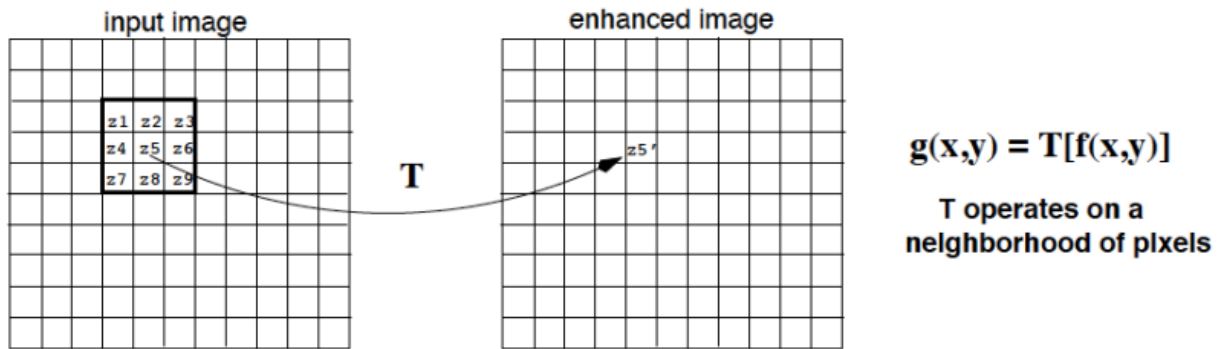
$$z_s = z_1 \cdot u_1 + z_2 \cdot u_2 + z_3 \cdot u_3 + \dots + z_9 \cdot u_9 = \sum_{i=1}^9 z_i \cdot u_i$$

$$0 \leq z_s \leq L-1$$

max. gray level

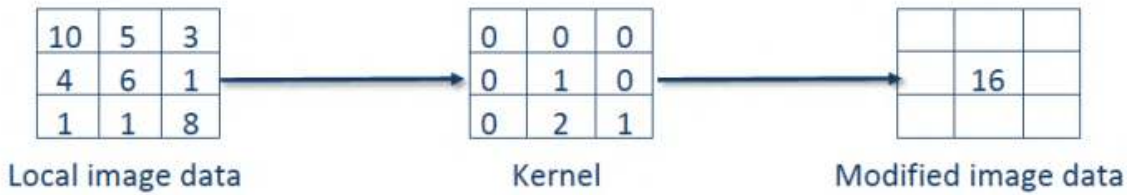
- ① $z_s < 0$ ise $z_s' = 0$
 $z_s > 255$ ise $z_s' = 255$
- ② min-max normalization





w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

$$z_5' = R = w_1z_1 + w_2z_2 + \dots + z_9w_9$$



$$R = w_1z_1 + w_2z_2 + \dots + w_{mn}z_{mn} = \sum_{i=1}^{mn} w_i z_i$$

$$R = w_1z_1 + w_2z_2 + \dots + w_9z_9 = \sum_{i=1}^9 w_i z_i$$

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

Where the w 's are mask coefficients, the z 's are the value of the image gray levels corresponding to those coefficients

Smoothing (Low Pass) Spatial Filters

amaç → Görüntüdeki parazitleri (noise) azaltmak ve görüntüyü bulanıklaştırmak (blurring).

iki kategori, linear ve order-statistics filters

Averaging Filter

en basit doğrusal filtredir. görüntüyü yumuşatır

- The key requirement is that all coefficients are positive.
- Neighborhood averaging is a special case of LPF where all coefficients are equal.
- It blurs edges and other sharp details in the image.

• Example: $\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$

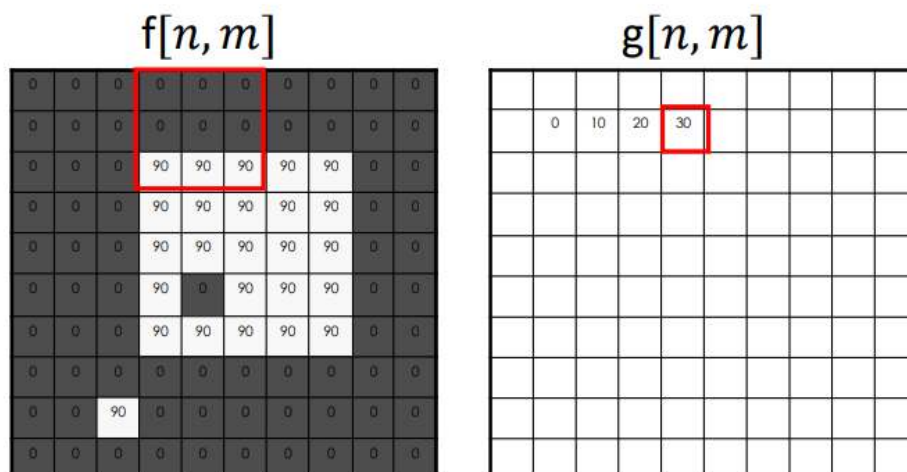
$$\frac{1}{9} \times \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array}$$

Standard average

$$\frac{1}{16} \times \begin{array}{|c|c|c|} \hline 1 & 2 & 1 \\ \hline 2 & 4 & 2 \\ \hline 1 & 2 & 1 \\ \hline \end{array}$$

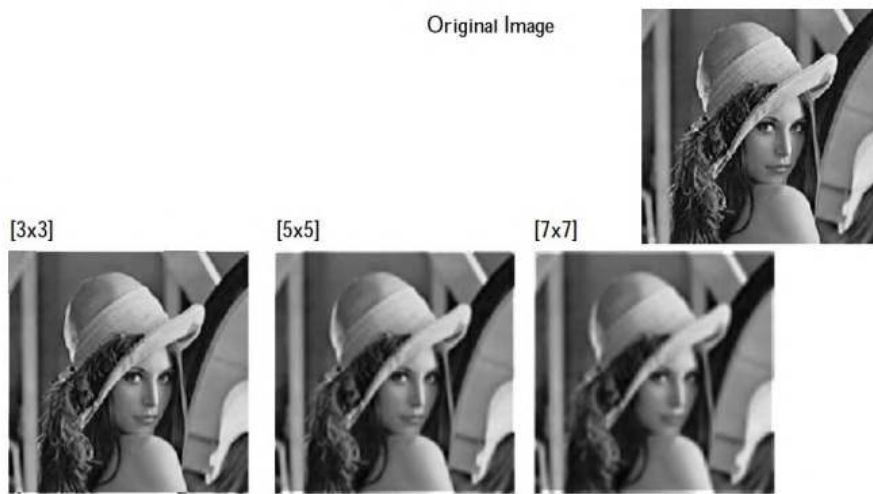
Weighted average

example process



$f[n, m]$										$g[n, m]$									
0	0	0	0	0	0	0	0	0	0										
0	0	0	0	0	0	0	0	0	0										
0	0	0	90	90	90	90	90	90	0										
0	0	0	90	90	90	90	90	90	0										
0	0	0	90	90	90	90	90	90	0										
0	0	0	90	0	90	90	90	90	0										
0	0	0	90	90	90	90	90	90	0										
0	0	0	90	90	90	90	90	90	0										
0	0	0	0	0	0	0	0	0	0										
0	0	0	90	0	0	0	0	0	0										
0	0	0	0	0	0	0	0	0	0										
0	0	0	0	0	0	0	0	0	0										
0	0	0	0	0	0	0	0	0	0										

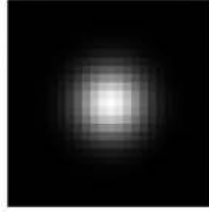
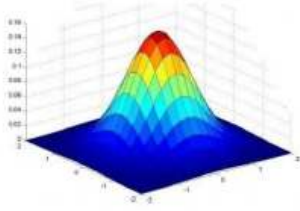
Filtre boyutu arttırılırsa yumuşama artar



Gaussian Smoothing Filter

Merkeze yakın olan pikseller, yeni değeri belirlemede çok daha etkilidir. Uzaklaştıkça etki "çan eğrisi" şeklinde azalır.

Gaussian Smoothing Filter



1-D Gaussian Function

$$G(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2}{2\sigma^2}}$$

$$G_\sigma = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

2-D Gaussian Function

$$G(x) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Daha doğal bir yumuşatma yapar

Order-statistic (Nonlinear) Filters

- Nonlinear

- Based on ordering (ranking) the pixels contained in the filter mask

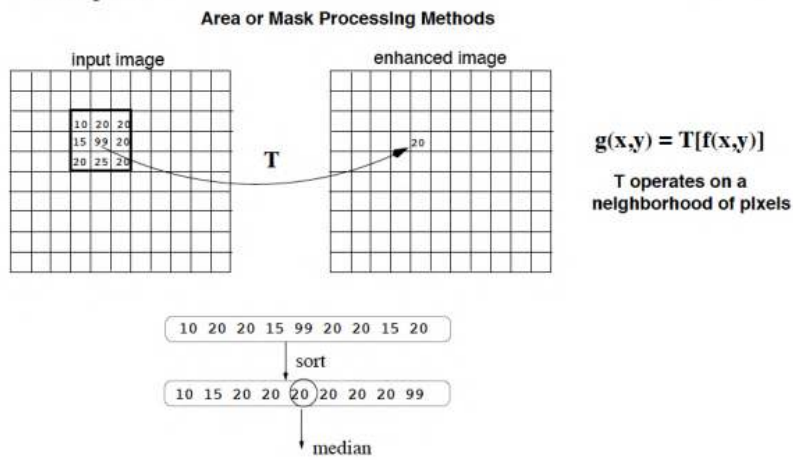
- Replacing the value of the center pixel with the value determined by the ranking result

E.g., median filter, max filter, min filter

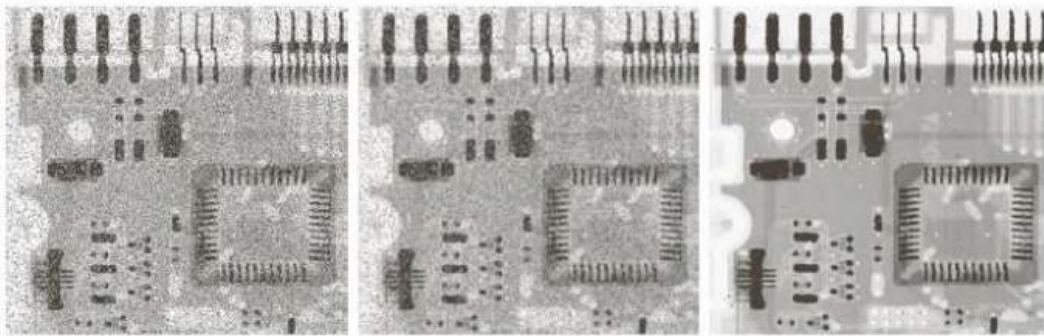
Median Filter

Amaç görüntüyü bulandırmak değil de gürültüyü yok etmekse, medyan filtresi kullanılmalı, ani spike componentleri varken çok iyi sonuç verir

- Replace each pixel value by the median of the gray-levels in the neighborhood of the pixels

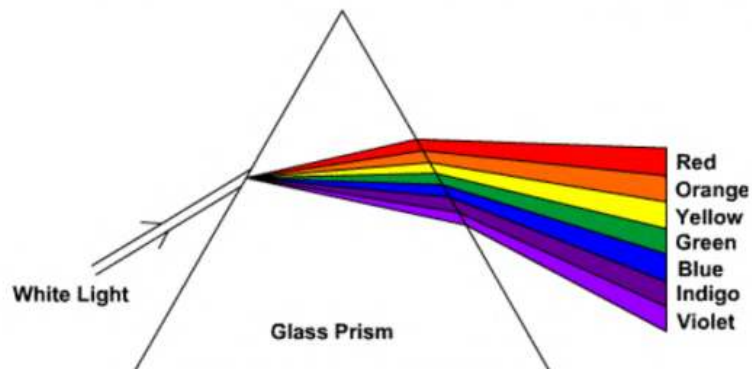


Example: Use of Median Filtering for Noise Reduction

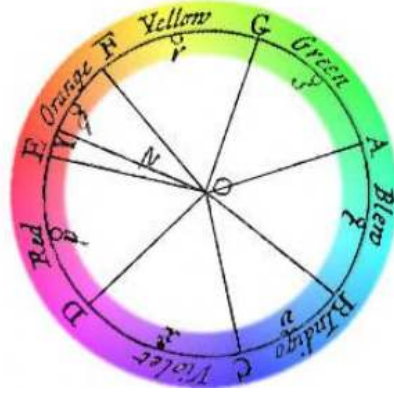


Color

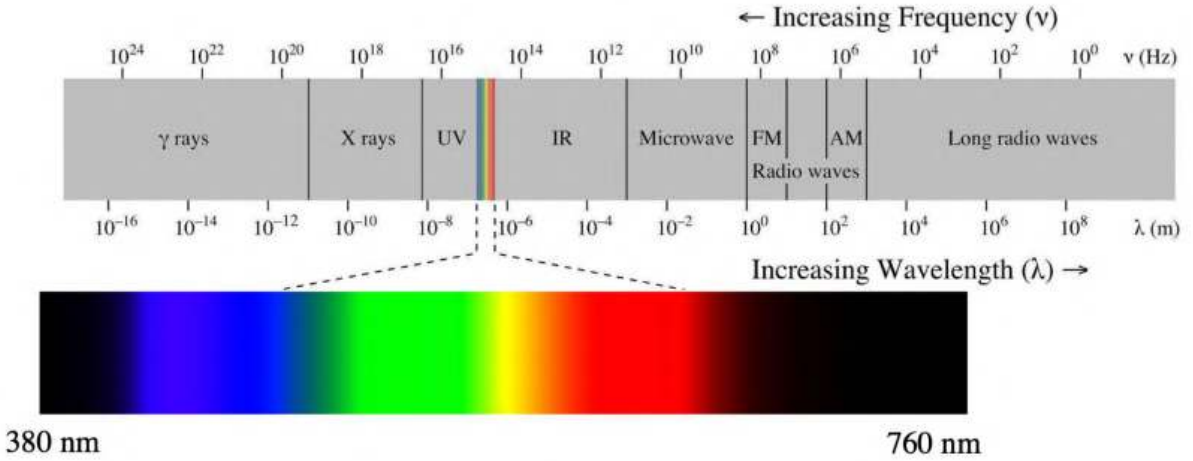
- The multicolored band of light is called **a color spectrum**.



- In his spectrum, Newton initially identified five colors :
 - red yellow green blue and violet
 - but later named two more, orange and indigo, to give a total of seven spectral components of white light.



bizim renk dediğimiz şey aslına çok düşük bir spektrum aralığında



Renk bir algıdır. Işığın farklı dalga boyları (λ) gözümüzdeki retinaya ulaştığında beynimizde renk algısı oluşur.

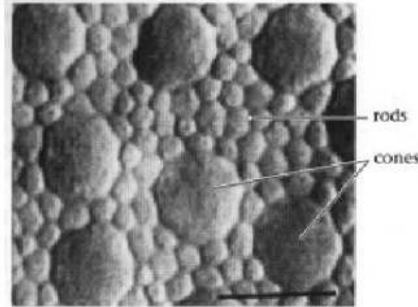
- Renkler farklı dalga boylarıyla algılanır.
- Eğer tüm dalga boyları aynı anda gözümüze ulaşırsa "beyaz", hiç ışık ulaşmazsa "siyah" görürüz.
- İnsan gözü 380-760nm arasındaki dalga boylarını algılayabilir

Gözümüzün içindeki "sensörler" (iki çeşit)

- Çubuk Hücreleri (Rods)
- Koni Hücreleri (Cones)

- There are two types of receptor cells in the retina:
Rods: (Gray tones) For the detection of low light values (monochrome night vision) : about 120 million

Cones: (Color vision) Three types of colour sensors consisting of different chemical pigments sensitive to **blue**, **green** and **red** wavelengths : about 6-7 million

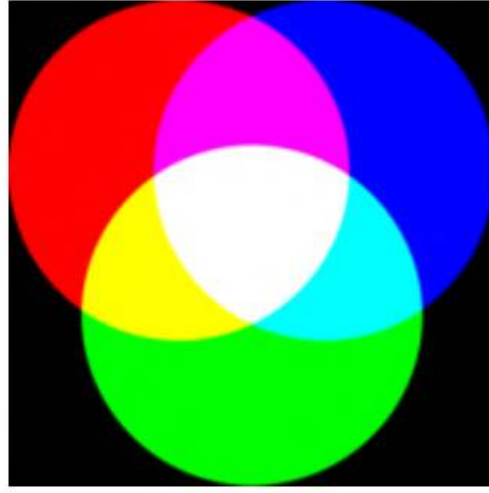


Color Spaces

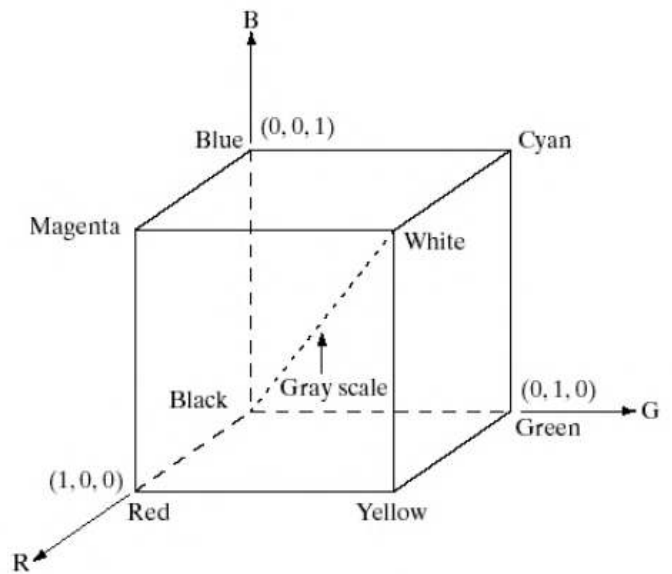
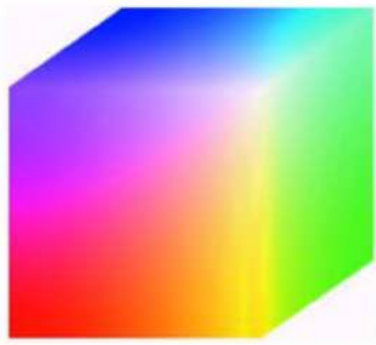
- **RGB (Red, Green, Blue):** CRT, Plazma ve LCD monitörlerde kullanılır.
- **CMYK (Cyan, Magenta, Yellow, Black):** Yazıcılarda (printer) kullanılır.
- **HSI (Hue, Saturation, Intensity):** Renk bilgisinin (insan algısına yakın şekilde) işlenmesi gereken yerlerde tercih edilir.
- **YIQ - YUV:** Televizyon yayıncılığında (NTSC ve PAL) kullanılır.
- **YCbCr:** Dijital videolarda ve sıkıştırma formatlarında (JPEG gibi) kullanılır.

RGB (additive) color space

insan gözündeki ışık dalga boyu sensörlerini (konileri) temel alır.



$$\begin{aligned} G + B &= C \\ G + R &= M \\ R + G &= Y \end{aligned}$$



24 bit RGB color cube

$$\text{Sum : } (2^8)^3 = 16,777,216 \text{ color}$$

RGB Normalizasyonu

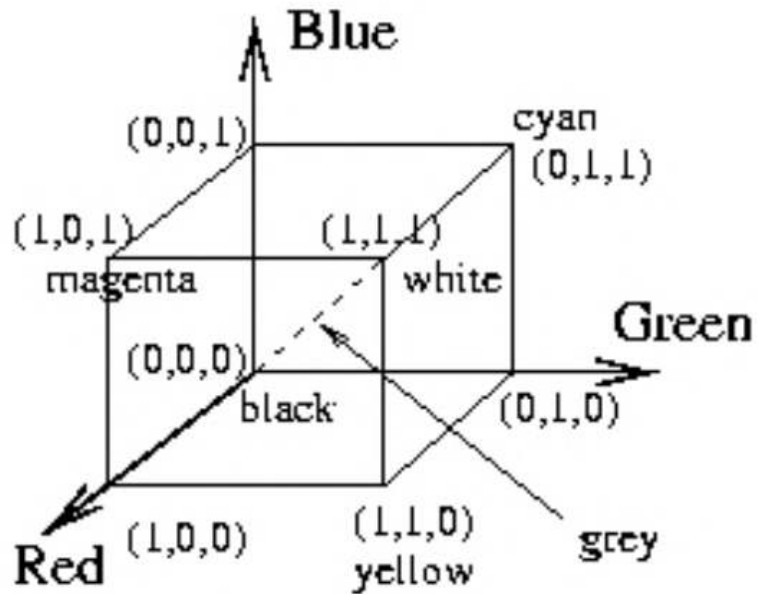
Görüntü işlemede bazen mutlak renk değerleri (0-255) yerine, renklerin birbiri içindeki oranları daha önemlidir. (**normalizasyon**)

$$r = R / (R + G + B)$$

$$g = G / (R + G + B)$$

$$b = B / (R + G + B)$$

$$r + g + b = 1$$

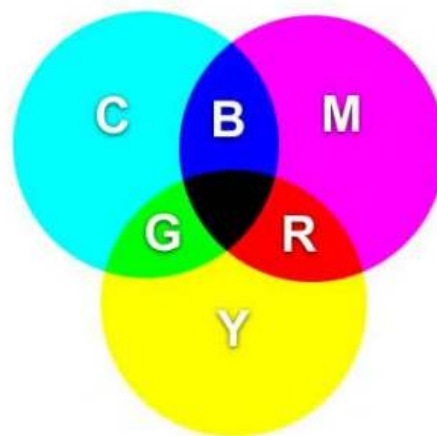
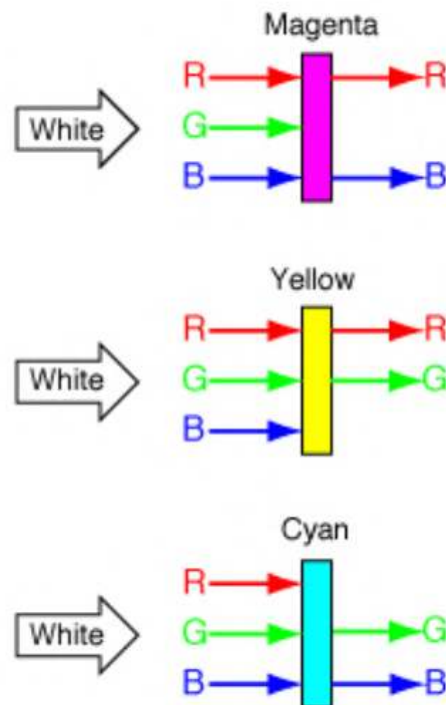


CMY Color Space

Bilgisayar ekranlarının aksine, kağıt üzerine baskı yaparken **CMY (Cyan, Magenta, Yellow)** modeli kullanılır.

Subtractive (Çıkarsal) Model: RGB ışıkları "üst üste ekleyerek" (toplayarak) renk oluştururken, CMY boyaları beyaz ışıktan bazı renkleri "emerek/çıkartarak" renk oluşturur. printerlarda da bu yüzden kullanılır

CMY Color Space



Colours are obtained by subtracting white.

$$C = 1 - (B + G)$$

$$M = 1 - (B + R)$$

$$Y = 1 - (G + R)$$

Luminance and Chromaticity

Luminance (Luma/Yoğunluk): Rengin ne kadar "parlak" veya "karanlık" olduğudur (Intensity, Brightness, Gray Value).

Chromaticity (Renklilik): Rengin tonu (Hue) ve doygunluğunun

(Saturation) birleşimidir.

Luminance : Luminance – Intensity – Brightness -
Lightness – Luma -Value

Chromaticity (Colourfulness) : Hue + Saturation

Color : Chromaticity + Brightness

HSI Color Space

- **Hue (Ton)**: Rengin ana dalga boyudur; "mavi mi, pembe mi?" sorusunun cevabıdır.
- **Saturation (Doygunluk)**: Rengin saflık ölçüsüdür. Doygunluk ne kadar fazlaysa renk o kadar canlıdır; doygunluk sıfırsa renk gridir.
- **Brightness (Parlaklık)**: Işığın şiddetidir (Gri seviyesi değeri).

HSI-HSV color Spaces

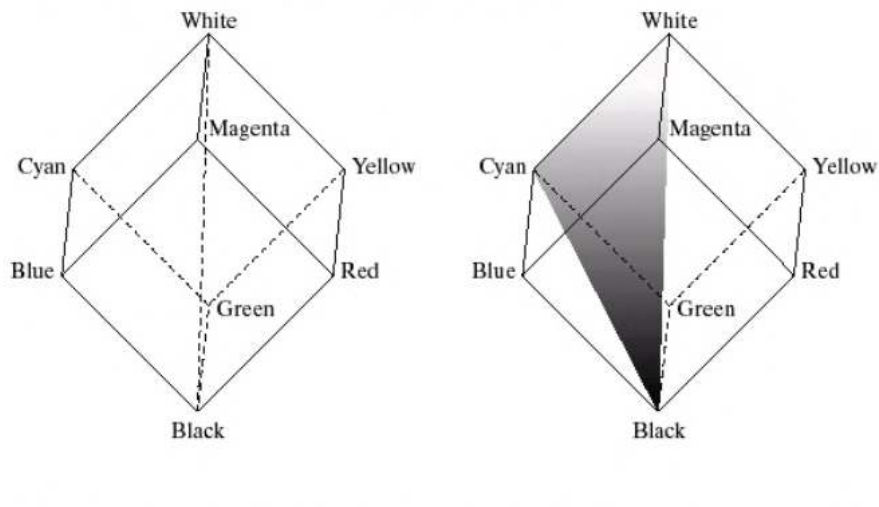
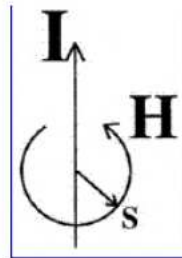


FIGURE 6.12 Conceptual relationships between the RGB and HSI color models.

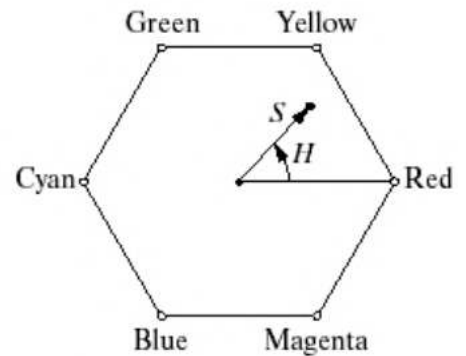
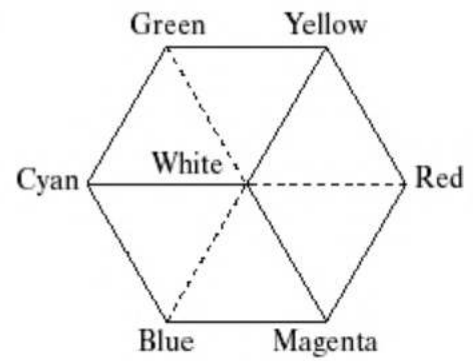
The colour in the grey area on the right is cyan

parametreler

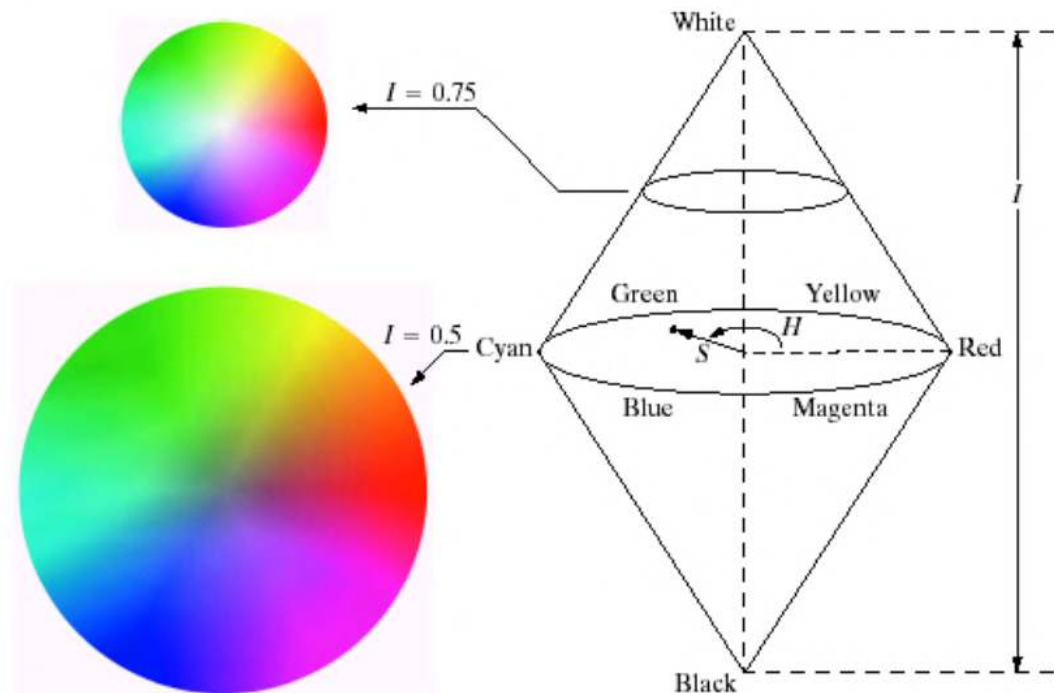
HSI-HSV Color Space



- Hue:** Angle to reference (0-360 degree)
- Saturation:** Distance to centre(0:gray 1:white)
- Intensity :** Height of the colour spot on the cone
(0: darkest, 1:brighest)
- Value :** largest value of 3 colours



HSI Color Space as a Cone



RGB - HSI Transform

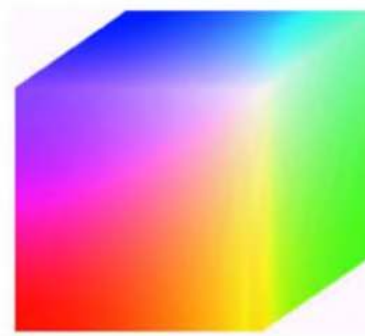
karmaşık formüller vs

$$H = \begin{cases} \theta & \text{if } B \leq G \\ 360 - \theta & \text{if } B > G \end{cases} \quad \theta = \cos^{-1} \left\{ \frac{\frac{1}{2}[(R-G) + (R-B)]}{\left[(R-G)^2 + (R-B)(G-B) \right]^{\frac{1}{2}}} \right\}$$

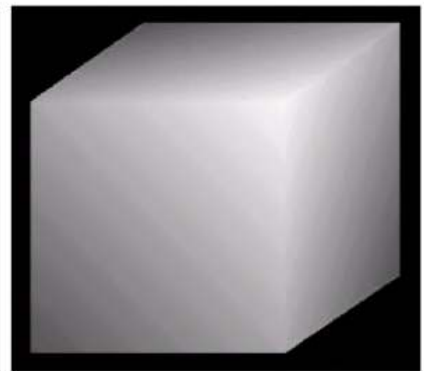
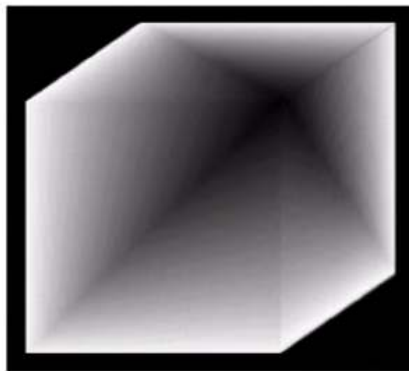
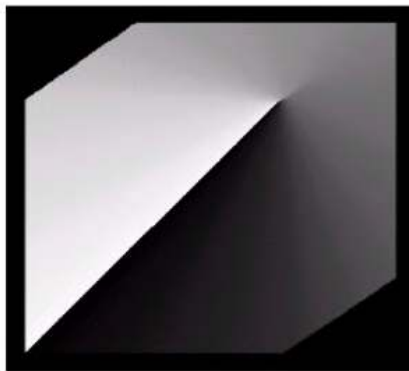
$$S = 1 - \frac{3}{(R+G+B)} [\min(R, G, B)]$$

H : 0-360 S: 0-1 I:0-1

$$I = \frac{1}{3} (R + G + B)$$



RGB Colour Cube

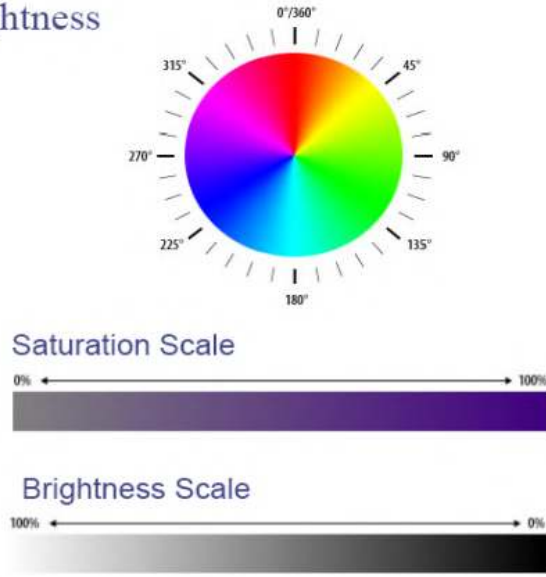
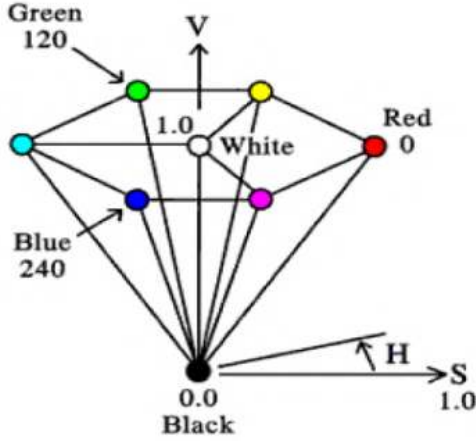


H, S, and I Components of RGB Colour Cube

HSV Model

HSV = Hue Saturation Value

a.k.a. HSB = Hue Saturation Brightness



HSI'nin RGB'ye kıyasla daha iyi bir yaklaşım olabileceği durumlar

1. **Renk ve Parlaklığın Ayrılması:** RGB'de renk ve parlaklık iç içedir. Bir rengi parlatmak istersen R, G ve B değerlerinin üçünü birden değiştirmen gerekir ve bu sırada renk tonu (pembe, kırmızıya kayabilir) bozulabilir. HSI'da ise sadece **Intensity (I)** kanalına müdahale edersen, renk (H ve S) hiç etkilenmez.
2. **İnsan Algısına Uygunluk:** İnsanlar bir rengi "yüzde şu kadar kırmızı" diye değil, "şu tonda, şu kadar canlı" diye tarif eder. HSI bu doğal algıyı taklit ettiği için manuel müdahale (renk düzenleme vb.) yaparken çok daha kontrollü bir yaklaşımdır.
3. **Işık Değişimlerine Karşı Direnç:** Bir sahnede ışık azaldığında veya gölge düştüğünde RGB değerleri altüst olur. Ama HSI uzayında nesnenin **Hue (Ton)** değeri (yani nesnenin "mavi" olduğu bilgisi) büyük oranda sabit kalır. Bu yüzden nesne takibi ve tanıma algoritmalarında "better approach" olarak kabul edilir.

Luminance-Chrominance (YUV & YIQ)

Temel Amaç: Parlaklık (Brightness) bilgisini renk bilgisinden ayırmaktır.

- YUV space used on PAL televisions in Europe
- YIQ space used on NTSC televisions in North America
- The aim is to correlate brightness with colour information.
- Advantage is displaying colours with less information, disadvantage is losing information

İnsan gözü parlaklıktaki değişimlere çok hassastır ama renk detaylarındaki değişimlere karşı daha "töleranslıdır". Bu yüzden veriyi iletirken parlaklığa (Y) tam bant genişliği ayırıp, renk bilgilerini (U-V veya I-Q) sıkıştırarak göndermek devasa bir bant genişliği tasarrufu sağlar.

YUV space used on NTSC, PAL,
SECAM Tv's

Y : luminance

U : Blue Chroma V : Red Chroma

$$\begin{pmatrix} Y \\ U \\ V \end{pmatrix} = \begin{pmatrix} 0.299 & 0.587 & 0.114 \\ -0.14713 & -0.28886 & 0.436 \\ 0.615 & -0.51499 & -0.10001 \end{pmatrix} \begin{pmatrix} R \\ G \\ B \end{pmatrix}$$

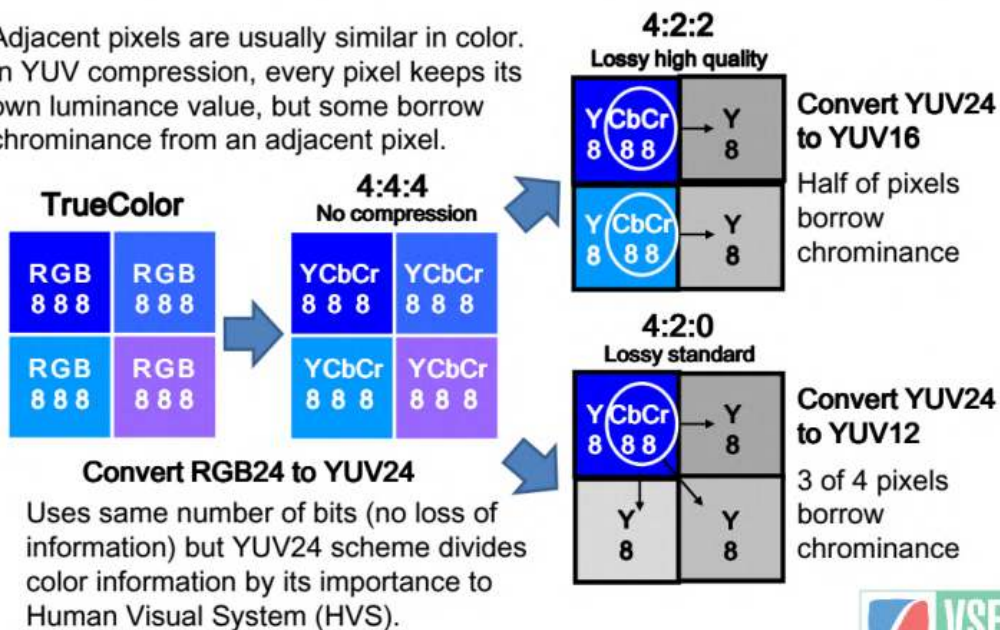
Used in colour TVs :

- YCbCr, YPbPr, YDbDr, YIQ
- RGB representation is divided into 4 parts :
 - 1 luminans (grayscale) component : Y
 - 3 chrominance (color) component: Cr , Cg, Cb
- Cr, Cb is enough, Cg is calculated from them

★ Sıkıştırılmalarda kullanılan en önemli tekniklerden → komşu hücreler chrominance değerini kopyalayabilir

YUV Compression Schemes

Adjacent pixels are usually similar in color. In YUV compression, every pixel keeps its own luminance value, but some borrow chrominance from an adjacent pixel.



YIQ Color Space

Y : luminance

I + Q : Chrominance I : blue chroma Q: red chroma

$$\begin{bmatrix} Y \\ I \\ Q \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ 0.595716 & -0.274453 & -0.321263 \\ 0.211456 & -0.522591 & 0.311135 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1 & 0.956295719758948 & 0.621024416465261 \\ 1 & -0.272122099318510 & -0.647380596825695 \\ 1 & -1.106989016736491 & +1.704614998364648 \end{bmatrix} \begin{bmatrix} Y \\ I \\ Q \end{bmatrix}$$

YCbCr Color Space

YCbCr color space used on **JPEG** and **MPEG** formats

Y : luminance

Cb : chroma-blue (B-Y)

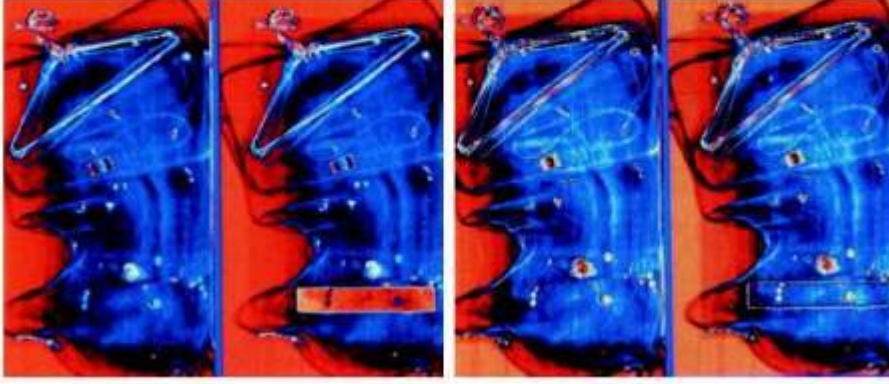
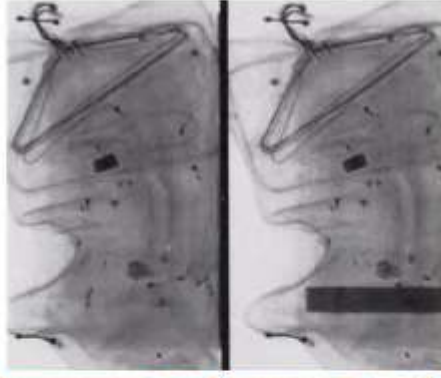
Cr: chroma-red (R-Y)

$$\begin{bmatrix} Y' \\ C_b \\ C_r \end{bmatrix} = \begin{bmatrix} 65.481 & 128.553 & 24.966 \\ -37.797 & -74.203 & 112 \\ 112 & -93.786 & -18.214 \end{bmatrix} \begin{bmatrix} R' \\ G' \\ B' \end{bmatrix} + \begin{bmatrix} 16 \\ 128 \\ 128 \end{bmatrix}$$

Veriyi bu formda saklamak, görüntüyü kayıplı şekilde sıkıştırmayı (gereksiz renk detaylarını atmayı) mümkün kılar.

Pseude Coloring

We can convert gray image to color şmage

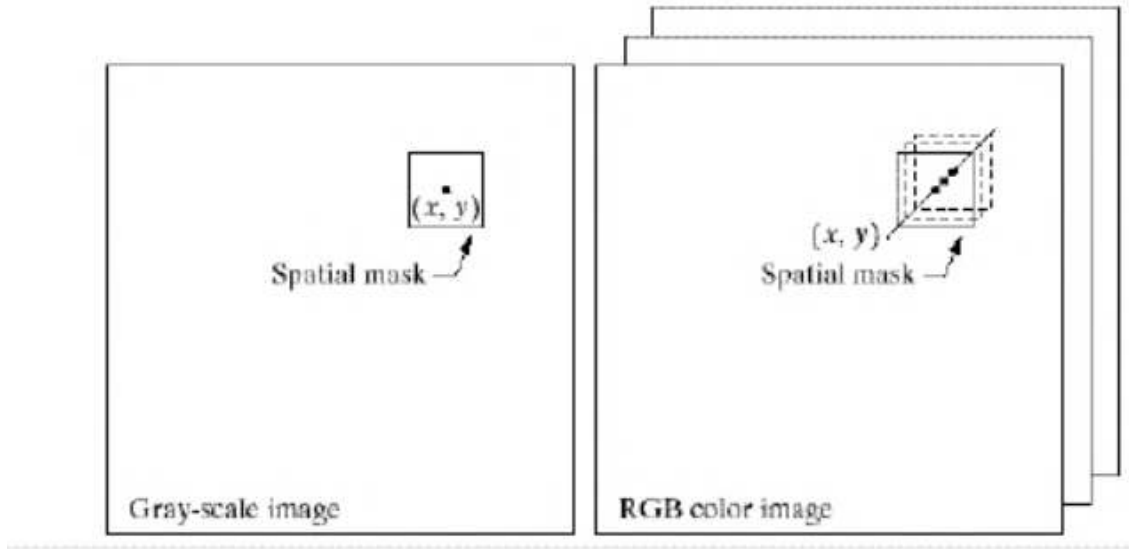


- İnsan gözü bir görüntüdeki yaklaşık 20-30 farklı gri tonunu anca ayırt edebilirken, binlerce farklı rengi ayırt edebilir. Siyah-beyaz bir röntgen filmindeki çok küçük bir yoğunluk farkını (örneğin bir tümörü) gözden kaçırabiliriz.
- **Çözüm:** Gri seviyeli görüntüyü renge dönüştürürüz.
- **Yöntem:** Belirli gri aralıklarına (örneğin 0-50 arası koyu griler için mavi, 200-255 arası açık griler için kırmızı) farklı renkler atarız.
- **Sonuç:** Termal kameralar veya hava durumu haritaları bunun en iyi örneğidir; sıcaklık farklarını gri tonu olarak değil, renk olarak çok daha hızlı anlarız.

Color Image Processing

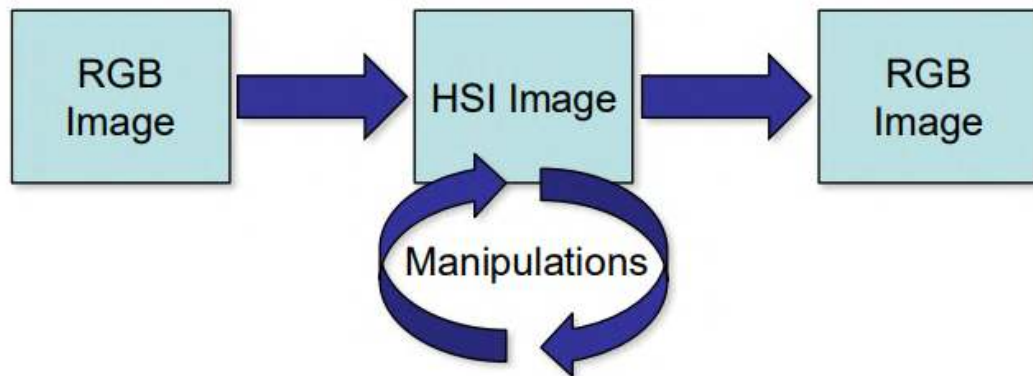
Renkli bir resme işlemler (yumuşatma, keskinleştirme vs.) nasıl uygulanabilir?

1. RGB space → kanalları tek tek işlemek



2. Yol: HSI Uzayında İşlemek (HSI Space)

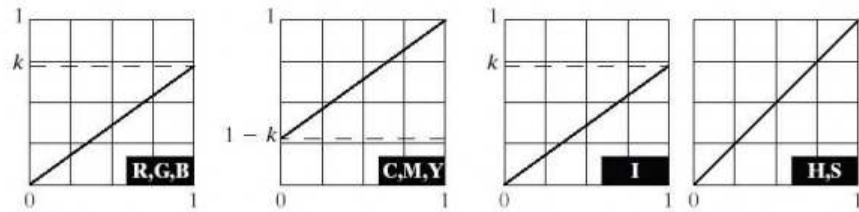
- **Adım 1:** RGB görüntüsünü HSI uzayına çevirirsin.
- **Adım 2 (Kritik):** Sadece **Intensity (I)** kanalına işlem uygularsın (çünkü renk bilgisini yani H ve S'yi bozmak istemezsin).
- **Adım 3:** İşlenmiş görüntüyü tekrar RGB'ye çevirirsin.



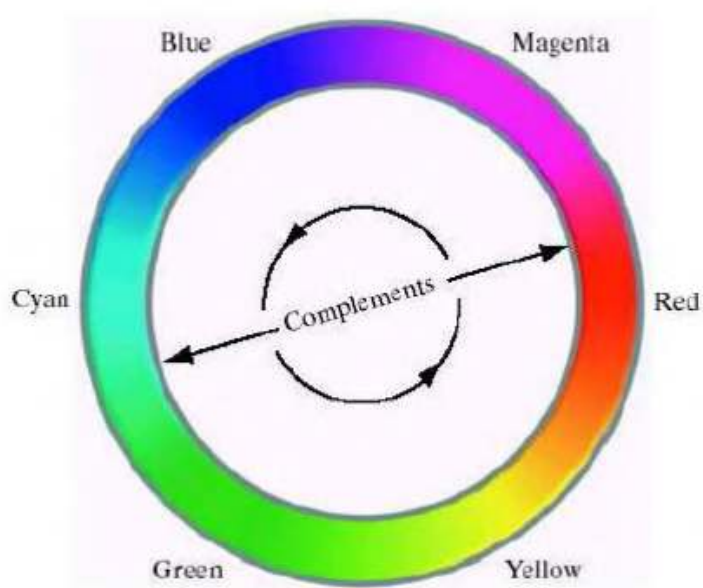
intensity changing daha kontrollü parlaklık ayarlaması sağlar

a b
c d e

FIGURE 6.31
Adjusting the intensity of an image using color transformations. (a) Original image. (b) Result of decreasing its intensity by 30% (i.e., letting $k = 0.7$). (c)–(e) The required RGB, CMY, and HSI transformation functions. (Original image courtesy of MedData Interactive.)



negative of a color image ($255-X$)

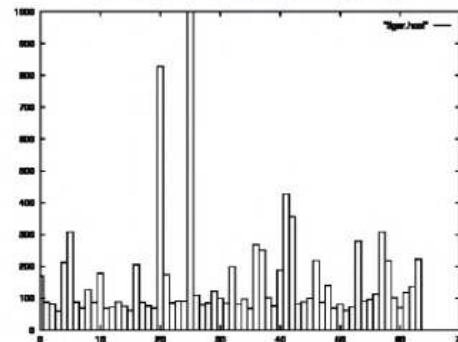
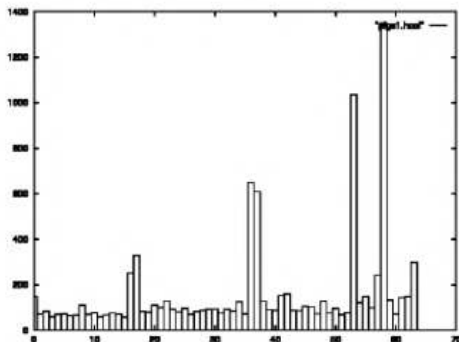


How to make a color histogram

- Make a single 3D histogram.
- Make 3 histograms and concatenate them
- Create a single pseudo color between 0 and 255 by using 3 bits of R, 3 bits of G and 2 bits of B (which bits?)
- Use normalized color space and 2D histograms.

özellikle CBIR (İçerik Tabanlı Görüntü Erişimi) çalışmalarında milyonlarca renk tonunu sadece 64 ana gruba indirgemek gibi işlemler yapılabilir

64-bin Color Histogram

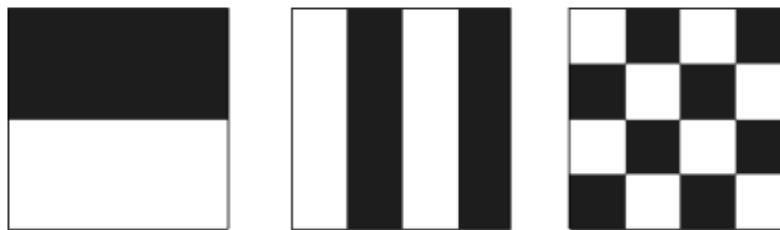


Texture (Doku)

- Texture is a repeating pattern of local variations in image intensity.
 - Texture cannot be defined for a point



- For example, an image has a 50% black and 50% white distribution of pixels
- Three different images with the same intensity distribution, but with different textures :



Bark



Bark



Fabric



Fabric



Fabric



Flowers



Flowers



Flowers

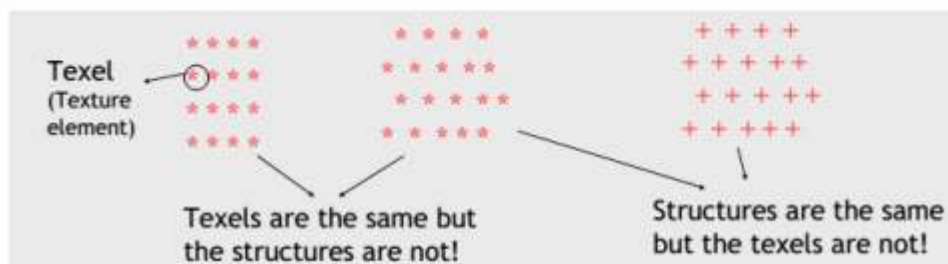


Two approaches for texture analysis:

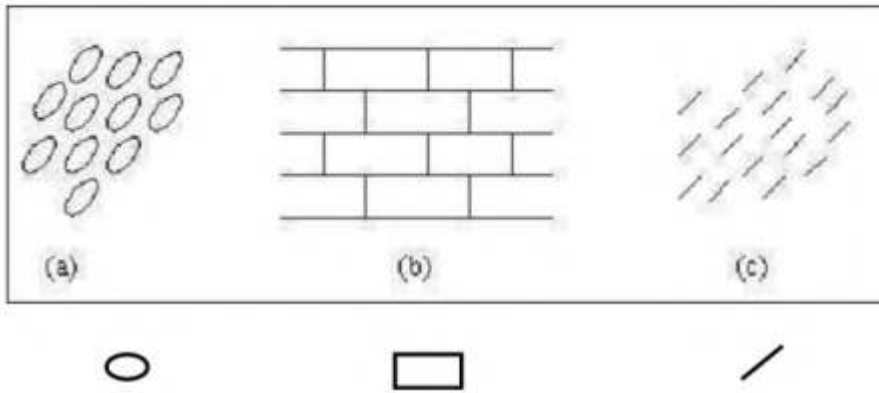
- **Statistical**; convenient for natural textures
- **Structural**; convenient for artificial textures

Structural Approach

- Texture can be thought as a **set of primitive texels** in some regular or repeated relationship
- A **texel** is a group of pixels that have similar tonal properties such as constant color, gray level, average intensity, etc... .



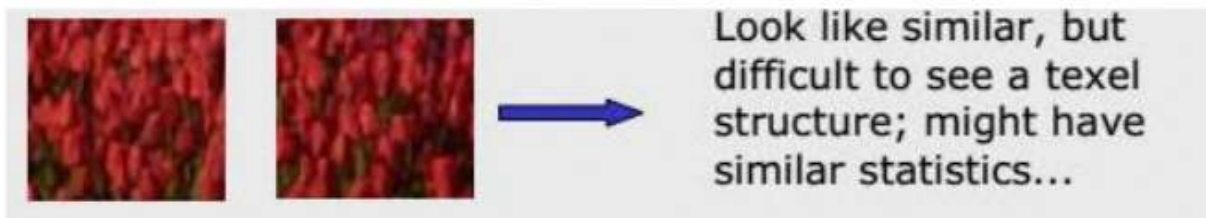
texels



Statistical Approach

Gerçek (doğal) görüntülerde tek bir texel birimini görüntüden ayırmak (segmenting) genellikle çok zordur veya imkansızdır. Bu yaklaşımda ise görüntünün gri tonlarından veya renklerinden doğrudan hesaplanan sayısal miktarlar ve istatistikler kullanılır.

- Segmenting out texels is difficult or impossible in real images
- Numeric quantities or statistics that describe a texture can be computed from the gray tones (or colors) alone



İstatistiksel Doku Betimleme Yöntemleri

Bu sayfa, doku analizi literatüründe en çok kullanılan istatistiksel yöntemlerin bir listesini sunar.

- **Kenar Yoğunluğu ve Yönü (Edge Density and Direction):**
Görüntüdeki kenarların ne kadar sık ve hangi yönde olduğuna

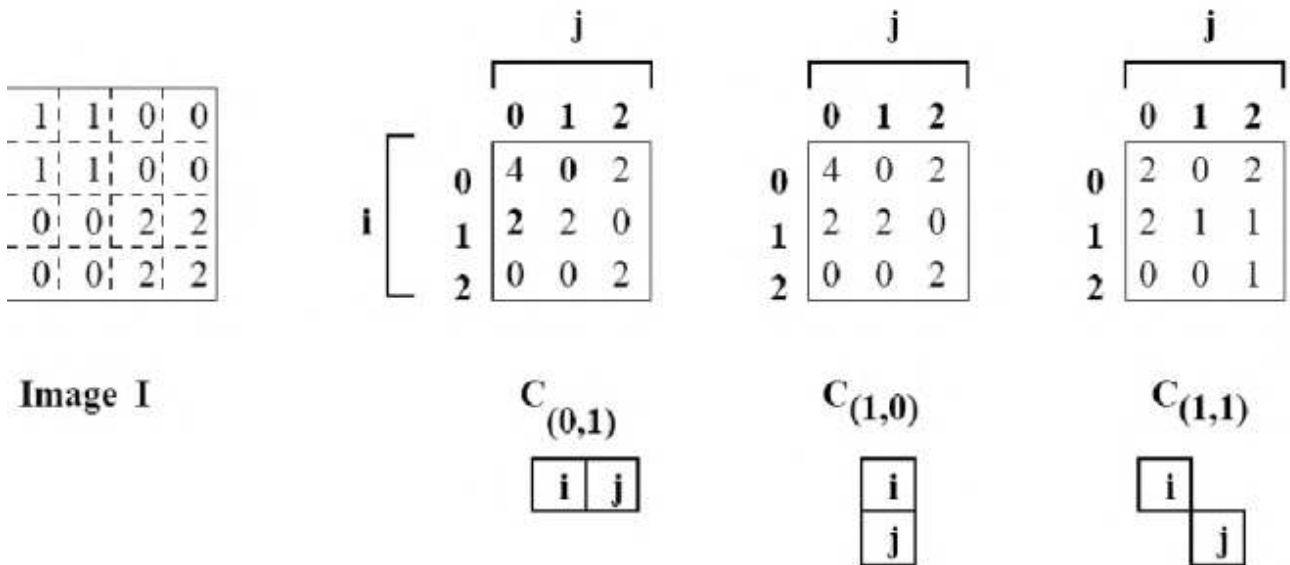
odaklanır.

- **Histogram ve Özellikler:** Piksellerin yoğunluk dağılımını analiz eder.
- **Eş Oluşum Matrisleri (Co-occurrence Matrices):** Piksellerin birbirine göre konumsal ilişkilerini inceler (en yaygın yöntemlerden biridir).
- **Gelişmiş Filtreler:** Fourier güç spektrumu, Gabor filtreleri ve Otoregresyon (Autocorrelation) gibi frekans tabanlı yöntemler.
- **LBP (Local Binary Partition):** Yerel ikili örüntüler ile doku tanımlama.

Gray Level Co-Occurrence Matrices

- Measure direct image statistics
- A co-occurrence matrix is a 2D array C in which
- Both the rows and columns represent a set of possible image values
 - $C[i,j]$ indicates how many times value i co-occurs with value j in a particular spatial relationship d
 - The spatial relationship is specified by a vector $d = (dr,dc)$
 - pixel to the left, $d = [1\ 0]$
 - pixel above, $d = [0\ 1]$
 - pixel below, $d = [0\ -1]$

★ örnek



GLCM Varyasyonları

- ★ **Normalize Edilmiş GLCM:** Matris elemanlarını toplam eleman sayısına bölerek olasılık değerlerine dönüştürür.
- Simetrik GLCM:** Simetrik komşulukları gruplar (yani i 'den j 'ye olan ilişki ile j 'den i 'ye olanı aynı kabul eder)



Gray Level Co-Occurrence Matrices

- Two variations of the standard gray-level co-occurrence matrix
 - Normalised gray-level co-occurrence matrix

$$= \frac{\sum \sum}{\sum \sum}$$

- Symmetric gray-level co-occurrence matrix
 - groups pairs of symmetric adjacencies

$$= \begin{vmatrix} & + & - \\ & & \end{vmatrix}$$

örnek soru

4) Aşağıdaki görüntüde [1,1] komşulukları için Normalized Gray Level Co-Occurance matrisini hesaplayınız. (10 puan)

0	0	1	1	2	2
0	0	1	1	2	2
1	1	2	2	0	0
1	1	2	2	0	0
2	2	0	0	1	1
2	2	0	0	1	1

*doku analizinde normalizasyon yaparken denenecek toplam ilişki sayısı baz alınır! Yani burada matriste kaç [1,1] (alt sağ çapraz) ilişkisi mevcutsa ona bölüyoruz 6x6 matriste çapraz yönünde bakılabilen çift sayısı: 25

elemanlar {0,1,2}

C[1,0] -> aşağı doğru

C[0,1] -> sağa doğru

C[1,1] -> çapraz doğru

$$C_{(0,1)}$$

i	j
---	---

$$C_{(1,0)}$$

i	j
---	---

$$C_{(1,1)}$$

i	j
---	---

0	0	1	1	2	2
0	0	1	1	2	2
1	1	2	2	0	0
1	1	2	2	0	0
2	2	0	0	1	1
2	2	0	0	1	1

GLCO matrisi:

	0	1	2
0	3	4	1
1	2	3	4
2	4	1	3

normalizasyon
(*1/25)

Normalize GLCO matrisi:

	0	1	2
0	3/25	4/25	1/25
1	2/25	3/25	4/25
2	4/25	1/25	3/25

Moments

GLCM matrisinden tek bir sayısal özellik (feature) çıkarmak için kullanılan matematiksel formüller

- The order k element difference moment can be defined as:

$$Mom_k = \sum_i \sum_j (i - j)^k P_d[i, j]$$

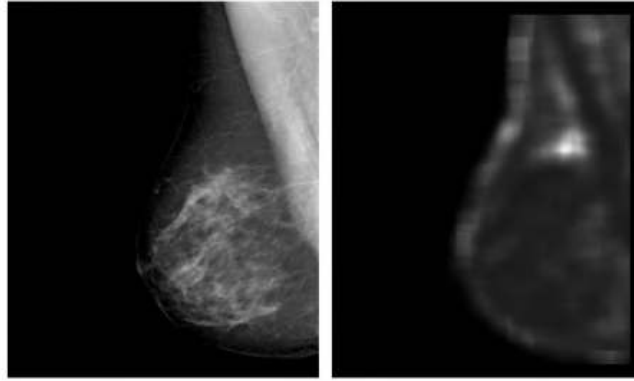
- This descriptor has small values in cases where the largest elements in P are along the principal diagonal. The opposite effect can be achieved using the inverse moment.

$$Mom_k = \sum_i \sum_j \frac{P_d[i, j]}{(i - j)^k}, i \neq j$$

örnek kullanım

- Pencere Boyutu ($w = 21$):** Analizin her piksel çevresindeki 21x21'lik bir alanda yapıldığını gösterir.
- Mesafe ($d = (2, 2)$):** Piksellerin 2 birim sağa ve 2 birim aşağı çapraz ilişkisine bakıldığını ifade eder

- Moments with $w=21$, and $d=(2,2)$



Pencere boyutunu seçmek çok önemlidir; çok küçük seçilirse doku örüntüsü yakalanamaz, çok büyük seçilirse farklı dokular birbirine karışır.

GLCM üzerinden türetilen bir başka istatistiksel özelliğin görselleştirilmesine örnek

Maximum Probability

- Maximum probability with $w=21$, and $d=(2,2)$



Belirli bir komşuluk ilişkisi içinde en sık tekrar eden piksel çiftinin olasılığını temsil eder.

Laws Texture Energy Measures

Giriş görüntüsü özel doku filtreleri kullanılarak filtrelendir

- Her pikselin etrafındaki yerel bir mahallede, filtreleme sonuçlarının mutlak değerleri toplanarak "doku enerjisi" hesaplanır.
- Sonuçta her piksele dokusal bir özellik değeri atanmış olur.



Laws Texture Energy Measures

- A set of 9 5x5 convolution masks is used to compute a vector of nine numeric features for each pixel.
- The 2D masks are computed from the following 1D masks:
- L5 : Gaussian, E5:Gradient, S5:LOG, R5 : Gabor

L5 = (1 4 6 4 1) → Level (Center-weighted local average)
 E5 = (-1 -2 0 2 1) → Edge (Detects edges)
 S5 = (-1 0 2 0 -1) → Spot (Detects spots)
 R5 = (1 -4 6 -4 1) → Ripple (Detects ripples)

The mask E5L5, for instance, is computed as:

$$E5^T L5 = \begin{bmatrix} -1 \\ -2 \\ 0 \\ 2 \\ 1 \end{bmatrix} \times \begin{bmatrix} 1 & 4 & 6 & 4 & 1 \end{bmatrix} = \begin{bmatrix} -1 & -4 & -6 & -4 & -1 \\ -2 & -8 & -12 & -8 & -1 \\ 0 & 0 & 0 & 0 & 0 \\ 2 & 8 & 12 & 8 & 2 \\ 1 & 4 & 6 & 4 & 1 \end{bmatrix}$$



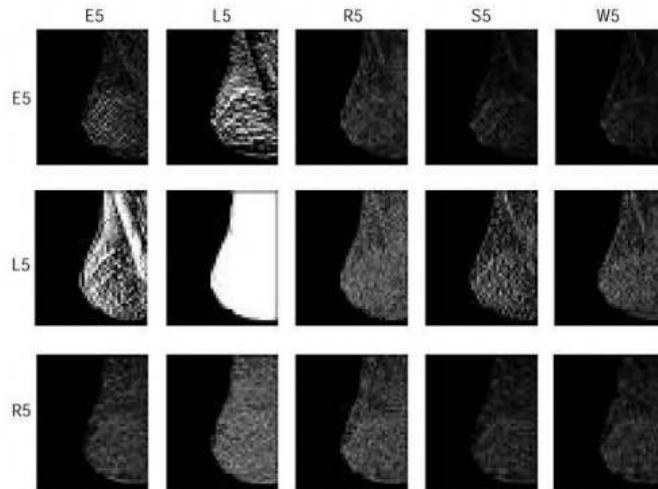
Commonly used masks

L5E5/E5L5	L5S5/S5L5
L5R5/R5L5	E5E5
E5S5/S5E5	E5R5/R5E5
S5S5	S5R5/R5S5
R5R5	

Her maske farklı bir dokusal özelliğe (örneğin dikey kenarlar veya dairesel benekler) duyarlıdır. Bir görüntüyü bu kombine edilmiş maskelerin birçoğuyla tarayarak o görüntünün "dokusal imzasını" çıkarabilirsiniz.



Mammogram Example



Local Binary Partition (LBP)

- For each pixel p in the image, create an 8-bit number $B = b_0b_1b_2b_3b_4b_5b_6b_7$
- Check the eight neighbour of p

$$= \begin{cases} 1 & \text{if } p > \text{neighbour} \\ 0 & \text{otherwise} \end{cases}$$

0	1	2
3	*	4
5	6	7

- Texture is represented by a histogram of B
- L1 distance can be used to compare two image/region

Bir piksel için 8 bitlik sayı oluşturulur → Eğer komşu pikselin değeri merkez pikselden büyük veya eşitse **1**, küçükse **0** değeri verilir.

An example

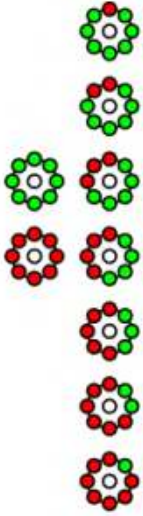
110	20	198	30	64
110	147	137	75	64
47	89	56	27	6
9	11	73	112	167
1	3	6	143	186

	01100000	11010000	10010000	
	11100000	11110001	11110111	
	11101000	11001001	00001011	

Dimensionality Reduction

LBP çok iyi ama 256 farklı değerle uğraşmak bazen bilgisayarı yorar (curse of dimensionality). İşte burada **Uniform LBP** devreye girer. → Bir bit dizisinde 0'dan 1'e veya 1'den 0'a geçiş sayısı **en fazla 2** ise o desen "uniform" (tekdüze) kabul edilir

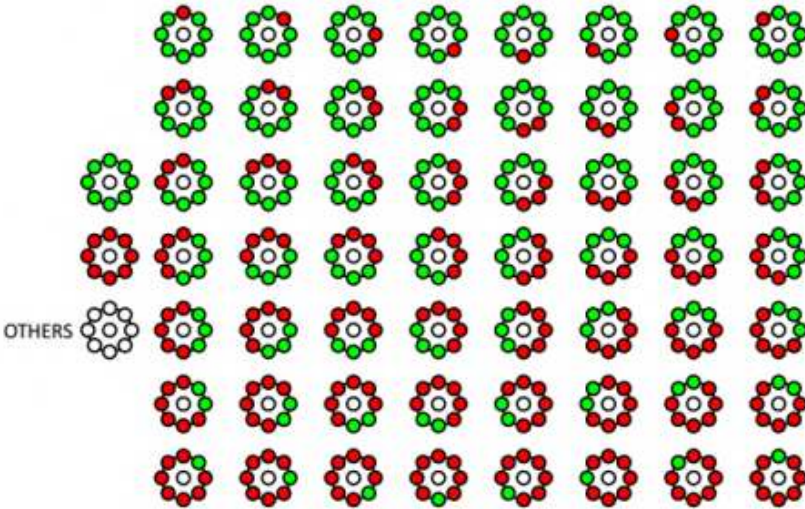
- Uniform Local Binary Patterns



– Patterns with at most two contiguous regions

- Two patterns with one contiguous region
- Seven basic patterns with two contiguous regions
- Each basic pattern has eight orientations

- 58 Uniform Local Binary Patterns plus one



- Doğal görüntülerdeki dokuların yaklaşık **%90'ı** aslında bu uniform desenlerden oluşur.
- Bu yöntemle 256 farklı özellik yerine sadece **59** özellik (58 uniform desen + 1 "diğerleri" grubu) kullanılır.

- Uniform olmayan desenler genellikle gürültü (noise) kaynaklı olduğu için bunları elemek sonucu daha güvenilir yapar.

LBP'yi bilgisayarlı görüde bu kadar popüler yapan şey sadece hızı değil, **ışık değişimlerine karşı direncidir**. Bu yüzden yüz tanıma sistemlerinde (yüzün bir tarafı gölgede kalsa bile) çok başarılı sonuçlar verir.

- Competitive with state-of-the-art for many image processing tasks
- Robust to monotonic changes in illumination



örnek soru

- 3) Aşağıda verilen görüntü parçasında işaretli merkez piksele (Local Binary Partition) işlemi uygulayınız. İşlem sonucu oluşan değer Uniform LBP şartını sağlıyor mu? (5 puan)

100	70	10
200	50	100
10	70	50

Piksel için 8 bitlik LBP sayısı oluşturulacak -> 80'den küçük değerler 0, büyük veya eşit değerler 1

b1	b2	b3
b8	B	b4
b7	b6	b5

B = b1b2b3b4b5b6b7b8 -> B=10010001

Uniform LBP DEĞİL

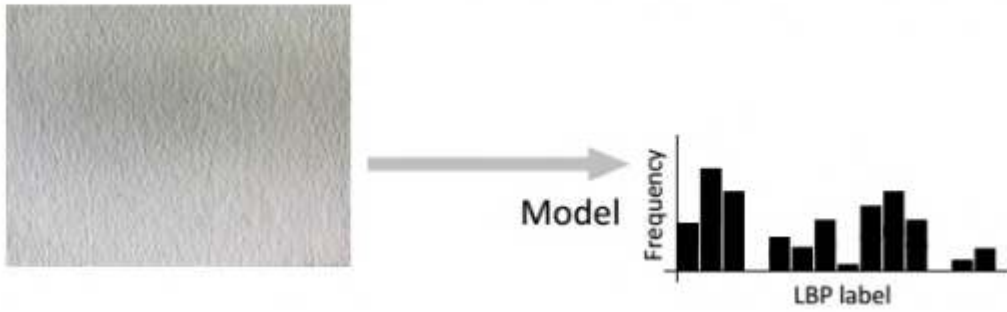
1->0 0->1 1->0 0->1 4 geçiş var✗

Uniform olması için en fazla 2 geçiş olmalı

Texture Classification

LBP kullanarak "bu doku o doku mu?" sorusuna nasıl cevap veriyoruz

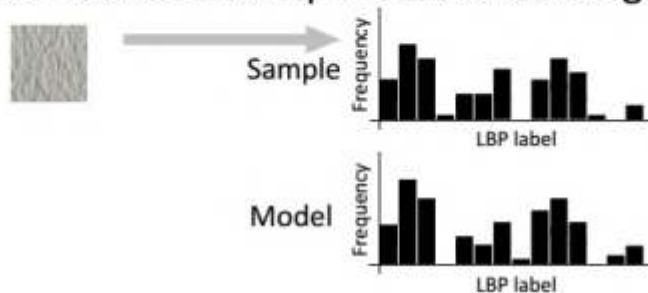
- Calculate LBP for every pixel in exemplar
- Histogram is model of micro-textures in exemplar



Tanımak istediğimiz yeni bir örnek (sample) geldiğinde, onun da LBP histogramını çıkarıyoruz. Ardından bu iki histogram arasındaki mesafeyi (distance) hesaplıyoruz.

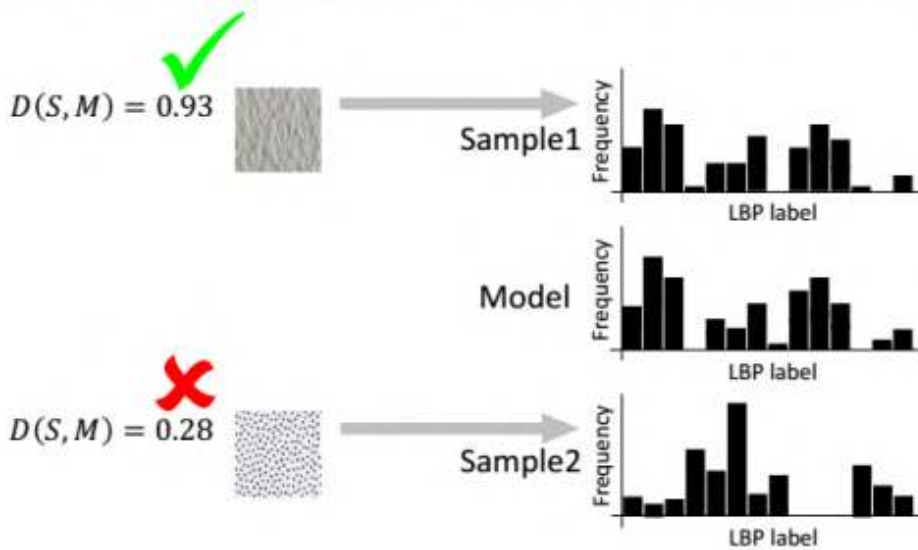
Texture Classification: Recognition

- Create LBP histogram from sample
- Find distance between sample & model histograms



Makine öğrenmesi algoritmaları burada bir "eşik değeri" (threshold) bulur. Eğer mesafe bu eşiğin altındaysa "eşleşme başarılı", üstündeyse "farklı doku" (kırmızı çarpı) kabul edilir.

- Machine learning finds threshold for classification



$D(S, M) \rightarrow$ Similarity

$\text{Dist}(A, B) \rightarrow$ Uzaklık

★ Distance ölçümü için meşhur yöntemler: Euclidian ve Manhattan Distance

Manhattan Distance $\rightarrow$ Histogramların mutlak değer farkı

$$D_{L1}(S, M) = \sum_{i=1}^n |S_i - M_i|$$

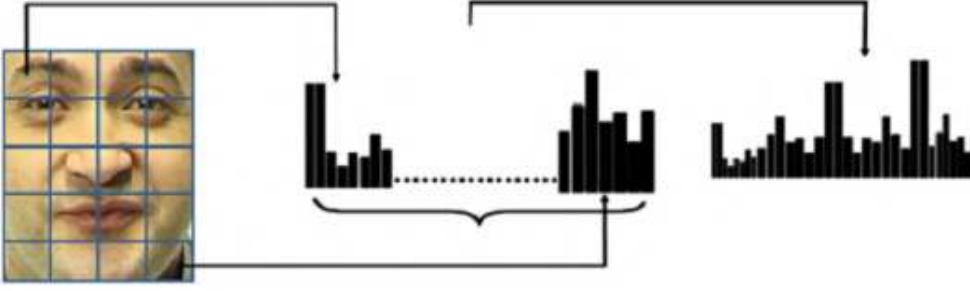
Euclidian Distance $\rightarrow$ Kare farklarıyla hesap yapar

$$D_{L2}(S, M) = \sqrt{\sum_{i=1}^n (S_i - M_i)^2}$$

LBPnin en meşhur uygulama alanlarından $\rightarrow$ yüz tanıma
parça parça değerlendirmek göz vs. yerlerin kaybolmasını engeller?

□ Face Descriptor

- Divide face into blocks, concatenate LBP histograms



Gabor Filters for Texture Analysis

Gabor filtresi, bir **sinüzoidal** (yön veren) ve bir **Gausyen** (ağırlık/pencere veren) bileşenin birleşimidir.

- Sinüzoidal bileşen filtreye **yön** bilgisini katar.
- Gausyen bileşen ise filtreye **ağırlık** verir (merkezdeki piksellere daha çok odaklanır).
- Gabor filter is a convolution filter representing a combination of sinusoidal and gaussian components.
 - the sinusoidal component provides directions
 - the gaussian component provides weights.

$$g(x, y; \lambda, \theta, \psi, \sigma, \gamma) = \exp\left(\frac{-x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \exp\left(i\left(\frac{2\pi x'}{\lambda} + \psi\right)\right)$$

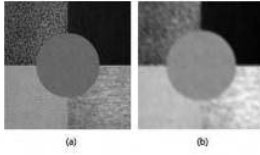
Where, $x' = x\cos\theta + y\sin\theta$
 $y' = -x\sin\theta + y\cos\theta$

Labels in the diagram:

- Wavelength of the sine component (points to λ)
- Phase offset (points to ψ)
- Spatial aspect ratio (points to γ)
- Orientation of the Gabor filter (points to θ)
- Standard deviation of the Gaussian envelope (points to σ)

Farklı filtre kombinasyonlarıyla farklı açılardaki dokular yakalanabilir

Segmentation by Gabor Filter



(a) Input image,
(b) filtered image by Gabor filter



(a) filtered image by Gabor filter
(b) Input image

Tek bir Gabor filtresi yetmez; çünkü doku her yönde ve her boyutta olabilir. Bu yüzden bir "Filtre Bankası" oluşturulur.

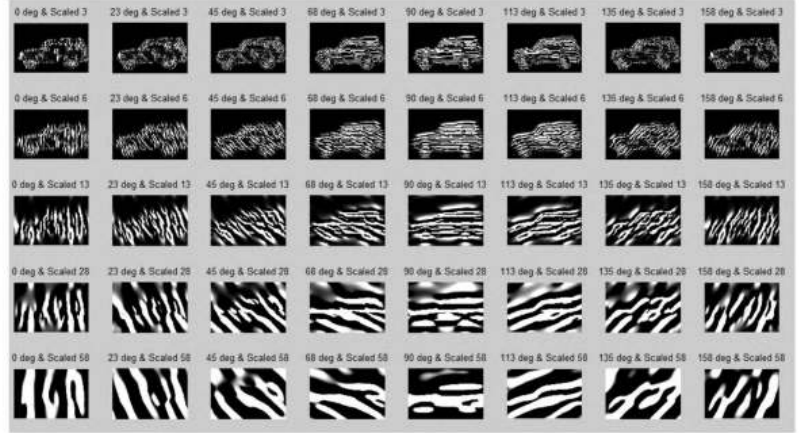
Gabor Filter Bank

$f \backslash \theta$	0	$\frac{\pi}{8}$	$\frac{2\pi}{8}$	$\frac{3\pi}{8}$	$\frac{4\pi}{8}$	$\frac{5\pi}{8}$	$\frac{6\pi}{8}$	$\frac{7\pi}{8}$
0.25								
0.18								
0.13								
0.09								
0.06								

Real parts of the Gabor filter bank. Generated for different combinations of θ (in radians) and f (in Hz) with $\phi = 0$.

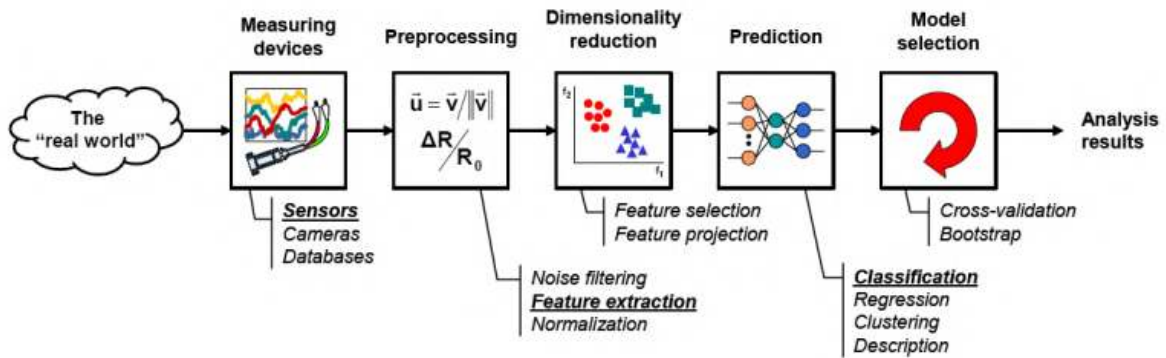
Gabor Filter Bank Example

Suppose you have an image like



Convolutional Neural Networks

Stages of a machine learning system

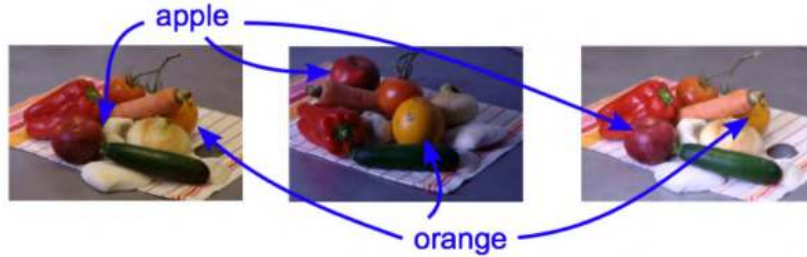


- **Veri Toplama (Measuring devices):** Sensörler, kameralar veya veri tabanlarından ham verinin alınması.
- **Ön İşleme (Preprocessing):** Gürültü filtreleme, özellik çıkarma (feature extraction) ve *normalizasyon* gibi işlemlerle verinin temizlenmesi.
- **Boyut İndirgeme (Dimensionality reduction):** Tahmin için en önemli özelliklerin seçilmesi (feature selection) veya verinin daha düşük boyutlu bir alana iz düşürülmesi (feature projection).
- **Tahmin (Prediction):** Sınıflandırma, regresyon veya kümeleme (clustering) yöntemleriyle bir sonuç üretilmesi.

- **Model Seçimi (Model selection):** Çapraz doğrulama (cross-validation) gibi yöntemlerle en iyi modelin belirlenmesi.

Why is Object Recognition Difficult ?

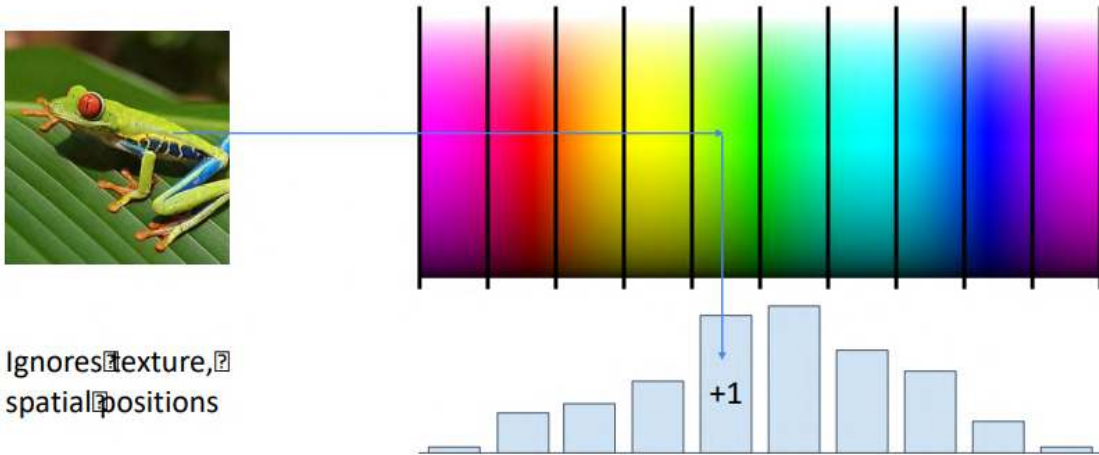
- Birden fazla nesnenin bulunması, bakış açısı farklılıkları, ışık değişimleri, nesnelerin deformasyona uğraması, birbiriyle ilişkili çok fazla özelliğin olması vb.



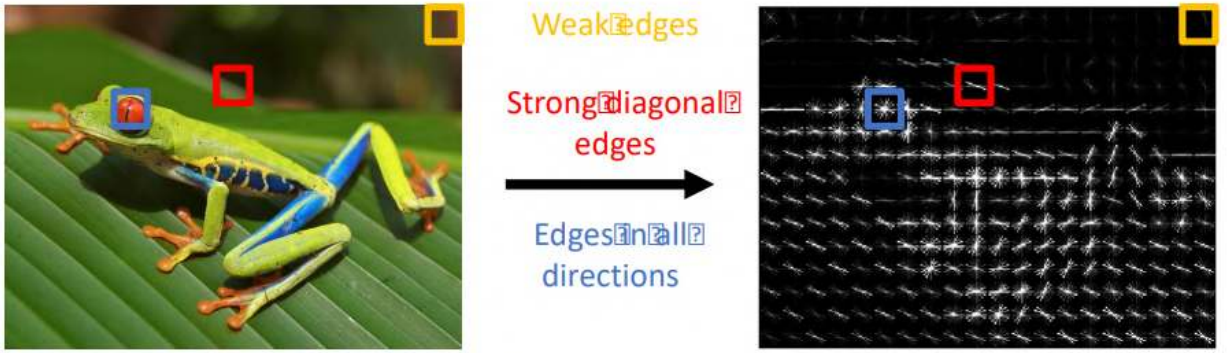
Geleneksel Özellikler

CNN'ler öncesinde araştırmacılar özellikleri elle tanımlardı:

- **Renk Histogramı (Color Histogram):** Görüntüdeki renk dağılımını ölçer. Ancak dokuyu (texture) ve piksellerin konumlarını (spatial positions) görmezden gelir.



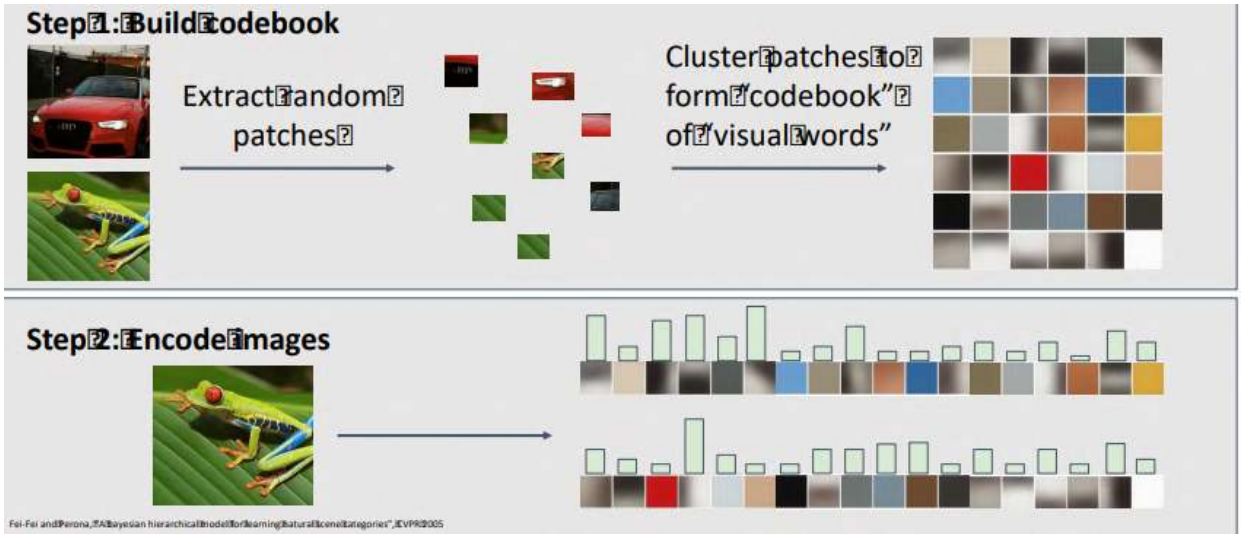
- **HOG (Histogram of Oriented Gradients):** Görüntüdeki kenar (edge) yönelimlerinin yoğunluğunu temel alır. Görüntüyü 8x8 hücrelere bölerek her hücre için kenar yönü histogramı oluşturur.



- **SIFT (Scale-Invariant Feature Transform):** Ölçekten bağımsız özellik dönüştürümüdür. İki görüntü arasındaki anahtar noktaları (keypoints) eşleştirerek nesneyi tanımasını sağlar.



- **Bag of Words (Görsel Kelime Çantası):** Görüntülerden yamalar (patches) çıkarılır, bunlar kümelenecek bir "görsel sözlük" oluşturulur ve görüntü bu sözlükteki kelimelere göre kodlanır.



Modern Yaklaşım:

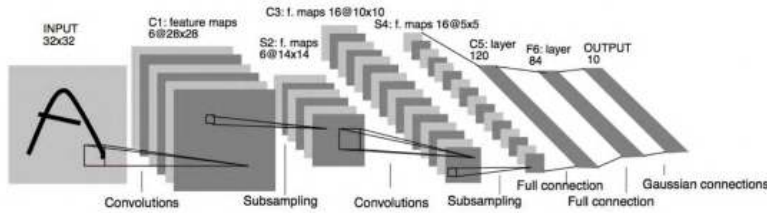
LeNet-5

CNN'in temeli

Giriş görüntüsü (32x32) sırasıyla Konvolüsyon (Convolution) ve Alt-örnekleme (Subsampling/Pooling) katmanlarından geçerek özellik haritaları (feature maps) çıkarır. Son aşamada Tam Bağlantılı (Full connection) katmanlarla çıktı üretilir.

Gradient-Based Learning Applied to Document Recognition

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner



LeCun et al., 1998

El yazısı rakamlardan oluşan MNIST Veri Seti , LeNet'in başarısını kanıtladığı alandır. 60.000 orijinal eğitim verisi bulunur ve test hata oranı %0.95 civarındadır

ImageNet projesi

Fei-Fei Li tarafından 2006'da başlatılmıştır. Amacı, yapay zeka algoritmalarını eğitmek için mevcut veri miktarını devasa oranda artırmaktır. WordNet isimli kelime veritabanı temel alınarak inşa edilmiştir

IMAGENET Large Scale Visual Recognition Challenge

The Image Classification Challenge:
1,000 object classes
1,431,167 images



Output:
Scale
T-shirt
Steel drum
Drumstick
Mud turtle

Deng et al, 2009
Russakovsky et al. IJCV 2015

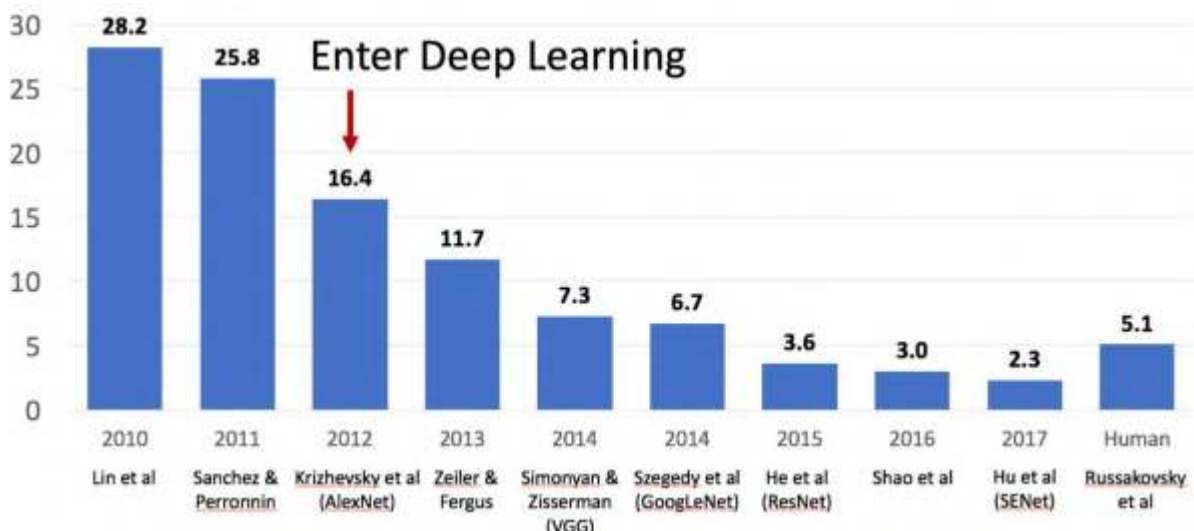
Classification goals:

- ☐ Make 1 guess about the label (Top-1 error)
- ☐ make 5 guesses about the label (Top-5 error)



yarışma tarihçesi vs vs vs.

2012: Alex Krizhevsky ve Geoffrey Hinton'ın ekibi, ilk kez bir Evrişimli Sinir Ağı (CNN) olan AlexNet ile yarışmayı kazandı. Devasa hata düşüşü

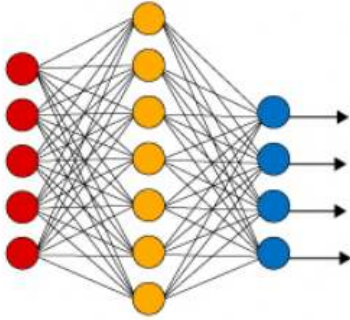


artificial neural network tarihçesi vs vs vs.

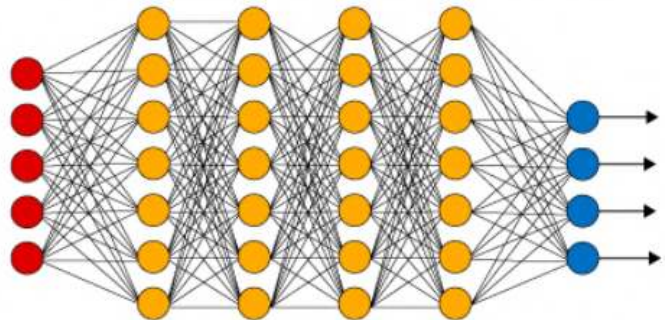
Deep Neural Network

- A neural network that consists of more than **three layers**—*which would be inclusive of the inputs and the output*—can be considered a deep learning algorithm

Simple Neural Network



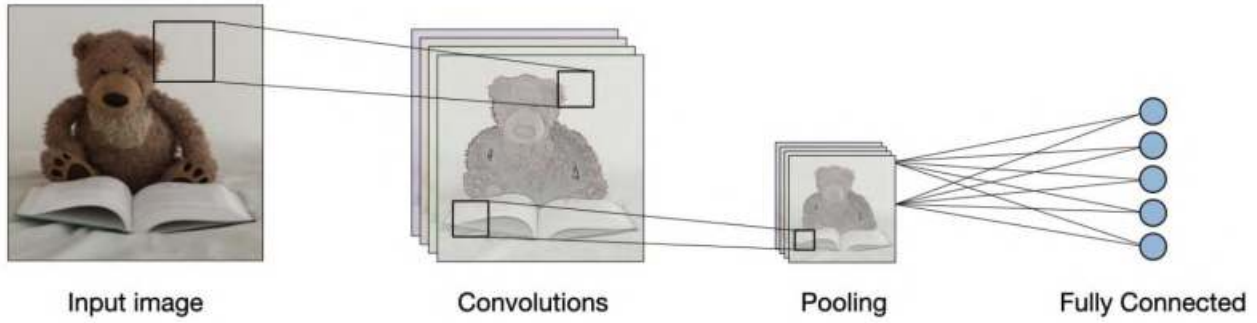
Deep Learning Neural Network



● Input Layer ● Hidden Layer ● Output Layer

- **Aşırı Öğrenme (Overfitting):** Sinir ağları her türlü fonksiyonu öğrenebildiği için veriyi "ezberleme" eğilimindedir.
- **Önleme Yöntemleri:** Daha fazla veri toplamak, daha az gizli katman kullanmak, Regularization (Düzenleştirme), Dropout (Eğitim sırasında nöronların rastgele kapatılması) ve Early Stopping (Hata artmaya başlayınca eğitimi durdurma) gibi teknikler kullanılır.

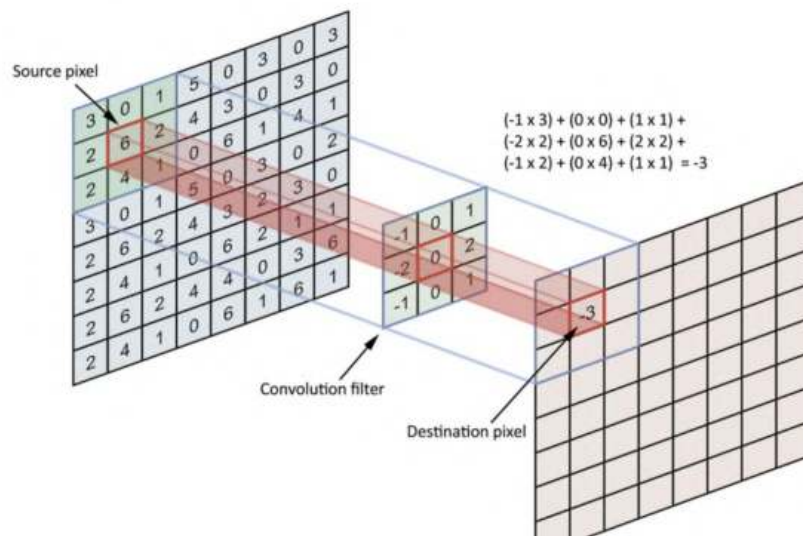
Eskiden nesne tanıma için özellikler elle tasarlanırdı, kötü yaklaşım. CNN'ler ise bu özellikleri (kenarlar, köşeler vb.) filtreler aracılığıyla otomatik olarak öğrenir



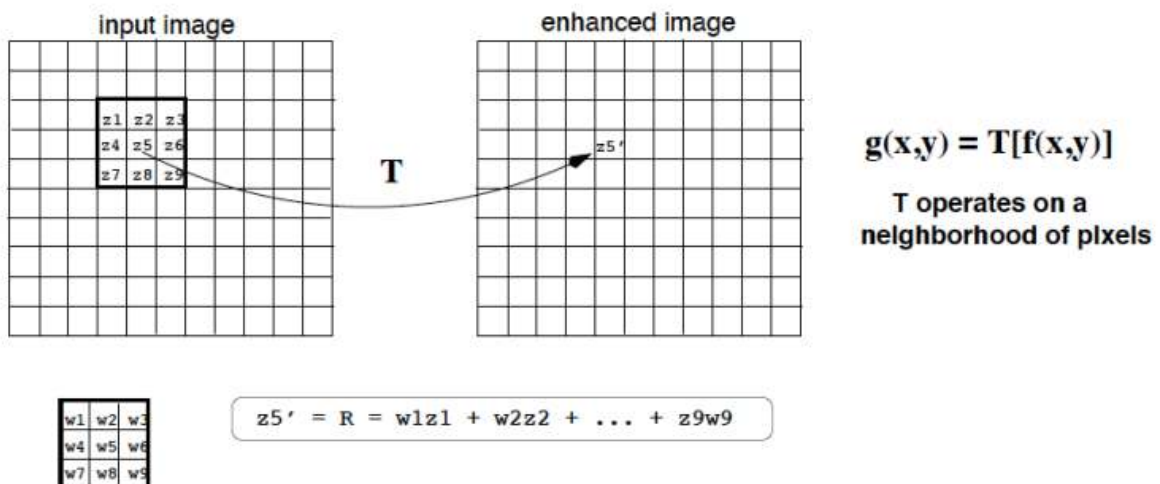
Convolution Layer

görüntü üzerinde gezen filtreler (kernels) kullanarak "Feature Map" (Özellik Haritası) oluşturur.

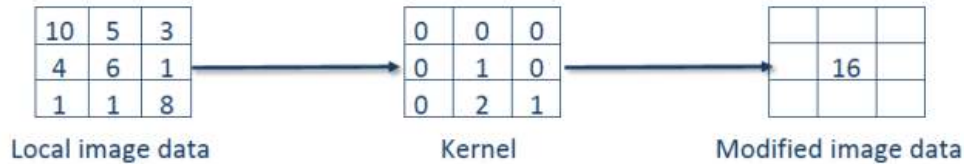
- The convolution layer (CONV) has filters that perform convolution operations



konvolüsyon işlemi



Applying a kernel to an image



$$R = w_1 z_1 + w_2 z_2 + \dots + w_{mn} z_{mn} = \sum_{i=1}^{mn} w_i z_i$$

$$R = w_1 z_1 + w_2 z_2 + \dots + w_9 z_9 = \sum_{i=1}^9 w_i z_i$$

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

Where the w 's are mask coefficients, the z 's are the value of the image gray levels corresponding to those coefficients

Applying a kernel to an image

Averaging filter

$$F(x, y) * H(u, v) = G(x, y)$$

$\frac{1}{9}$

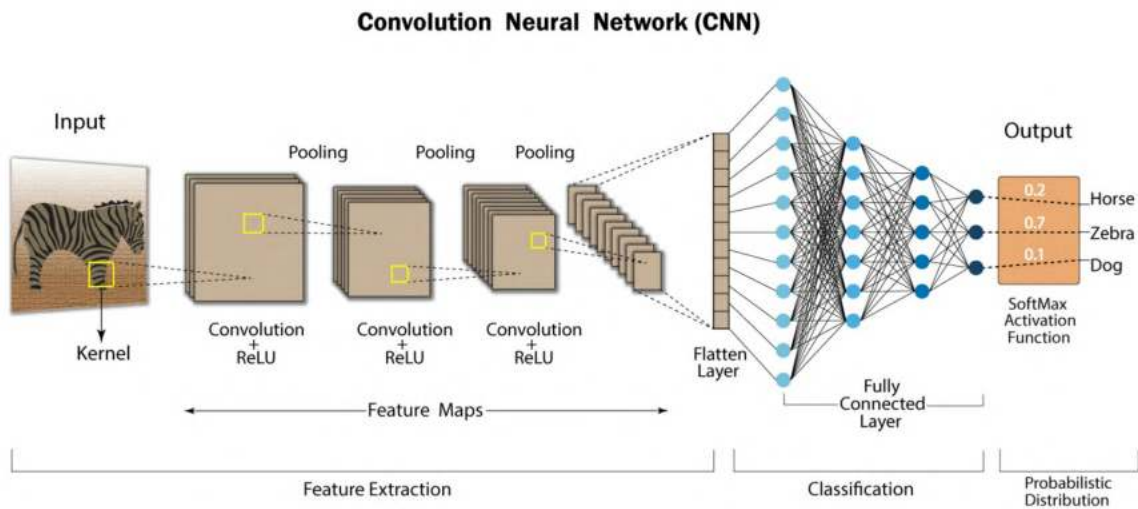
"box filter"

$$G = F * H$$

--

texture analizleri, filtre bankları (gabor vs)

- [Genel CNN Mimarisi](#)



Convolution Layer

Description:

- Learnable filters (e.g. edge detector) organized in feature maps
- Each filter scans the image and detects a specific pattern
- Convolution refers to the spatial dimensions
- Input and outputs channels are still fully connected

bazı filtreler

Edge detection



$$* \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

$$=$$


Kernel
↙

Sharpen



$$* \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

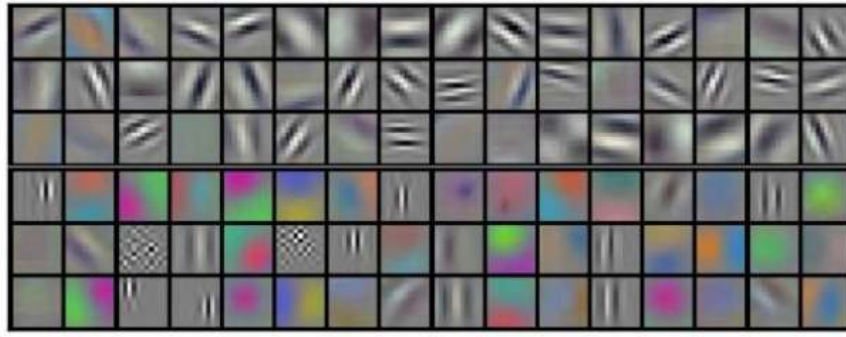
$$=$$


Hyper Parameters

farklı filtrelerle tecrübeye yönelik kendi problemine uygun denemeler yapmak için değiştirilen değerler

- depth – number of filters (also known as kernels)
- size – dimension of the filter e.g. 3×3 or $3 \times 3 \times 4$
- stride – step size while sliding the filter through the input
- padding – behavior of the convolution near the borders

The first Convolution Layer



96 convolutional kernels of size $11 \times 11 \times 3$ learned by the first convolutional layer on the $224 \times 224 \times 3$ input images.

The top 48 kernels were learned on GPU1 while the bottom 48 kernels were learned on GPU2

Looks like Gabor wavelets, ICA filters...

Çıktı boyutu

Eğer 8×8 bir görüntüye 3×3 filtre uygularsanız, sonuç 6×6 olur. Bu durum, her katmanda görüntünün doğal olarak küçüldüğünü gösterir.

If the input image size is $n \times n$ and filter size is $f \times f$:
after convolution, the size of the output image is:
(Size of input image – filter size + 1)

X

(Size of input image – filter size + 1)

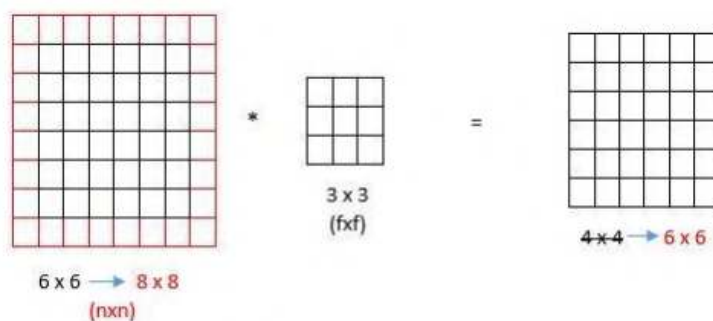


Fig: Padding the input image

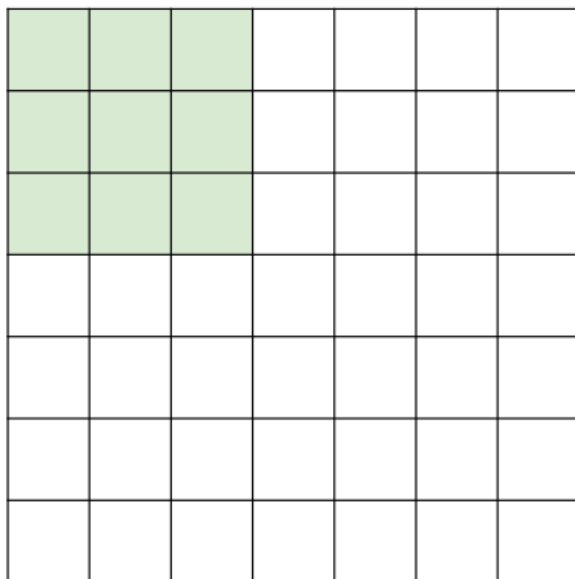
Görüntünün küçülmesini engellemek veya kenar piksellerini daha iyi işlemek için kullanılan teknikler:

Convolution Layer - Padding

- There are two common choices for padding:
 - 1. Valid convolutions:** This Means no padding. Thus, in this case, we might have $(n \times n)$ image convolve with $(f \times f)$ filter & this would give us :
 $(n-f+1) \times (n-f+1)$ dimensional output.
 - 2. Same convolutions:** In this case, padding is such that the output size is the same as the input image size. When we do padding by 'p' pixels then, size of the input image changes from $(n \times n)$ to $(n + 2p - f + 1) \times (n + 2p - f + 1)$.

2. → Çıktı boyutunun giriş boyutuyla aynı kalması için görüntünün etrafına piksel (genellikle sıfır) eklenmesi

Convolution Layer



Input: 7x7

Filter: 3x3

In general: Problem: Feature
Input: W maps "shrink"
Filter: K with each layer!
Output: $W - K + 1$

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Input: 7x7

Filter: 3x3

Output: 5x5

In general:

Input: W

Filter: K

Output: $W - K + 1$

Problem: Feature maps "shrink" with each layer!

Solution: **padding**

Add zeros around the input

In general:

Input: W

Filter: K

Padding: P

Output: $W - K + 1 + 2P$

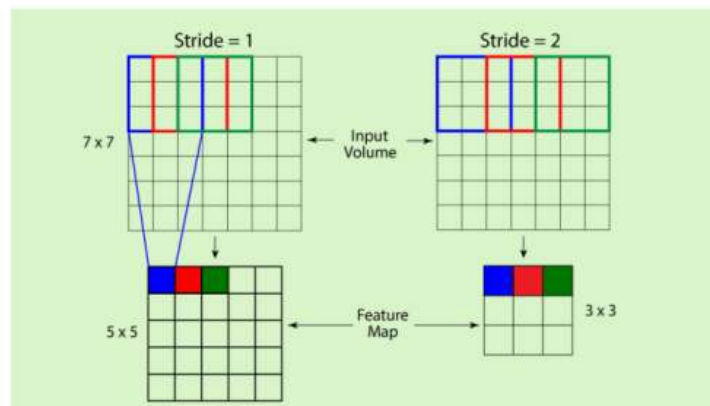
Very common:

Set $P = (K - 1) / 2$ to make output have same size as input!

Convolution Layer - Stride

Filtrenin görüntü üzerinde kaç piksel atlayarak ilerleyeceğini belirleyen kavram: **Stride**

- If stride is set to 1, filter moves across 1 pixel at a time and if stride is 2, filter moves 2 pixels at a time.
- the smaller the steps, the more details will be reflected in the resulting feature map

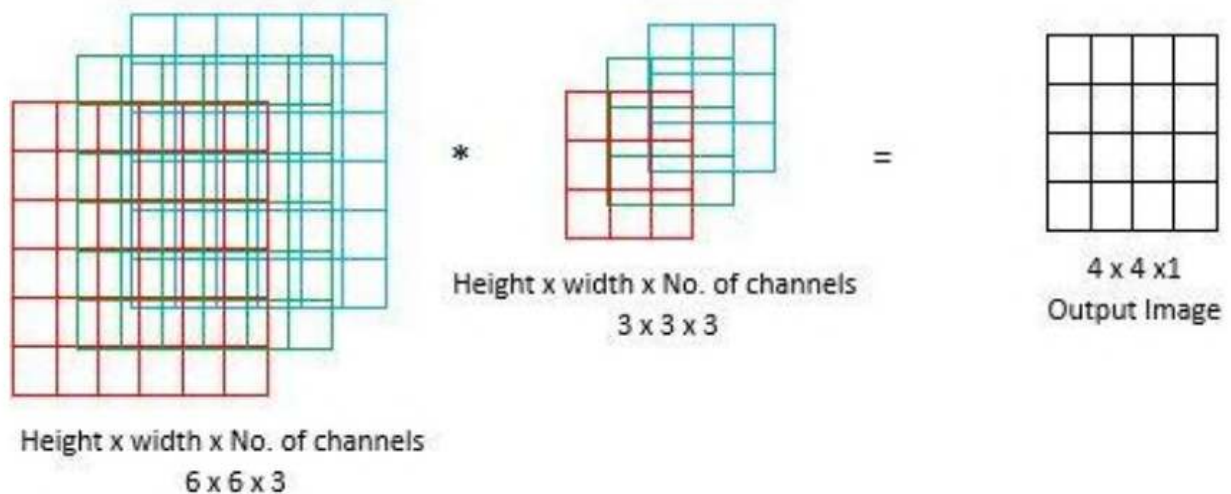


Eğer çıktı haritasında ince detayların (fine-grained) kalması

isteniyorsa küçük stride; eğer hesaplama maliyetini düşürmek ve genel (macro) özellikleri görmek isteniyorsa büyük stride seçilir

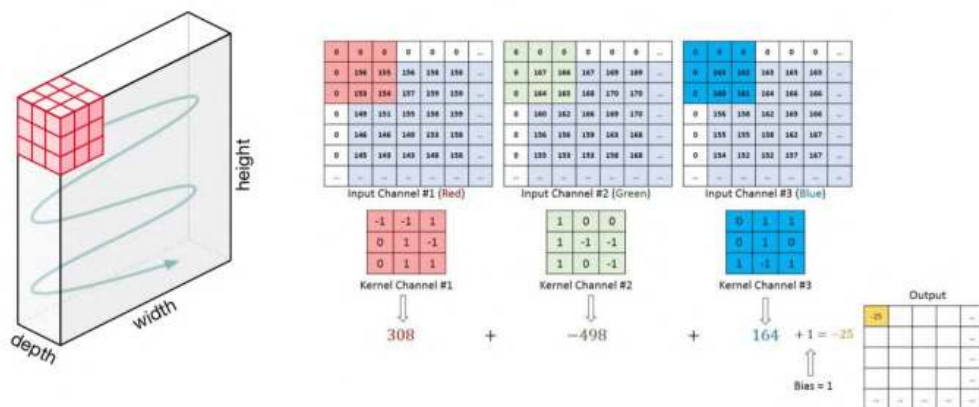
RGB image Konvolüsyonu

Convolution over RGB images

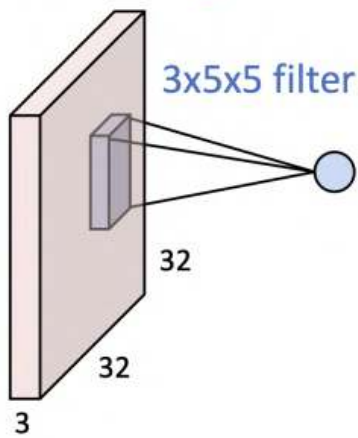


How does this work if there is more than one input channel?

- Usually, use a 3 dimensional **tensor** as the kernel to combine information from each input channel



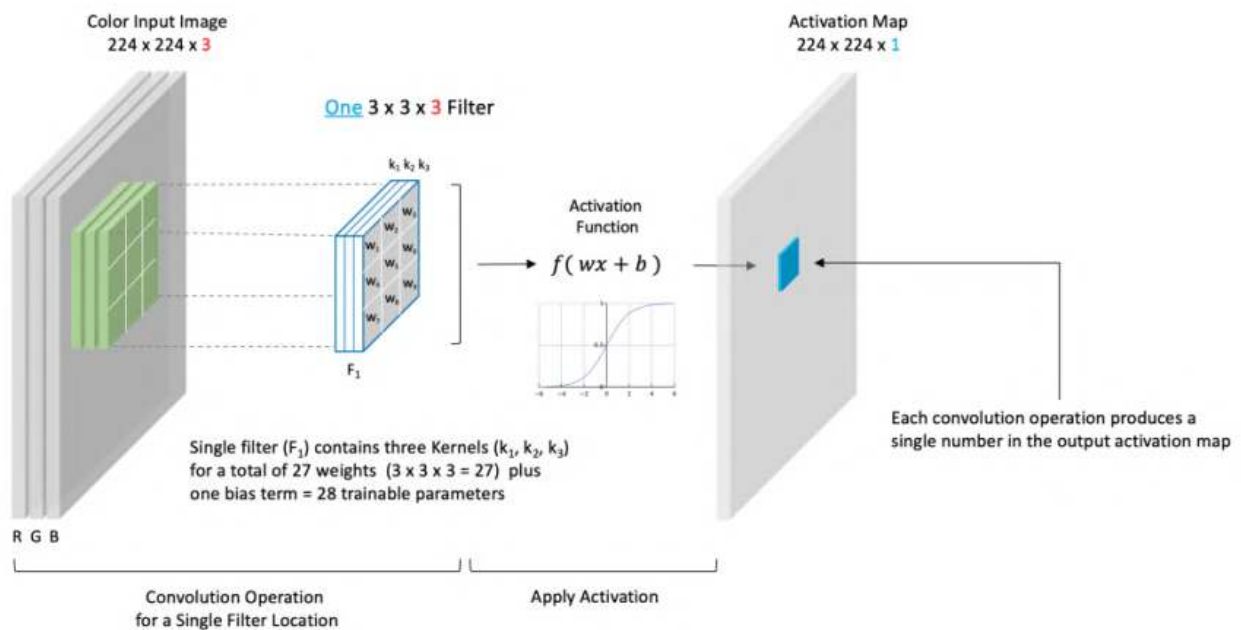
3x32x32 image



1 number:

the result of taking a dot product between the filter and a small 3x5x5 chunk of the image
(i.e. $3 \times 5 \times 5 = 75$ -dimensional dot product + bias)

$$w^T x + b$$



Convolution Layer – One Filter

Convolution Layer

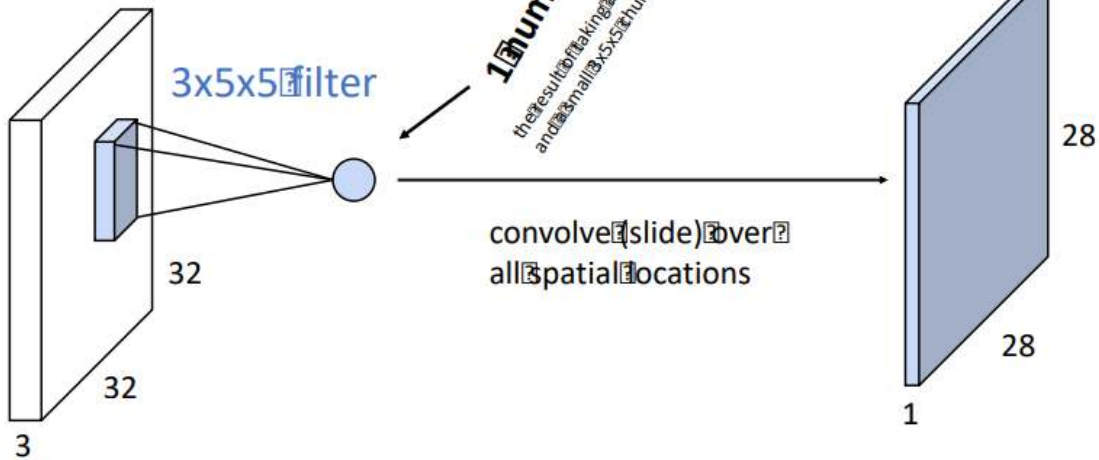
3x32x32 image

3x5x5 filter

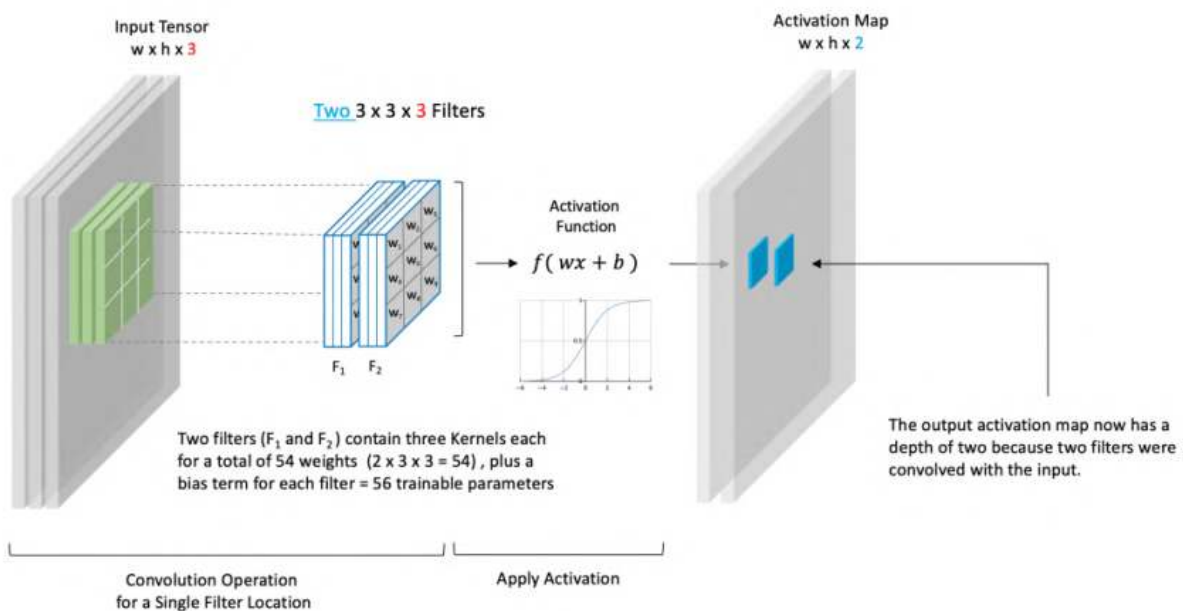
1 number

convolve (slide) over all spatial locations

1x28x28 activation map



Convolution Layer – Two Filters



S

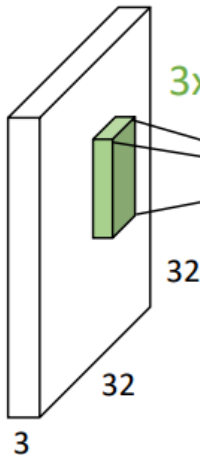
Convolution Layer – Two Filters

Convolution Layer

3x32x32 image

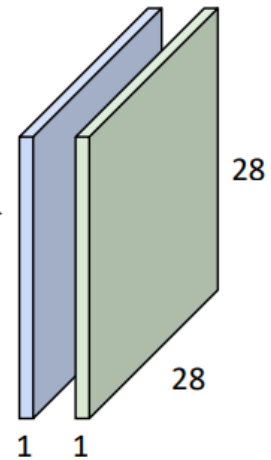
Consider repeating with a second (green) filter:

3x5x5 filter



convolve (slide) over all spatial locations

two 1x28x28 activation maps

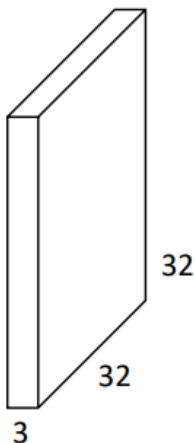


Stacking (İstifleme): Farklı filtrelerden gelen çıktıların birleştirilerek yeni bir derinlik oluşturmaktır. CNN'lerin ilerleyen katmanlarında "görüntü" artık piksellerden değil, bu istiflenmiş özellik haritalarından oluşur.

Convolution Layer

3x32x32 image

Consider 6 filters, each 3x5x5

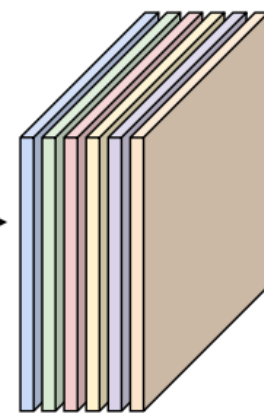


6x3x5x5 filters



Convolution Layer

6 activation maps, each 1x28x28



Stack activations to get a 6x28x28 output image!

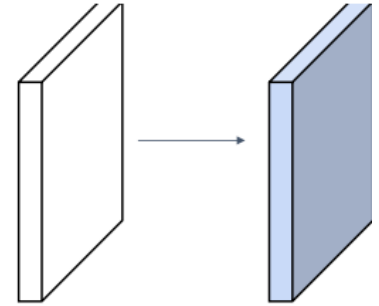
örnek

Example

Convolution Example

Input Volume: $10 \times 32 \times 32$
10 5×5 filters with stride 1, pad 2

Output Volume size:
 $(32 + 2 \cdot 2 - 5) / 1 + 1 = 32$ spatially, so
 $10 \times 32 \times 32$



In general:

Input: W

Filter: K

Padding: P

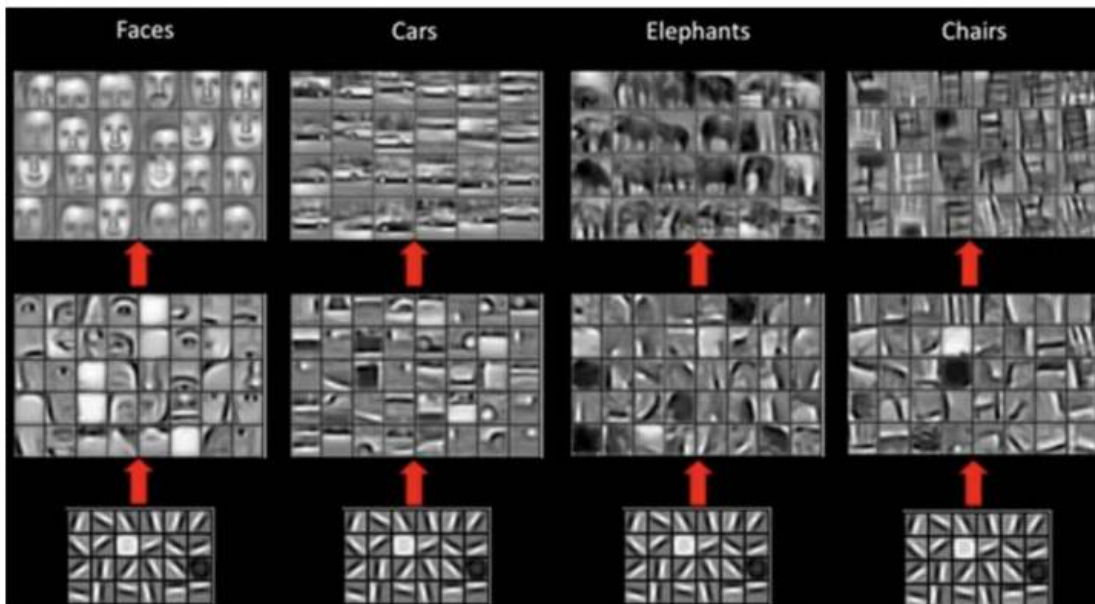
Stride: S

Output: $(W - K + 2P) / S + 1$

- W : Giriş boyutu (genişlik/yükseklik).
- K : Filtre (Kernel) boyutu.
- P : Padding (Dolgu) miktarı.
- S : Stride (Adım) miktarı.
- **Formül:** $Output = \frac{W - K + 2P}{S} + 1$.

Example of feature maps

- Examples of CNN's trained to recognize specific objects and their generated feature maps.



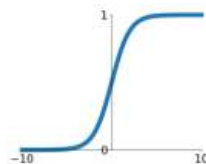
Activation functions

hatayı minimize edecek şekilde ağırlıkları değiştiriyoruz

Activation Functions

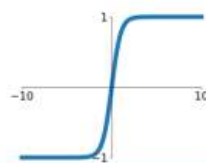
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



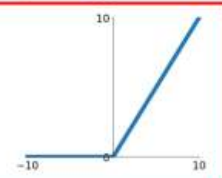
tanh

$$\tanh(x)$$



ReLU

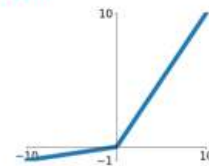
$$\max(0, x)$$



ReLU is a good default choice for most problems

Leaky ReLU

$$\max(0.1x, x)$$



Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



ReLU filtresi basit ve etkilidir, sık kullanılır

Vanishing Gradient Problem ReLU layer cart curt

o kadar üşendim ki yapay zeka metni koyuyorum buraya

Sorun Nasıl Oluşur?

Zincirleme türev kuralı gereği, hata payı en sondan en başa doğru taşınırken her katmandaki türevler birbiriyle çarpılır. Eğer kullandığın aktivasyon fonksiyonunun türevi küçükse (örneğin **Sigmoid** veya **tanh** gibi), bu küçük sayılar birbiriyle çarpıldıkça sonuç hızla sıfıra yaklaşır.

Sonuçları Nelerdir?

1. **Öğrenme Durur:** Ağın ilk katmanlarındaki ağırlıklar neredeyse hiç güncellenmez.
2. **Eğitim Süresi Uzar:** Model çok yavaş öğrenir veya belirli bir noktadan sonra hata payı hiç düşmez.
3. **Derinlik Dezavantajı:** Ağ ne kadar derinse (katman sayısı fazlaysa), bu problem o kadar şiddetli hissedilir

Nasıl Çözülür

ReLU'nun neden "iyi bir varsayılan" olduğu tam olarak burada devreye giriyor:

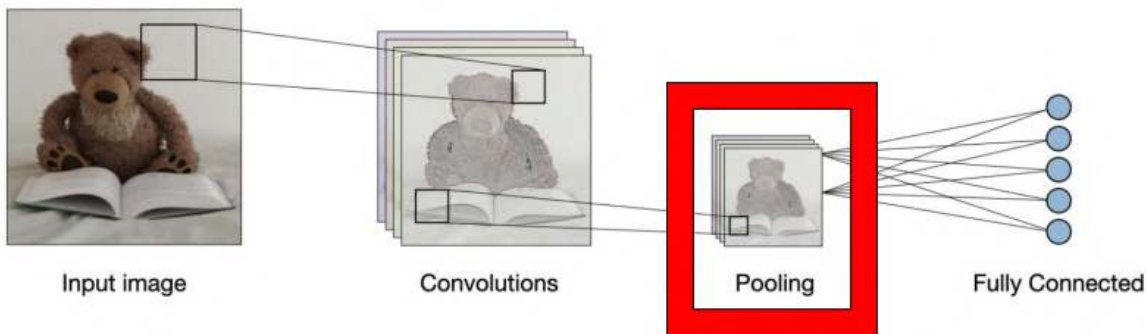
- **ReLU Kullanmak:** ReLU fonksiyonunun pozitif tarafındaki türevi her zaman 1'dir. Bir sürü 1'i birbiriyle çarpmak sonucu küçültmez, bu yüzden gradyan "kaybolmadan" en baştaki katmanlara kadar ulaşabilir.
- **Ağırlık Başlatma (Weight Initialization):** Ağırlıkları rastgele değil, *He Initialization* veya *Xavier Initialization* gibi özel yöntemlerle başlatmak.
- **Batch Normalization:** Verileri katmanlar arasında normalize ederek değerlerin aşırı küçülmesini veya büyümesini engellemek.
- **Residual Networks (ResNet):** "Skip connections" denilen yöntemle hatanın bazı katmanları atlayarak doğrudan geriye iletilmesini sağlamak.

ReLU → Vanishing Gradient Problemden fazla etkilenmez

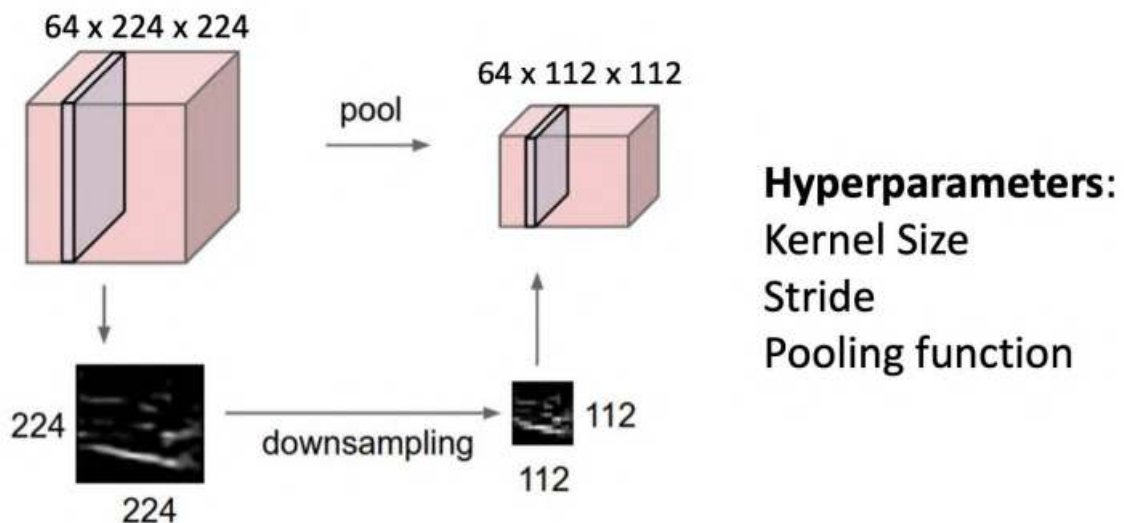
CNN Pooling

bir CNN mimarisinde veriyi "özetleme" ve "boyut düşürme" görevini üstlenen kritik bir adımdır.

- Help to reduce complexity, improve efficiency, and limit risk of overfitting.

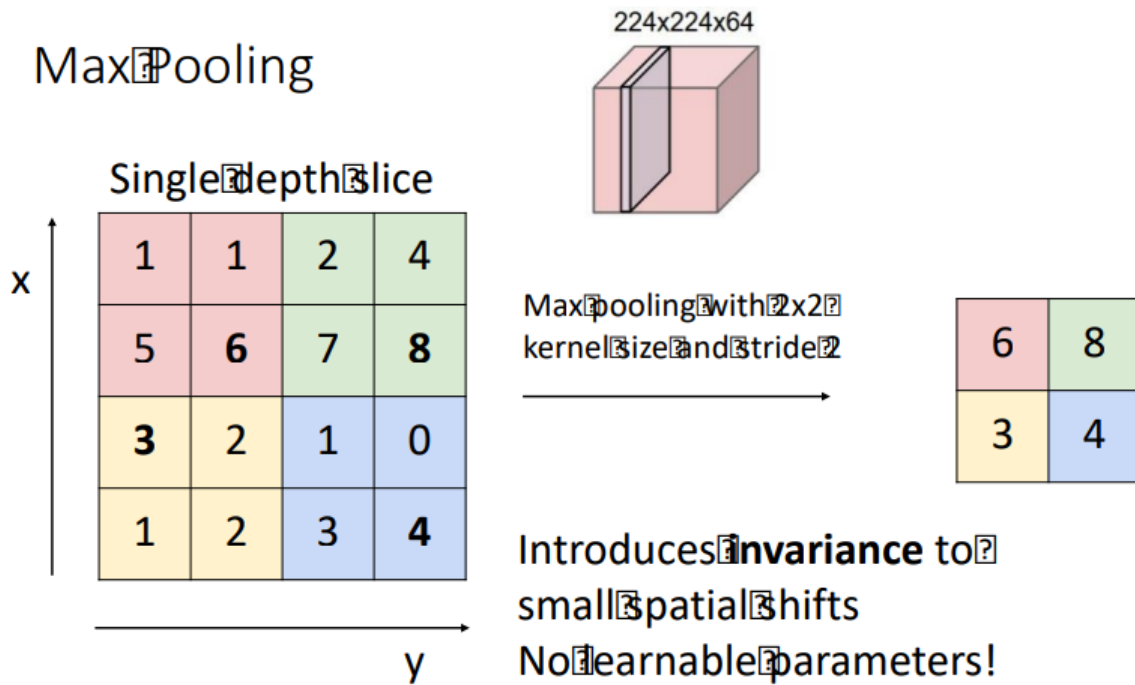


Boyut azaltmak temel görevidir (**Downsampling**)

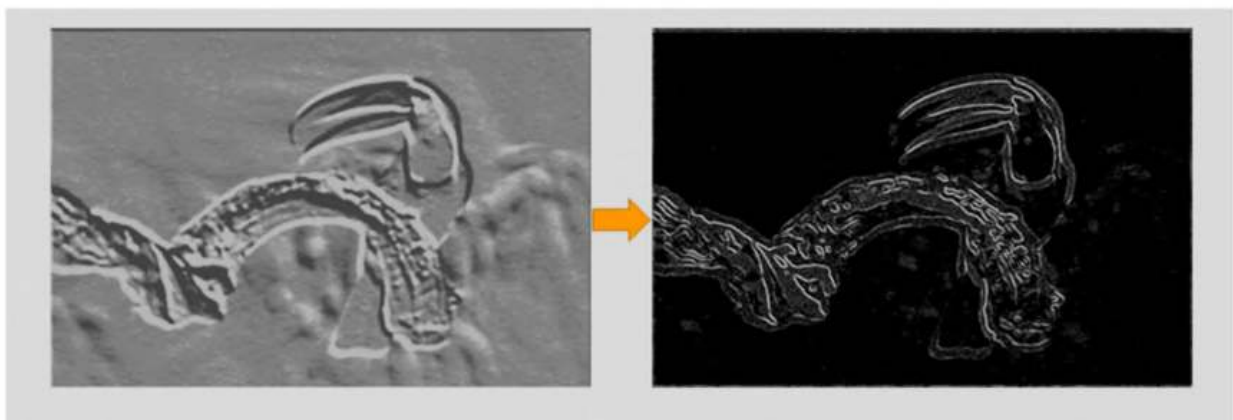


- Two main types of pooling:
 - **Max pooling**: Each pooling operation selects the maximum value of the current view.
 - **Average pooling**: Each pooling operation averages the values of the current view.

max pooling → window içinden en büyük eleman seçilir
stride 2 ve 2x2 window filtrelili örnek:

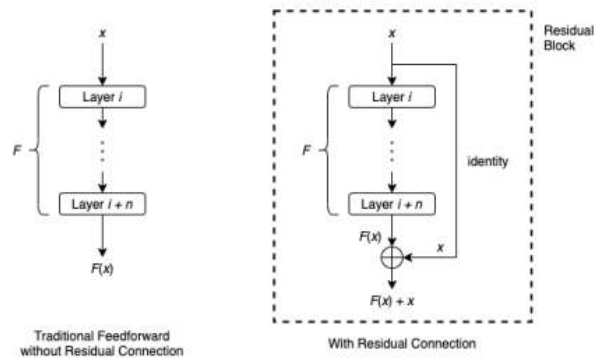


The pooling layer uses various filters to identify different parts of the image like edges, corners, body, feathers, eyes etc.



Residual Networks

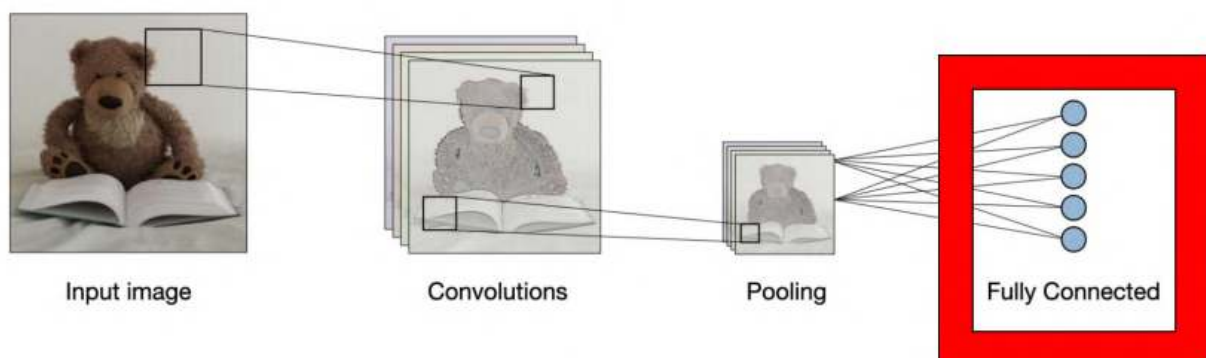
Skip the training of few layers using skip connections or residual connections



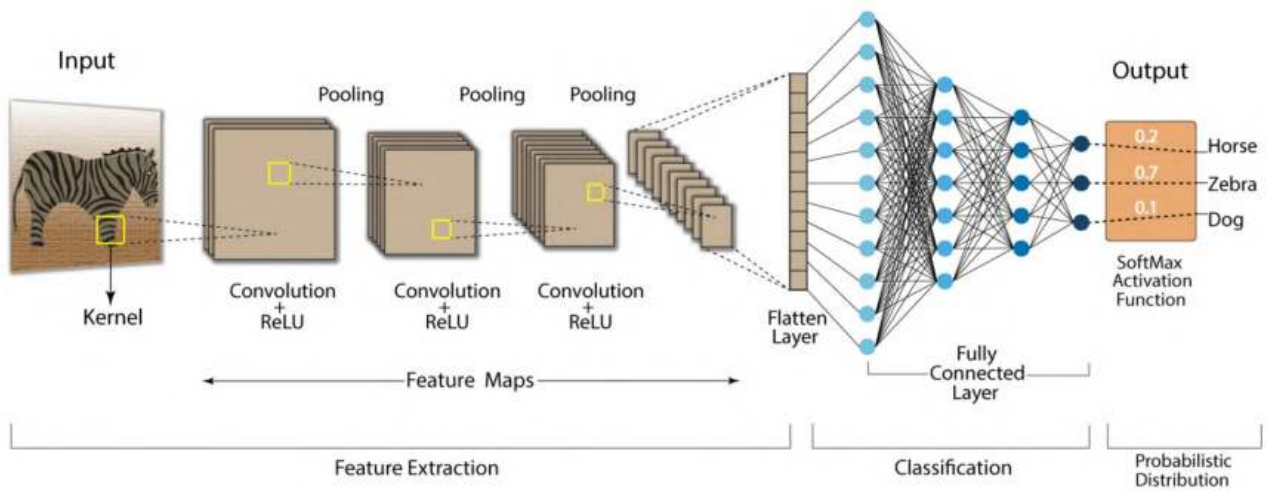
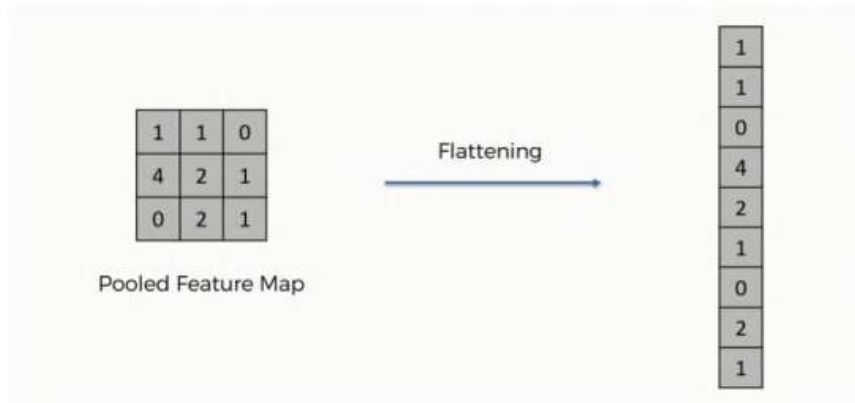
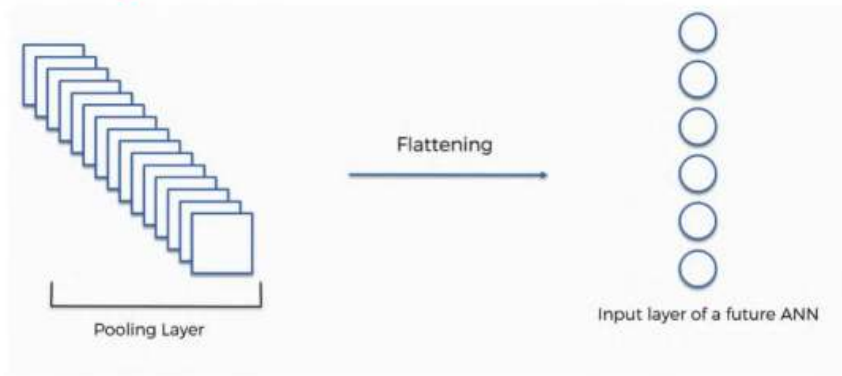
Residual connection does not resolve the exploding or vanishing gradient problems.
Rather, it avoids those problems by having shallow networks in the “ensembles”.

vanishing gradient problemini önler

CNN – Fully Connected Layer



Flatten Layer



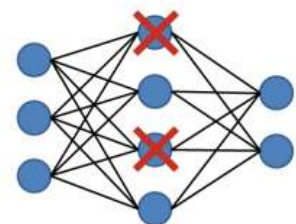
bazı ekstra şeyler vs

Batch Size

- Neural networks are trained using the stochastic gradient descent optimization algorithm.
- The number of training examples used in the estimate of the error gradient is the **batch size**.
 - **Stochastic Gradient Descent:** Batch size is set to one.
 - **Batch Gradient Descent:** Batch size is set to the total number of examples in the training dataset.
 - **Minibatch Gradient Descent:** Batch size is set to more than one and less than the total number of examples in the training dataset.

Regularization with Dropout

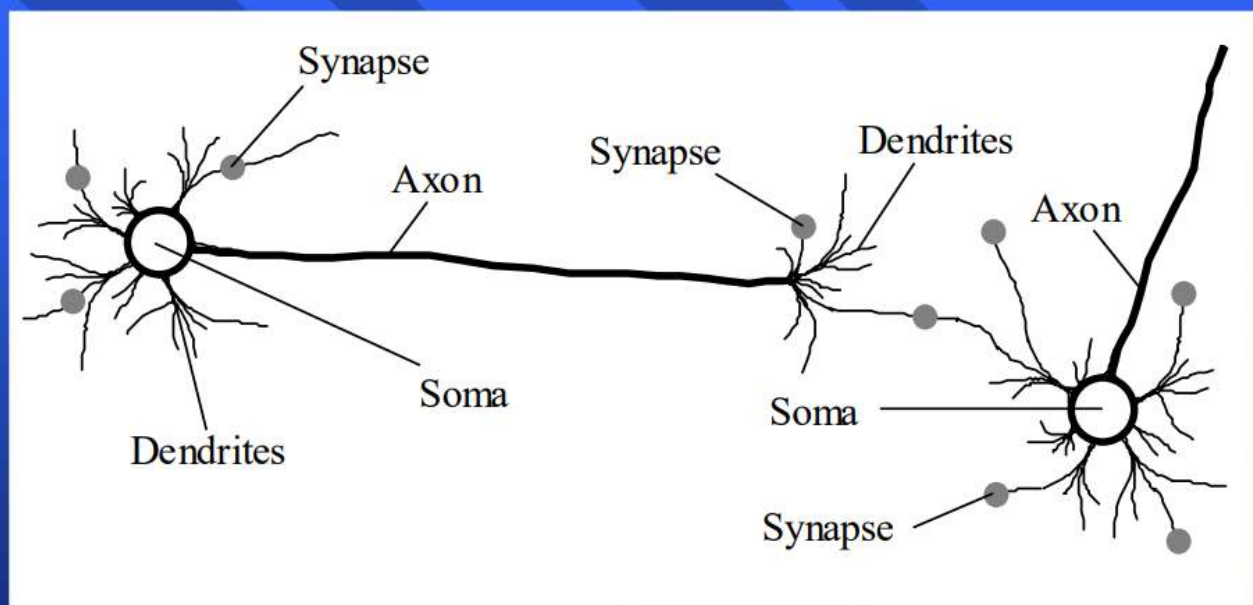
- On each iteration, we randomly shut down some neurons (units) on each layer and don't use those neurons in both forward propagation and back-propagation
- All the forward and backwards connections with a dropped node are temporarily removed, thus creating a new network architecture out of the parent network.
- The nodes are dropped by a dropout probability of p .
- Since we drop some units on each iteration, this will lead to smaller network which in turns means simpler network (regularization).
- Force the learning algorithm to spread out the weights and not focus on some specific features (units)



Artificial Neural Networks

- A **neural network** can be defined as a model of reasoning based on the human brain. The brain consists of a densely interconnected set of nerve cells, or basic information-processing units, called **neurons**.

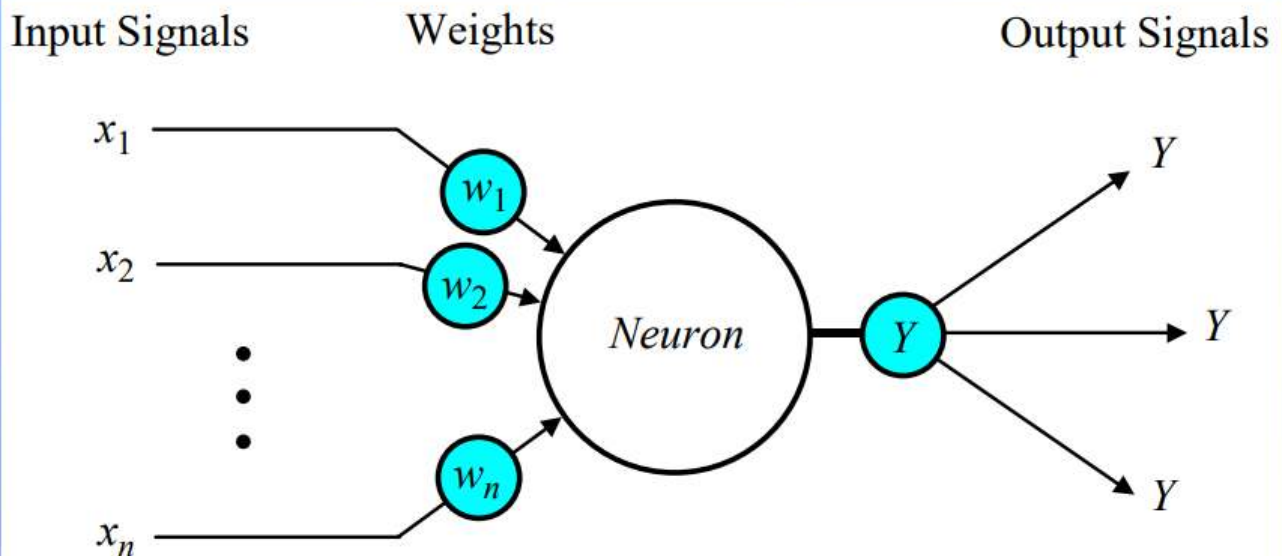
Biological neural network



Analogy between biological and artificial neural networks

<i>Biological Neural Network</i>	<i>Artificial Neural Network</i>
Soma	Neuron
Dendrite	Input
Axon	Output
Synapse	Weight

Diagram of a neuron



Activation functions of a neuron

Step function	Sign function	Sigmoid function	Linear function
$Y^{step} = \begin{cases} 1, & \text{if } X \geq 0 \\ 0, & \text{if } X < 0 \end{cases}$	$Y^{sign} = \begin{cases} +1, & \text{if } X \geq 0 \\ -1, & \text{if } X < 0 \end{cases}$	$Y^{sigmoid} = \frac{1}{1 + e^{-X}}$	$Y^{linear} = X$

Perceptron → En basit yapay sinir ağı modelidir. Verileri sınıflandırmak için kullanılır ama kapasitesi sınırlıdır.

"Perceptron hangi problemleri çözebilir, hangilerini çözemez?"

- **AND ve OR gibi doğrusal olarak ayrılabilen (linearly separable)**

problemleri çözebilir. Ancak Exclusive-OR (XOR) problemini çözemez.**

- **Aktivasyon Fonksiyonunun Amacı:**
 - Gelen sinyallerin ağırlıklı toplamını alıp, belirli bir eşik değerini (threshold) geçip geçmediğine bakarak nöronun "ateşlenip ateşlenmeyeceğine" (-1 veya +1) karar verir.

How does the perceptron learn its classification tasks?

This is done by making small adjustments in the weights to reduce the difference between the actual and desired outputs of the perceptron. The initial weights are randomly assigned, usually in the range $[-0.5, 0.5]$, and then updated to obtain the output consistent with the training examples.

konunun kalanı önemsiz ve not almaya üşendim

kalan her şey SEMİNER

Dersin geri kalanında her hafta gelen misafirlerin kendi alanlarında verdikleri sunumlar var

kalan her şey SEMİNER

Haftalık derse gelen misafirlerin kendi alanlarında verdikleri sunumlar.

SEMİNER 1: MONOLİTİK CEHENNEMDEN MODÜLER GELECEĞE



Konu: Mikroservis Mimari Desenleri ve Dönüşüm Stratejileri

Sunum Tarihi/Kişi: 08.04.2026 - Göksel Biricik

Monolitik Yapı (Örn: Food to Go - Hexagonal)

- Sistem, mobil uygulamalar tarafından tetiklenen monolitik bir yapıdır.
- Dış Entegrasyonlar: Bulut servisleri ve veritabanlarına adaptörler aracılığıyla bağlanır (Twilio adaptörü, Stripe adaptörü, Amazon SES adaptörü, MySQL adaptörü).

Monolitik Cehennemin Belirtileri

Monolitik yapılarda Kullanıcı Arayüzü, İş Mantığı ve Veri Erişimi

birbirine dolanmış (tangled), kırılğan (fragile) ve yüksek bağımlılığa (high coupling) sahip bir hale gelir.

- **Yavaş Teslimat:** Küçük bir değişiklik bile tüm uygulamanın yeniden test edilmesini ve dağıtılmasını gerektirir.
- **Kırılğan Sürümler:** Kod tabanındaki "spagetti" yapı nedeniyle, bir modüldeki hata tüm sistemi çökertebilir.
- **Ölçeklenme Darboğazı:** Yüksek CPU gerektiren bir modül ile yüksek bellek gerektiren modül aynı donanımda ölçeklenmek zorundadır.
- **Teknoloji Kilidi:** Eski ve hantal teknoloji yığınınına (legacy stack) mahkumiyet yaşanır; yeni bir framework'e geçiş imkansızla yakındır.

Paradigma Değişimi: Ölçek Küpü (The Scale Cube)

- **X-Eksen (Yatay Klonlama):** Aynı uygulamanın birden fazla kopyasını çalıştırmaktır. Bir yük dengeleyici (load balancer) ile istekler bu kopyalara dağıtılır. Kapasiteyi artırır, ancak karmaşıklığı çözmez.
- **Z-Eksen (Veri Bölümleme):** İstekleri kullanıcı ID'sine göre yönlendirmektir. Dev veri hacmini yönetir. Her uygulama örneği (instance) kullanıcıların belirli bir alt kümesinden sorumludur.
- **Y-Eksen (Mikroservisler / Fonksiyonel Ayırıştırma):** Uygulamayı, her biri tek bir iş odağına sahip bağımsız servislere (küçük binalara) bölmektir. Mikroservislerin gerçek tanımı boyutu değil, bu ayırıştırma. Her servis genelde X-ekseni ve Z-ekseni kullanılarak ölçeklendirilir.

API Gateway ve Servis İzolasyonu

- Mobil uygulamalardan (istemcilerden) gelen istekler, API Gateway (API Ağ Geçidi) üzerinden ilgili servislere yönlendirilir.
- Servisler, iş yeteneklerine veya Domain-Driven Design (DDD) alt alanlarına göre kurgulanır.

- Her servisin kendi API'si vardır ve bir servisin verisi tamamen kendisine özeldir (private).

Mimari Evrim: Hangi Durumda Ne Kullanmalıyız?

Özellik	Monolit	SOA (Service Oriented Architecture)	Mikroservisler
İletişim (Communication)	Dil seviyesinde metod çağrılar	Akıllı borular (Smart pipes, ESB, SOAP)	Aptal borular (Message broker, REST/gRPC)
Veri Yönetimi (Data)	Tek bir global veritabanı	Global veya paylaşılan veritabanları	Her servisin kendi özel veritabanı
Dağıtım (Deployment)	Tek bir büyük paket (WAR/JAR)	Büyük ve entegre servisler	Bağımsız dağıtılabilir konteynerler
Hata İzolasyonu (Failure Isolation)	Zayıf (Bir sızıntı sistemi çökertir)	Orta	Yüksek (Sadece ilgili servis etkilenir)

Mikroservislerin Stratejik Avantajları

- **Bağımsız Dağıtım:** CI/CD hatları her servis için izoledir. Tek bir servisteki güncelleme dakikalar içinde canlıya alınır.
- **Hata İzolasyonu:** Bir servisteki bellek sızıntısı veya çökme, diğer servislerin çalışmasını etkilemez; sistem ayakta kalır.
- **Teknoloji Çeşitliliği:** Her servis kendi iş problemine en uygun programlama dili ve veritabanı ile yazılabilir.
- **Otonom Takımlar:** Conway Kanunu'na uygun olarak, iki pizzayla doyabilecek küçük takımlar kendi servislerinin uçtan uca sahibidir.

Gizli Maliyetler: Mikroservislerin Zorlukları

- **Dağıtık Sistem Karmaşıklığı:** Metot çağrıları ağ çağrılarına dönüşür. Gecikme ve kısmi hatalar artık günlük bir gerçektir.
- **Dağıtık Veri Tutarlılığı:** Her servisin kendi veritabanı olması, geleneksel ACID transaction'larını imkansız kılar.
- **Sınırları Doğru Çizmek:** Sistemi yanlış yerlerden bölmek, monolitik cehennemden daha kötü olan "Dağıtık Monolit" yaratır.
- **Operasyonel Yük:** Yüzlerce küçük uygulamanın izlenmesi, loglanması ve dağıtılması ileri düzeyde DevOps kültürü gerektirir.

Mikroservis Ayırıştırma Süreci Adımları

Başlangıç noktası her zaman gereksinimlerdir (örneğin kullanıcı hikayeleri / user stories).

1. **Sistem Operasyonlarını Belirle:** Sistem operasyonları dışarıdan gelen istekleri temsil eder (Örn: müşterinin `createOrder()` isteği veya restoranın `acceptOrder()` isteği).
2. **Servisleri Belirle:** Operasyonları yönetecek mantıksal servisleri grupta.
3. **Servis API'lerini ve İşbirliklerini Tanımla:** Servislerin birbirleriyle nasıl konuşacağını ve hangi verileri paylaşacağını belirle.

1. İş Yeteneğine Göre Ayırıştırma (Business Capability)

- **Prensip:** İşletmenin değer üretmek için yaptığı temel eylemleri belirle ve her bir yeteneği bir servise dönüştür.
- **Kural:** Teknik kaygılara göre değil, *iş süreçlerine* göre böl.
- **Sonuç:** İş diliyle konuşan, iş birimleriyle doğrudan eşleşen ve organizasyonel yapıyı yansıtan servisler elde edilir.
- **Örnek Servisler:**

- Sipariş Alma → **Order Service**
- Muhasebe ve Faturalandırma → **Accounting Service**
- Teslimat Yönetimi → **Delivery Service**
- **Önemli Not:** Farklı iş kolları farklı servisler gerektirir. (Örn: Kuryeler ve restoranlar tamamen farklı tedarikçi tipleridir, bu yüzden **Courier Service** ve **Restaurant Service** olarak ayrılmalıdır. Ödemeler ve faturalandırma ise başlangıçta tek bir **Accounting Service** altında tutulabilir).

2. Alt Alana Göre Ayırıştırma (Subdomain / DDD Yaklaşımı)

- **Sınırlı Bağlam (Bounded Context):** Her alt alanın kendi Ortak Dili (Ubiquitous Language) ve kendi sınırları vardır.
- **Tanrı Sınıflarını (God Class) Yok Et:** Tüm sistemi kapsayan devasa ve tek bir **User** veya **Order** tablosu olamaz.
- **Veri Modelini Parçala:** Her Bounded Context'in (Sınırlandırılmış Bağlam) kendi otonom veri modeli olmalıdır.
 - **Örnek (Sipariş Kavramı):** "Sipariş" kavramı Sipariş Servisi (Order Service) için iptal edilebilir eylemler bütünü iken; Teslimat Servisi (Delivery Service) için sipariş sadece "bir adres ve paket ağırlığı"ndan ibarettir.

Ayırıştırmanın Önündeki Engeller

Monolitik yapıdan mikroservislere geçerken (veya sistemi bölerken) karşılaşılan başlıca zorluklar:

1. **Ağ gecikmesi (Network Latency)**
2. **Veri tutarlılığı gereksinimi (Data Consistency)**
3. **"God Class" (Hantal Sınıf) problemi:** (Örn: Kargo, fatura, sipariş iptali, kullanıcı bilgisi gibi her şeyin tek bir **Order** nesnesi veya tablosu içine yığılması).

Servisler Arası İletişim Stilleri (Etkileşim Modelleri)

İletişim Türü	Bire-Bir (One-to-One)	Bire-Çok (One-to-Many)
Senkron (Synchronous)	Request/Response (İstek/Yanıt)	-
Asenkron (Asynchronous)	Asynchronous Request/Response	Publish/Subscribe (Yayınla/Abone Ol)
	One-way Notifications (Tek Yönlü Bildirimler)	Publish/Async. Responses

Senkron İletişim (REST/gRPC) ve Tehlikeleri

- **İşleyiş ve Avantaj:** İstemci, bir servise istek gönderir ve anında yanıt bekler. İletişim o an gerçekleşmek zorundadır. Geliştirilmesi, izlenmesi ve anlaşılması kolaydır.
- **Büyük Risk (Zamana Bağlılık / Blokasyon):** Akıştaki servislerden biri çökerse çağıran servis de yanıt beklediği için bloke olur.
- **Erişilebilirlik Düşüşü:** Eğer 3 servis arka arkaya senkron konuşuyorsa ve her birinin uptime'ı (ayakta kalma süresi) %99 ise, toplam sistem erişilebilirliği matematiksel olarak %97'ye düşer.

Hata İzolasyonu: Circuit Breaker Deseni (Devre Kesici)

Arka arkaya başarısızlık veya zaman aşımı (timeout) belirli bir eşiği geçerse şalter atar ve bekleyen istekler reddedilerek kaynakların

tükenmesi (*cascading failures - zincirleme hatalar: bir hatanın diğer servisleri de sırayla bozması*) önlenir.

- **Kapalı (Closed):** Sorun yok, akım normal bir şekilde geçiyor.
- **Açık (Open):** Şalter atmış. İstekler anında reddediliyor (*Fail Fast - Hızlı Başarısız Olma: Bekleyip sistemi yormak yerine anında hata döndürme*).
- **Yarı-Açık (Half-Open):** Deneme modu; servisin düzeliş düzelmediğini anlamak için sadece birkaç isteğe izin veriliyor.

Asenkron Mesajlaşma

- **İşleyiş:** Servisler doğrudan birbirleriyle konuşmaz. Biri aracıya (*Message Broker - Mesaj Aracısı*) bir olay bırakır (*Publisher - Yayıncı*), ilgilenen servisler bunu dinler (*Subscriber - Abone*).
- **Avantajı (Yüksek Erişilebilirlik):** İstek gönderen servis diğerlerinin ayakta olup olmadığını umursamaz; tüketici servis çökse bile mesajlar kuyrukta güvende bekler. Servis ayağa kalktığında işleme kaldığı yerden devam eder.
- **Gevşek Bağımlılık (Loose Coupling):** Servislerden birinin çökmesi diğerlerini doğrudan etkilemez.



(SORULDU)

Dağıtık Veri Yönetimi

Verinin Parçalanması ve Dağıtık İşlem Zorlukları

- **Dağıtık Sistem Gerçeği:** Her servisin kendi veritabanı vardır.
- **2PC (2-Phase Commit - İki Aşamalı Doğrulama):** Dağıtık sistemlerde tabloları aynı anda kilitler, gecikmeyi artırır ve sistemin erişilebilirliğini ciddi şekilde düşürür.
- **CAP Teoremi Sıkıntısı:** Dağıtık veri yönetimi, CAP teoreminde (*Consistency/Tutarlılık, Availability/Erişilebilirlik, Partition Tolerance/Bölünme Toleransı*) Availability'den (Erişilebilirlikten) ödün verir.

Dağıtık İşlemlerin Çözümü: Saga Deseni

- **Nihai Tutarlılık (Eventual Consistency):** Veri bir anda değil, yerel işlemler zinciri tamamlandığında tutarlı hale gelir (*ACID yerine BASE prensibi: Temelde erişilebilir, yumuşak durum, nihai tutarlılık*).
- **Telafi Edici İşlemler (Compensating TX):** Bir adım başarısız olursa, Saga sistemi geriye doğru çalışır ve önceki adımların etkilerini geri almak için özel, ters işlemler (*örn: Kredi Kartı Çekildi → İade Yapıldı*) tetiklenir.

SAGA Yönetimi: Koreografi vs. Orkestrasyon

- **Koreografi (Choreography):** Merkezi yönetici yoktur. Her servis işini bitirince bir olay fırlatır, sıradaki dinleyip harekete geçer. Gevşek bağıllık maksimize edilir ancak adım sayısı arttığında akışı takip etmek imkansızlaşır.
- **Orkestrasyon (Orchestration):** Merkezi bir kontrolcü (*Maestro*) durum makinesi (*State Machine: Sistemin o anki durumunu ve sonraki aksiyonları belirleyen yapı*) gibi çalışır ve servislere ne yapacağını söyler. Kompleks akışları görmek kolaydır ancak Orkestratörün 'Tanrı Sınıfı' olma riski vardır.

Veri Sınırlarını Çizmek: DDD Aggregates (Domain-Driven Design Kümeleri)

- **Aggregate (Küme):** Birlikte bir bütün olarak ele alınması gereken nesneler grubudur.
- **Konsistans Sınırı (Consistency Boundary):** İş kuralları bu çemberin içinde %100 garanti edilir.
- **Kural:** Dış dünyadaki bir servis, bu kümeye sadece Root ID (*Kök Kimliği*) üzerinden referans verebilir. Bu durum, veritabanları arası yabancı anahtar (*Foreign Key: Farklı tablolar/veritabanları arası doğrudan bağ*) kullanımını kesin olarak bitirir.

Domain Events (Alan Olayları) ve Event Sourcing (Olay Kaynağı)

- **Domain Event:** Geçmişte gerçekleşmiş bir iş olayının (*örn. 'Sipariş Onaylandı'*) inkar edilemez bir gerçeğidir. CQRS sistemlerini senkronize eden ve Saga'ları ilerleten temel itici güçtür.
- **Event Sourcing (Veriyi Geçmişle Saklamak):** Nesnenin sadece 'şu anki durumunu' kaydedip eskisinin üzerine yazmayı bırakma paradigmasıdır. Bunun yerine, duruma yol açan tüm olayların zaman çizelgesi (*Event Stream*) saklanır. Kusursuz bir denetim izi sunar.

Veri Sorgulama, Test ve Canlıya Geçiş

CQRS Deseni (Command Query Responsibility Segregation - Komut ve Sorgu Sorumluluklarının Ayrılması)

- Verinin yazılma (*Command*) modeli ile okunma (*Query*) modelinin çok farklı donanım ve indeks gereksinimlerine sahip olması sebebiyle birbirinden ayrılmasıdır.

- Sorgu tarafı, yazma tarafından gelen Domain Event'leri dinleyerek kendi okuma kopyasını sürekli güncel tutar.

Dağıtık Veriyi Sorgulamak: API Composition (API Kompozisyonu)

- SQL'deki JOIN işlemleri mikroservislerde yapılamaz.
- **Sorun:** API Gateway / Kompozitör üzerinden basit sorgular birleştirilebilir ancak milyonlarca satırlık filtrelemelerde bellek darboğazına ve devasa ağ trafiğine yol açar.

İstemci İzolasyonu: API Gateway ve BFF

- **BFF (Backends for Frontends - Önyüzler İçin Arkayüzler):** İstemcilerin ham mikroservis ağına doğrudan bağlanması riski yerine, her istemci türü (*Mobil, Web, IoT*) için özel optimize edilmiş uç nokta servisleri (*Gateway*) tasarlanmasıdır.

Dağıtık Sistemlerde Test Stratejileri

- **E2E (Uçtan Uca Testler):** Onlarca servisi canlıya benzer bir ortamda eşzamanlı ayağa kaldırıp test etmektir. Kırılgan, yavaş ve pahalıdır. Sadece kritik mutlu yollar (*Happy Paths: Sistemin beklendiği gibi hatasız çalıştığı ana senaryolar*) için sınırlı kullanılmalıdır.
- **Sözleşme Testleri (Contract Tests):** Odak noktasıdır; servislerin birbirini anladığından ve anlaştığı mesajlaşma şablonlarından emin olunmasını sağlar.
- **Birim Testleri (Unit Tests):** Yeşil (başarılı), hızlı ve ucuzdur.

Dağıtım ve Service Mesh (Servis Ağı)

- **Dağıtım Deseni:** Servisleri sanal makineler yerine Konteyner (*Docker*) veya Sunucusuz (*Serverless*) mimarilerle (*Orkestrasyon: Kubernetes*) paketlemek
- **Service Mesh (Envoy/Istio):** Ağ iletişimini, güvenliği ve loglamayı yazılım kodundan ayırıp direkt altyapı katmanına

devretmektir.

Monoliti Dönüştürmek: Strangler (Boğucu Sarmaşık) Deseni

- **Big Bang Felaketi:** Tüm sistemi baştan ve tek seferde yazmak felakettir.
- **Strateji:** Yeni özellikleri monolite dokunmadan bağımsız yeni servisler olarak geliştirmek. API Gateway üzerinden yeni trafiği yeni servislere yönlendirerek eski özellikleri yavaşça dışarı çıkarmaktır; böylece zamanla monolit "kurur ve yok olur".

SEMİNER 2: VSM: VALUE STREAM MAPPING IN SOFTWARE DEVELOPMENT

by Ömer Çelikkanat

Agile Coach / Trainer / Scrum Master / Team Facilitator @SIEMENS

1. Temel Kavramlar ve Tanım

- **Value Stream Mapping (VSM):** Bir ürün ailesinin ham maddeden sevkiyata kadar olan akışını haritalamak, ardından israfı belirlemek ve gelecekteki durumu iyileştirmek için zamanlama ve envanter verilerini ekleme işlemidir.
- **Temel Amaç:** Yerel optimizasyon (tek bir makineyi düzeltmek) yerine tüm akışı görmek ve en büyük gecikmeleri gidermektir.
- **Üretim Akışındaki Adımlar:** Ham madde, Kesim, Montaj, Kalite/Paketleme ve Sevkiyat.
- **Haritaya Eklenen Veriler:** Çevrim süresi (cycle time), bekleme süresi, değişim süresi, envanter/WIP ve bilgi devir teslimleri.

2. Neden VSM Önemlidir?

- **İsraf Krizi:** Tipik iş faaliyetlerinin %80-90'ı israftır ve müşteri değeri eklemeyiz (PwC araştırması).

- **Gizli Verimsizlikler:** Fikirden üretime kadar olan darboğazları, devir teslimleri ve kaynakları tüketen gecikmeleri ortaya çıkarır.
- **Odak Noktası:** Ekiplerin değer sunmaya odaklanmasını sağlar.

3. Kökeni: Yalın Düşünce (Lean Thinking)

- **Kaynak:** Toyota Production System (TPS).
- **Misyon:** İsrafi (Muda) ortadan kaldırmak ve değer akışını optimize etmektir.
- **Yazılımda VSM:** Bilgi işçiliğinde gecikmeler genellikle devir teslimlerde, geri bildirim döngülerinde ve görünmez bekleme sürelerinde oluşur.

4. Yazılımda Değer Akışı (Value Stream) Bileşenleri

1. **Initial Idea (İlk Fikir):** İş ihtiyacı veya müşteri talebiyle başlar.
2. **Cross-Functional Work (Çapraz Fonksiyonel Çalışma):** Departmanlar arası insanlar, süreçler, araçlar ve bilgi akışları.
3. **Working Software (Çalışan Yazılım):** Kullanıcılara teslim edilen ve gerçek değer yaratan yazılım.

5. Beş Temel Yalın (Lean) Prensipli

1. **Identify Value (Değeri Tanımla):** Müşterinin neye ihtiyaç duyduğunu ve ne için ödeme yapacağını anlayın.
2. **Map the Value Stream (Değer Akışını Haritala):** Uçtan uca süreçteki her adımı ve karar noktasını görselleştirin.
3. **Create Flow (Akış Yarat):** Kesintisiz ilerleme için engelleri ve gecikmeleri kaldırın.
4. **Establish Pull (Çekme Oluştur):** Spekülasyon yerine gerçek müşteri talebine göre teslimat yapın.
5. **Seek Perfection (Mükemmelliği Ara):** İsrafi yok etmek için sürekli iyileştirme yapın.

6. Yazılımdaki Yaygın 5 İsrar (Waste)

- **Waiting (Bekleme):** Onaylar, kod incelemeleri veya test döngülerindeki gecikmeler.
- **Overprocessing (Fazla İşleme):** Gereksiz dokümantasyon, toplantılar veya istenmeyen özellikler.
- **Defects (Hatalar):** Yeniden çalışmaya (rework) neden olan buglar ve kalite sorunları.
- **Handoffs (Devir Teslimler):** Ekipler arası belirsiz sorumluluklar ve iletişim boşlukları.
- **Inventory (Envanter):** Kuyruklarda biriken işler, şişkin backloglar veya yarım kalmış özellikler.

SINAVDA SORULDU

7. VSM Uygulama Yolculuğu (7 Aşama)

1. **Awareness (Farkındalık):** Ekibin VSM prensiplerini anlamasını sağlamak.
2. **Roles (Roller):** Değer akışının farklı bölümlerine sahip çapraz fonksiyonel katılımcıları bir araya getirmek.
3. **SDLC Environment (Ortam Tanımı):** Analiz edilecek Yazılım Geliştirme Yaşam Döngüsü (SDLC) sınırlarını belirlemek.
4. **Identify Connections (Bağlantıları Belirle):** Mevcut durumdaki her eylemi ve devir teslimi haritalamak.
5. **Times (Zamanları Belirle):** Uygulama süresi (iş süresi) ile bekleme sürelerini toplamak.
6. **Review (İnceleme):** Sonuçları ekiple analiz etmek ve iyileştirme alanlarını belirlemek.
7. **Action (Aksiyon):** Acı noktalarını önceliklendirmek ve gelecek durum planlarını oluşturmak.

8. Başarı İçin En İyi Pratikler

- **Çapraz Fonksiyonel Katılım:** Paydaşları erkenden sürece dahil edin.

- **Veriye Dayalı Doğrulama:** Varsayımları doğrulamak için gerçek metrikleri kullanın.
- **Yaşayan Dokümantasyon:** Süreçler geliştikçe haritaları güncel tutun. (SINAVDA SORULDU)
- **İyileştirme Kültürü:** Deneyler için psikolojik güvenliği teşvik edin.

SEMİNER 3:

Sistem 101: Mainframe'e Giriş (IBM Z)

1. Mainframe Nedir ve Temel Özellikleri

- **Tanım:** Merkezi bir mimariyle çalışan, aynı anda yüz binlerce kullanıcıya kesintisiz hizmet verebilen, standart sunuculara kıyasla donanım kapasitesi ve dayanıklılığı çok yüksek sistemlerdir.
- **Kullanım Alanı:** Genellikle büyük ölçekli ticari iş yükleri için tasarlanmışlardır ve özel işletim sistemleri barındırırlar.
- **Sektörel Etkisi:** Fortune 500 şirketlerinin %71'inde bulunur ve dünya genelindeki kredi kartı işlemlerinin yaklaşık %90'ı bu sistemler üzerinden geçer.

2. Güncel Mainframe Teknolojileri: IBM Z17 & LinuxOne 5

- **Kapasite:** 5nm mimarisiyle üretilen 5.5 GHz işlemcilere, 208'e kadar çekirdeğe ve 64 TB bellek kapasitesine ulaşabilirler.
- **Yüksek Erişilebilirlik (High Availability):** Sistem %99.999999 oranında ayakta kalır. Bu, yılda sadece milisaniyelerle ifade edilen bir kesinti anlamına gelir.
- **Yapay Zeka ve Şifreleme:**
 - Çip üzerinde (on-chip) şifreleme ve veri sıkıştırma yeteneğine sahiptir.

- Spyre AI kartı sayesinde 1 milisaniye tepki süresiyle günde 450 milyar yapay zeka çıkarımı (inference) yapabilir.
- Kuantum sonrası kriptografi (Post-Quantum Cryptography) desteği barındırır.
- **Ekstra Özellikler:** Anlık kurtarma (Recovery Boost) ve esnek kapasite artırımı (Flex Capacity) imkanları sunar.

3. İşletim Sistemleri ve Sanallaştırma

- **İşletim Sistemleri:** z/OS, z/VM, z/VSE, TPF ve çeşitli Linux dağıtımları (Redhat, Suse, Ubuntu).
- **Sanallaştırma (Hypervisor) Türleri:**
 - **Tip 1 (LPAR):** Doğrudan donanım katmanında çalışır ve EAL5+ standartlarında en üst düzey yalıtım sağlar.
 - **Tip 2 (VM Guest):** İşletim sistemi üzerinde sanallaştırma yapar (z/VM veya KVM gibi).
- **Mimari:** Yüksek erişilebilirlik için Parallel Sysplex (kümeleme) mimarisi kullanılır.

4. Mainframe Ekosistemi (Üzerinde Neler Çalışır?)

- **Ara Katman (Middleware) ve İşlem Yöneticileri:**
 - CICS ve IMS (İşlem yönetim sistemleri).
 - MQ (Kuyruk tabanlı mesajlaşma).
 - WAS (Websphere Uygulama Sunucusu).
 - Arka planda çalışan yığın işlemleri (Batch Jobs).
- **Veritabanları:**
 - **DB2:** İlişkisel veritabanıdır. Sysplex ve Data Sharing özellikleriyle yüksek erişilebilirlik ve performans sunar.
 - **IMS:** Hiyerarşik yapıdaki veritabanıdır.
- **Geliştirme Araçları:**
 - COBOL, PL/I, Assembler gibi klasik dillerin yanında Java ve Python gibi modern diller de desteklenir.

- CI/CD ve DevOps süreçleri için Git, Jenkins, Bitbucket ve VSCode entegrasyonları mevcuttur.

5. Arayüzler ve Kullanılan Terimler

- **Geleneksel Ekranlar:** TN3270 (klasik siyah ekran), TSO, ISPF, SDSF.
- **Modern Arayüzler:** Açık kaynaklı ZOWE ve z/OSMF.
- **Temel Kavramlar:**
 - **Data Set:** Mainframe'deki dosya sistemi yapısı.
 - **JCL:** İş Kontrol Dili (Job Control Language).
 - **USS:** z/OS Unix Sistem Servisleri.
 - **GDPS:** Felaket kurtarma senaryoları için kullanılan yapı.

6. Güvenlik ("Dünyanın En Güvenli Sistemi")

- **SAF (System Authorization Facility):** Sistemin temel güvenlik kontrol mekanizmasıdır (RACF gibi ürünlerle entegre çalışır).
- **Kapalı Kutu Yapısı:** Doğrudan dış dünyaya açık değildir ve "Local Admin" konsepti yoktur. Bu da log4j gibi popüler zafiyetlerden etkilenmesini önler.
- **Denetim (Audit):** Sistemde gerçekleştirilen her işlem imzalı log verileriyle kayıt altına alınabilir.
- **Donanım Destekli Koruma:** CPACF ile çip seviyesinde hızlandırılmış şifreleme, Crypto Express ile HSM (Donanımsal Güvenlik Modülü) koruması ve Secure Service Containers (SSC) ile dış müdahalelere kapalı ortamlar sağlanır.

7. Platform Tercihi ve Pratik Yapma İmkânı

- Bir sistem kurulurken donanım seçimi (x86, UNIX veya IBM Z); TCO (toplam sahip olma maliyeti), erişilebilirlik, güvenlik gereksinimleri ve kapasite ihtiyacına göre yapılır.
- Bu ekosistemi deneyimlemek isteyenler, IBM Z Xplore ve LinuxOne Community Cloud üzerinden sistemlere erişip pratik

yapabilir.

Part 2: Kuantum Hesaplamanın Temelleri

1. Temel Kavramlar

- **Kuantum Biti (Qubit):** Klasik bitler sadece 0 veya 1 değerini alırken, qubitler aynı anda hem 0 hem de 1'in süperpozisyonunda (üst üste binmiş durum) bulunabilir.
 - *Bloch Küresi (Bloch Sphere): Qubitlerin 3 boyutlu durumunu görselleştirmek için kullanılır.
 - Durum Denklemi: $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ (Ölçüm yapıldığında 0 olma olasılığı $|\alpha|^2$, 1 olma olasılığı $|\beta|^2$ 'dir).
- **Süperpozisyon (Superposition):** Bir kuantum sisteminin ölçüm yapılana kadar birden fazla durumda aynı anda bulunabilme yeteneğidir. Ölçüm yapıldığı an durum "çöker" (collapse) ve 0 veya 1 olur.
 - **Avantajı:** Devsel paralel işlem kapasitesi sağlar. N adet qubit, aynı anda 2^N adet durumu temsil edebilir.
- **Dolanıklık (Entanglement):** Einstein'ın "uzaktan ürkütücü etki" (spooky action at a distance) olarak adlandırdığı olgudur.
 - İki veya daha fazla qubit birbirine kuantum seviyesinde bağlanır. Aralarındaki mesafe ne olursa olsun, bir qubitin durumu ölçüldüğünde diğerinin durumu da anında belli olur.
 - Kuantum ışınlanma (teleportation) ve kriptografinin temelini oluşturur.
- **Interference:** Quantum computing relies on probability distributions, with wave functions that can add constructively or cancel destructively, depending on the measurement.
(BU 3 TEMEL PRENSİBİ SORULDU)

2. Kuantum Kapıları (Quantum Gates)

Klasik bilgisayarlardaki mantık kapılarının (AND, OR, NOT) kuantum karşılıklarıdır. İşlemler tersine çevrilebilirdir (unitary).

- **Hadamard (H) Kapısı:** Qubiti 0 veya 1 durumundan çıkarıp süperpozisyon durumuna sokar.
- **Pauli-X Kapısı:** Kuantum dünyasının "NOT" kapısıdır (0'ı 1'e, 1'i 0'a çevirir).
- **CNOT (Controlled-NOT) Kapısı:** Genellikle iki qubiti birbiriyle dolanık hale getirmek (entanglement) için kullanılan kontrol kapısıdır.

3. Kuantum Algoritmaları

- **Shor Algoritması:** Büyük sayıların asal çarpanlarına ayrılmasını klasik algoritmalara göre üstel olarak (exponentially) daha hızlı yapar.
 - **Etkisi:** Günümüzde bankacılık ve internette kullanılan popüler RSA şifrelemesini kırma potansiyeli taşır. Bu yüzden "Post-Kuantum Kriptografi" çalışmalarını zorunlu kılmıştır.
- **Grover Algoritması:** Yapılandırılmamış (unstructured) veritabanlarında arama yapmak için kullanılır.
 - **Hız Kazancı:** Karesel (quadratic) hızlanma sağlar. Klasik bilgisayarlarla $O(N)$ süren bir arama işlemi, Grover algoritması ile $O(\sqrt{N})$ sürede çözülür.

4. Zorluklar ve Gelecek

- **Dekoharens (Decoherence) ve Gürültü:** Qubitler çok kırılgandır. Çevresel faktörlerle (ısı, manyetik alan) etkileşime girdiklerinde kuantum özelliklerini (süperpozisyon/dolanıklık) hızla kaybederler.

- **Hata Düzeltme (Error Correction):** Kuantum Hata Düzeltme (QEC) algoritmaları şarttır, ancak hatasız çalışan "mantıksal 1 adet qubit" elde etmek için fiziksel olarak çok sayıda (binlerce) qubite ihtiyaç vardır.
- **Kriyojenik Şartlar:** Özellikle süperiletken (superconducting) qubitlerin kararlı çalışabilmesi için mutlak sıfıra yakın (-273°C) aşırı soğuk ortamlarda tutulmaları gerekir.
- **Gelecek Vizyonu:** Günümüzde NISQ (Noisy Intermediate-Scale Quantum - Gürültülü Orta Ölçekli Kuantum) çağındayız. Gelecekte hatalara karşı tam toleranslı büyük ölçekli bilgisayarlar hedeflenmektedir.

SEMİNER 4: A Cybersecurity Journey Into the Age of Intelligent Threats

Sunum Özeti ve Kapsamı

Bu sunum, Ahmet Can KAN'ın siber güvenlik alanındaki kariyer yolculuğunu 5 ana rolde (Bölüm) ele alarak, her aşamadaki teknik konseptleri, saldırı türlerini ve savunma/analiz metodolojilerini detaylandırmaktadır.

Bölüm I: Sızma Testi (Penetration Testing) Uzmanlığı

Sızma testi (Pen-test); gerçek bir saldırganı (adversary) simüle eden, yetkilendirilmiş ve zaman sınırlı bir güvenlik testidir.

- **Aşamaları:** 1. Recon (Keşif/OSINT) → 2. Zafiyet Analizi → 3. Sömürü (Exploitation/PoC) → 4. Sömürü Sonrası (Lateral movement, veri sızdırma) → 5. Raporlama.
- **OWASP Top 10:** Uygulama güvenliğindeki en kritik 10 zafiyet standartıdır.

- **SQL Injection (SQLi - A03):** Kullanıcı girdisinin doğrulanmadan doğrudan SQL sorgusuna eklenmesidir (Örn: ' **OR 1=1 --** ile yetki atlatma veya **UNION SELECT** ile veritabanı sızdırma). Çözümü: Parametrik sorgular (Prepared statements).
- **IDOR (Insecure Direct Object Reference - A01):** Yetkilendirme kontrolünün obje seviyesinde yapılmamasıdır. URL'deki ID değeri değiştirilerek (Örn: **/invoices/9001** → **9002**) başka kullanıcılara ait verilere izinsiz erişilebilir.
- **Ağ Katmanı Zafiyetleri:** Buffer overflow (bellek taşması), protokol hataları (TLS/SMB/RDP) ve yama gecikmeleri (Patch lag).
 - **Örnek Vaka - Heartbleed (CVE-2014-0160):** OpenSSL TLS/DTLS heartbeat eklentisinde gerçekleşen sınır dışı okuma (Out-of-bounds read) zafiyetidir. Saldırganın sunucu belleğinden 64 KiB'lık parçalar sızdırmasına olanak tanır (Şifreler, oturumlar ve RSA özel anahtarları ifşa olur). (SINAVDA SORULDU)
- **Temel Dersler:** Girdiye asla güvenme (Trust no input), Derinlemesine savunma yap (Defence in depth), RFC (Protokol standartları) dokümanlarını oku.

Bölüm II: Zararlı Yazılım Analizi (Malware Analysis)

Sıfırdan bir zararlıyı alıp SIEM (Güvenlik Bilgi ve Olay Yönetimi) sistemleri için tespit kuralına dönüştürme sürecidir.

- **Analiz Süreçleri:** Triage (Sınıflandırma), Statik Analiz (Ghidra/IDA ile assembly/kod inceleme), Dinamik Analiz (Cuckoo/CAPE sandbox içinde çalıştırma), Tespit Mühendisliği (YARA/Sigma kuralları).
- **Örnek Vaka - Godfather Android Banker:**

- Sahte VPN veya Google Play Protect kılığında dağıtılır.
- Cihazda "Erişilebilirlik (Accessibility)" izni alarak sistemin kontrolünü ele geçirir.
- **VirtualApp-style Evasion:** Orijinal bankacılık uygulamasını modifiye etmeden kendi oluşturduğu sanal bir kapsayıcı (container) içinde çalıştırır. Böylece antivirüsler ve Play Protect imza kontrollerini atlatır.
- Kullanıcı ekrana dokunurken araya girerek (Hooking/Overlay) şifreleri çalar.

Bölüm III: Güvenlik Değerlendirmesi ve Sertifikasyon (Security Evaluation)

Mobil ödeme uygulamalarının (PCI, EMVCo) bağımsız laboratuvarlarda tersine mühendislikle test edilmesi.

- **Tehdit Modeli (Threat Model):** Mobil güvenlikte "Cihazın kendisi (veya sahibi) bir düşmandır". (Root'lu Androidler, Jailbreak'li -checkm8- iOS cihazlar vb.).
- **Analiz Araçları:**
 - *Statik:* apktool, jadx, class-dump.
 - *Dinamik (Frida):* Uygulama çalışırken içine JavaScript motoru enjekte ederek fonksiyonları değiştirmeyi (hooking) sağlayan araçtır. SSL Pinning ve Root tespitini atlatmak için kullanılır.
- **Kriptografi ve Donanımsal Saldırıları (DFA):**
 - **AES (Advanced Encryption Standard):** 128-bit bloklar halinde çalışan simetrik şifreleme algoritmasıdır. (SINAVDA HANGİSİ SİMETRİK ŞİFELEME ALGORİTMASIDIR DİYE SORULDU)
 - **DFA (Differential Fault Attack):** Cihaz kod çalıştırırken AES'in 9. turunda (round 9) bellek yamasını manipüle ederek "hata (fault)" enjekte etme işlemidir. Bu işlem

sonucunda bozulan şifreli metinler üzerinden matematiksel hesaplama yapılarak uygulamanın Master Key'i (Ana Şifresi) ele geçirilir.

Bölüm IV: Kırmızı Takım Operasyonları (Red Team Operator)

Zafiyet bulmanın ötesinde, kurumun insanlarını, süreçlerini ve teknolojilerini test etmek için gerçek bir Siber Tehdit Aktörünü (APT) simüle etme operasyonlarıdır.

- **Siber Ölüm Zinciri (Cyber Kill Chain):** Recon → Weaponise → Deliver → Exploit → Install → C2 (Komuta Kontrol) → Actions.
- **Senaryo Örneği (Tedarik Zinciri - Supply-Chain Saldırısı):**
 - Kurumun çalıştığı üçüncü parti bir yazılımcıya sahte iş teklifi PDF'i gönderilir (Phishing).
 - VS Code eklentisi üzerinden sisteme sızılır ve ayrıcalık yükseltilir (Kerberoasting).
 - Veriler (PII) fark edilmemek için DNS tünelleme ile yavaşça (<4 KB/s) sızdırılır ve sisteme fidye yazılımı (Ransomware) atılır.
- **Purple Teaming (Mor Takım):** Kırmızı (Saldırı) ve Mavi (Savunma - SOC) takımların beraber çalışarak SIEM sistemlerindeki tespit kurallarını (MTTD - Tespit Süresi, MTTR - Müdahale Süresi) iyileştirdiği döngüdür.

Bölüm V: Tehdit İstihbaratı (Threat Intelligence)

Siber suç ekosisteminin izlenmesi ve tehdit verilerinin toplanmasıdır.

- **RaaS (Ransomware as a Service - Hizmet Olarak Fidye Yazılımı):** Siber suçun şirketleşmiş halidir.
 - **Initial Access Broker:** Sisteme sızan ve erişimi satan kişiler.

- **Crypter / Malware as a Service:** Zararlı yazılımı kodlayan ve şifreleyen geliştiriciler.
- **Affiliate:** Bu yazılımları kiralayıp hedeflere saldıran ve fidyeden pay alan alt bayiler.
- **IOC (Indicator of Compromise) Pipeline:** Torrent ağları (LockBit, Qilin vb.), Dark Web ve STIX/TAXII feed'leri üzerinden toplanan verilerin zenginleştirilerek güvenlik sistemlerine otomatik engelleyici kural (Detection Rule) olarak girilmesidir.

Kariyer İçin Özet Tavsiyeler

- Evde bir laboratuvar kurun (Home Lab: DVWA, HackTheBox, Magisk yüklü eski bir telefon).
- Her şeyi öğrenmeye çalışmak yerine başlangıçta bir alanda "derinleşin" (Web, mobil veya gömülü sistemler).
- Gerçek zafiyet raporlarını (CVE write-ups, Project Zero, Phrack) okuyun.
- Raporlamayı ciddiye alın: "Kimsenin anlamadığı bir bug, kimsenin düzeltmediği bir bug'dır."

person of interest filmini izleyin çok güzel (sunan adamın önerisi)

SEMİNER 5: Security Governance and AI Security

Sunan: Aykut Yaprak

Garanti BBVA

1. Bilgi Güvenliği ve Risk Yönetimi

- Kurumsal güvenlik üç ana uzmanlık alanına ayrılır: IT Güvenliği, Müşteri Güvenliği ve Fiziksel Güvenlik.
- IT Güvenliği temel olarak Savunma (Önleme, Tespit, Müdahale) ve Saldırı (Zafiyet Tarama, Pentest, Red Teaming) yaklaşımlarını

içerir.

- Risk değerlendirmesi "Olasılık x İş Etkisi" formülüyle yapılır.
- Risk yönetimi için 4 strateji kullanılır: Kabul (Accept), Azaltmak (Mitigate), Aktarmak (Transfer) ve Kaçınmak (Avoid).

2. Yapay Zeka (AI) ve LLM Temelleri

Yapay Zeka Ekosistemindeki Roller

AI ürünleri tek bir kişinin değil, veriden güvenliğe uzanan bir uzmanlık zincirinin ortak ürünüdür.



- Bir Makine Öğrenmesi/Derin Öğrenme modeli iki ana parçadan oluşur: Mimari (şema/kod) ve Ağırlıklar (öğrenilmiş sayısal parametreler).
- LLM çalışma mantığı şu adımlardan oluşur: Tokenization (metni parçalama), Context Window (bağlamı görme), İşleme (Self-Attention) ve Decoding (üretim).
- LLM'ler metni autoregressive olarak, yani geçmiş token'lara bakarak bir sonrakini tek tek tahmin ederek üretir.

GenAI Concepts

Teknik	Açıklama	Amaç	Örnek Kullanım
RAG (Retrieval-Augmented Generation)	Model, dış kaynaklardan (DB, arama motoru, doküman deposu) bilgi çekerek cevaba ekler.	Güncel ve doğru bilgi sağlamak	Bir şirketin dokümanlarından chatbot'un cevap üretmesi
RLHF (Reinforcement Learning with Human Feedback)	İnsan geri bildirimiyle modelin cevapları ödüllendirilip cezalandırılır.	Daha güvenli, kullanıcı dostu yanıtlar	ChatGPT'nin kaba veya yanlış cevap vermekten kaçınması
Fine-tuning	Modelin parametreleri belirli veriyle yeniden eğitilir.	Özelleşmiş alan bilgisi kazandırmak	Bankacılık chatbot'u, hukuki metin analizi
Instruction Tuning	Modeli "talimatlara uyumlu" hale getirmek için özel veriyle eğitime.	Komutları daha iyi anlamak	GPT-3'ten GPT-3.5 Instruct'a geçiş
LoRA / PEFT (Low-Rank Adaptation / Parameter-Efficient Fine-Tuning)	Modelin tamamını değil küçük bölümlerini eğitmek.	Daha hızlı ve ucuz özelleştirme	Bir e-ticaret şirketinin kendi ürün verisiyle modelini özelleştirmesi
Distillation	Büyük modelden küçük modele bilgi aktarma.	Hafif, hızlı model elde etmek	Mobil cihazlarda çalışan küçük LLM
Prompt Engineering	Eğitimi değiştirmeden akıllı prompt tasarlamak.	İstenen formatta ve doğrulukta çıktı almak	"Sen bir hukuk danışmanısın, şu davayı analiz et..."
Agentic AI	Modelin araç kullanabilmesi (arama, API çağırısı, kod çalıştırma).	Karmaşık görev çözme	Kod yazarken hata bulup test çalıştırması
Memory-Augmented GenAI	Modelin önceki konuşmaları veya kullanıcı bilgisini hatırlaması.	Daha kişisel ve tutarlı yanıtlar	Kişisel asistanın senin geçmiş tercihlerini bilmesi

Prompt Yapısı

Bir LLM ile konuşma, üç farklı rolden oluşan mesajların birleşmesiyle kurulur.



3. LLM Özelleştirme Yöntemleri

- **Prompt Engineering (Çok Düşük Maliyet):** Modeli değiştirmeden, sadece verilen talimatları iyileştirerek yönlendirme işlemidir.
- **RAG - Retrieval-Augmented Generation (Orta Maliyet):** Modelin bilmediği bilgiyi dış bir vektör veritabanından çekip cevaba eklemesi işlemidir.
- **Fine-Tuning (Yüksek Maliyet):** Modelin kendi verimizle (binlerce örnek) yeniden eğitilerek ağırlıklarının kalıcı olarak değiştirilmesidir.

4. Otonom Ajanlar (AI Agents) ve İleri Kavramlar

- **Reasoning (Akıl Yürütme):** Modelin karmaşık bir problemi tek adımda değil, ara adımlara bölerek (Chain-of-Thought) çözmesidir.
- **Agentic Loop:** Bir AI ajanının hedefe ulaşmak için sürekli tekrarladığı Gözlem (Perceive), Düşünme (Reason), Eylem (Act) ve Değerlendirme (Reflect) döngüsüdür.
- **MCP (Model Context Protocol):** LLM'lerin dış kaynaklara ve araçlara (API, veritabanı vb.) bağlanmasını sağlayan standart arayüzdür ("AI için USB-C" olarak da adlandırılır).

5. Kritik LLM Güvenlik Riskleri ve Saldırıları

- **Prompt Injection:** Kötü niyetli girdiler kullanılarak modelin sistem talimatlarının ezilmesi veya aşılmasıdır.
- **Jailbreak (Rol Oyunu):** Modele farklı bir rol ("Sen DAN'sın" gibi) yaptırılarak normalde reddedeceği zararlı içerikleri üretmesinin sağlanmasıdır.
- **Hassas Bilgi Sızıntısı:** Eğitim verisinde bulunan kişisel veya gizli verilerin (PII) model tarafından ifşa edilmesidir.
- **Indirect Injection (RAG üzerinden):** RAG sistemlerinde belgenin içine gizlenmiş zararlı komutların model tarafından talimat olarak algılanıp çalıştırılmasıdır.